

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ MÔ HÌNH XE TỰ HÀNH CỠ NHỎ ĐI THEO
LÀN, BIỂN BÁO VÀ NHẬN DIỆN TÍN HIỆU ĐÈN GIAO
THÔNG**

SVTH: LÊ ANH QUÂN

MSSV: 22119218

BÙI MINH PHƯƠNG

MSSV: 22119213

GVHD: ThS. Trần Thị Quỳnh Như

TP. Hồ Chí Minh - 07/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ MÔ HÌNH XE TỰ HÀNH CỠ NHỎ ĐI THEO
LÀN, BIỂN BÁO VÀ NHẬN DIỆN TÍN HIỆU ĐÈN GIAO
THÔNG**

SVTH: LÊ ANH QUÂN

MSSV: 22119218

BÙI MINH PHƯƠNG

MSSV: 22119213

GVHD: ThS. Trần Thị Quỳnh Như

TP. Hồ Chí Minh - 07/2026



CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM

Độc lập - Tự do - Hạnh phúc

---***---

TP. Hồ Chí Minh, ngày 23 tháng 06 năm 2026

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên: Lê Anh Quân MSSV: 22119218

Bùi Minh Phương MSSV: 22119213

Ngành: Công nghệ kỹ thuật máy tính **Lớp:** 221191A

Giảng viên hướng dẫn: Trần Thị Quỳnh Như

Ngày nhận đề tài:

Ngày nộp đề tài:

1. Tên đề tài: Thiết kế mô hình xe tự hành cỡ nhỏ đi theo làn, phát hiện biển báo và nhận diện tín hiệu giao thông
2. Các số liệu, tài liệu ban đầu:
 - Khảo sát các nghiên cứu trong và ngoài nước về mô hình xe tự hành cỡ nhỏ, thuật toán bám làn và phát hiện đối tượng bằng học sâu.
 - Tìm hiểu các nền tảng phần cứng nhúng và thư viện thị giác máy tính, học sâu phù hợp để triển khai trên thiết bị có tài nguyên hạn chế.
 - Đề xuất kiến trúc phần cứng - phần mềm phân tán giữa bộ xử lý trung tâm và vi điều khiển chấp hành.
 - Xây dựng mô hình xe, lắp ráp phần cứng, huấn luyện mô hình nhận diện và triển khai các thuật toán bám làn, phát hiện vật cản, nhận diện tín hiệu giao thông; chạy thử nghiệm và đánh giá trên sa hình..
3. Sản phẩm: Mô hình xe tự hành cỡ nhỏ có khả năng bám làn đường, phát hiện vật cản và nhận diện tín hiệu giao thông trên sa hình thử nghiệm

TRƯỞNG NGÀNH

GIẢNG VIÊN HƯỚNG DẪN

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Thiết kế hệ thống mô hình xe tự hành cơ nhô đi theo làn, biển báo và nhận diện đèn giao thông

Sinh viên: + Lê Anh Quân

MSSV: 22.119218

+ Bùi Minh Phương

MSSV: 22.119213

Hướng dẫn: Trần Thị Quyên Như

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề rõ ràng, mục tiêu cụ thể, đề tài có tính mới.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Phương pháp hợp lý, tin cậy, có phân tích đánh giá định tính.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá/ kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Kết quả phù hợp mục tiêu.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Kết luận phù hợp.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Bố cục trình bày hợp lý, rõ ràng, đúng mẫu.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Kỹ năng giải quyết vấn đề, làm việc nhóm.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Có thực hiện trích dẫn.

8. Đánh giá về sự trùng lặp của đề tài

Cần không định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Tiêu lệ trùng lặp 9.2

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

Lỗi đánh số công thức.

Sản phẩm chỉ chạy được trong điều kiện ánh sáng đèn trong phòng.

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Thái độ học tập nghiêm túc, nghiêm túc.

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Bảo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Báo cáo đạt yêu cầu của KLTN kỹ sư, được phép bảo vệ KLTN

Tp. HCM, ngày 15 tháng 6 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)

Trần Thị Quyên Như

* Giáo viên hướng dẫn có thể ghi tiếp vào mặt sau

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP
(Dùng cho giảng viên phản biện)

Đề tài: Thiết kế hệ thống xe tự hành cỡ nhỏ đi theo làn, biển báo và nhận diện tín hiệu đèn giao thông

Sinh viên: Lê Anh Quân

MSSV: 22119218

Bùi Minh Phương

22119213

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và tính ứng dụng [15]:

Đặt vấn đề rõ ràng, mục tiêu cụ thể^[5]; đề tài có tính mới, cấp thiết^[5]; đề tài có khả năng ứng dụng, tính sáng tạo^[5].

2. Phương pháp thực hiện/ phân tích/ thiết kế [25]:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết^[10]; có phân tích và đánh giá phù hợp^[10]; có tính mới và tính sáng tạo^[5].

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế [25]:

Phù hợp với mục tiêu^[10]; phân tích và đánh giá / kiểm thử thiết kế hợp lý^[10]; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy^[5].

4. Kết luận và đề xuất [10]:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn^[5]; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục^[5].

5. Hình thức trình bày, bố cục và chất lượng báo cáo [20]:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu^[5]; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt^[15].

6. Tài liệu trích dẫn [5]:

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

7. Đánh giá về sự trùng lặp của đề tài:

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

8. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*:

Câu hỏi sinh viên phải trả lời trước hội đồng*:

Kết luận:

Cho bảo vệ ☐

Không cho bảo vệ* ☐

Điểm tổng:

/100

*Trường hợp giảng viên phản biện không cho bảo vệ, vui lòng ghi rõ lý do:

TP. HCM, ngày tháng năm 2026
Người phản biện

* Giảng viên phản biện có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án, nhóm đã nhận được sự hướng dẫn tận tình của cô Trần Thị Quỳnh Như - từ giai đoạn lựa chọn kiến trúc phần cứng, xác định cách phân chia công việc giữa Raspberry Pi 4 và ESP32, đến những lần phải quay lại sửa cách trình bày cơ sở lý thuyết cho khớp với những gì hệ thống thực sự làm. Nhiều vướng mắc tưởng như chỉ là vấn đề kỹ thuật thuần túy - tốc độ xử lý khung hình không ổn định, mô hình YOLO nhận diện sai trong điều kiện thiếu sáng, tín hiệu lái bị giật khi mất dấu làn đường - đều được nhìn nhận rõ hơn nhờ những câu hỏi và góp ý của thầy/cô trong các buổi báo cáo tiến độ.

Nhóm cũng xin cảm ơn Khoa Điện - Điện tử, Trường Đại học Công nghệ Kỹ thuật TP.HCM, đã tạo điều kiện về phòng thực hành để nhóm có không gian dựng sơ đồ và chạy thử mô hình trong điều kiện gần với thực tế vận hành.

Đồ án được hoàn thành trong khoảng thời gian một học kỳ, với kinh nghiệm nghiên cứu còn nhiều hạn chế, nên chắc chắn còn những phần trình bày chưa thấu đáo, đặc biệt ở các đánh giá định lượng và những giới hạn của hệ thống mà nhóm chưa lường hết được.

Nhóm mong nhận được góp ý từ thầy/cô và hội đồng để nhìn nhận đầy đủ hơn những điều còn thiếu sót này.

LỜI CAM ĐOAN

Đồ án tốt nghiệp với đề tài "Thiết kế mô hình xe tự hành cỡ nhỏ đi theo làn, phát hiện biển báo và nhận diện tín hiệu giao thông" là kết quả tìm hiểu, thiết kế và thực nghiệm do nhóm thực hiện trực tiếp tiến hành, dưới sự hướng dẫn của cô Trần Thị Quỳnh Như.

Các số liệu, hình ảnh thực nghiệm và kết quả huấn luyện mô hình trình bày trong đồ án là dữ liệu thật thu được từ quá trình xây dựng và vận hành mô hình của nhóm, không sao chép từ bất kỳ đồ án hay công bố nào khác. Mọi tài liệu, công trình nghiên cứu được tham khảo trong quá trình thực hiện đều đã được liệt kê và trích dẫn đầy đủ trong phần Tài liệu tham khảo, theo đúng quy định của Trường.

Nhóm xin chịu hoàn toàn trách nhiệm trước Hội đồng và Nhà trường nếu phát hiện bất kỳ hành vi sao chép hoặc gian lận học thuật nào trong nội dung đã trình bày.

Đại diện nhóm thực hiện đồ án

(ký và ghi rõ họ tên)

TÓM TẮT

Tai nạn giao thông phần lớn xuất phát từ lỗi chủ quan của con người, trong khi việc thử nghiệm các thuật toán lái tự động trên xe thật lại đi kèm chi phí cảm biến lớn và rủi ro an toàn không thể bỏ qua. Đề án này tiếp cận bài toán theo hướng thực tế hơn: xây dựng một mô hình xe tự hành thu nhỏ, đủ để người thực hiện trực tiếp làm chủ toàn bộ chuỗi xử lý của một hệ thống hỗ trợ lái, từ cảm nhận môi trường đến ra quyết định và chấp hành.

Hệ thống được tổ chức theo kiến trúc phân tán hai tầng: một máy tính nhúng Raspberry Pi 4 đảm nhận toàn bộ khối xử lý thị giác và ra quyết định, kết nối một chiều qua UART tới vi điều khiển ESP32 chịu trách nhiệm vòng điều khiển động cơ thời gian thực. Khối bám làn khai thác chuỗi xử lý ảnh cổ điển - phân ngưỡng màu trong không gian HSV, dò biên Canny, biến đổi Hough xác suất trong vùng quan tâm hình thang - kết hợp một luật điều khiển phi tuyến có vùng chết để chuyển sai lệch quỹ đạo thành góc lái. Khối nhận thức ngữ nghĩa sử dụng mạng YOLOv8n, huấn luyện theo phương pháp học chuyển giao và triển khai suy luận qua định dạng ONNX trên CPU, để phát hiện sáu nhóm đối tượng giao thông trên sa hình thử nghiệm; kết quả nhận dạng được đưa vào một máy trạng thái hữu hạn quy định hành vi tương ứng của xe.

Mô hình hoàn chỉnh đã vận hành ổn định trên sa hình thử nghiệm trong nhà, bám làn mượt qua các đoạn cong, và đạt độ chính xác nhận dạng cao đối với cả sáu nhóm đối tượng trong điều kiện môi trường đã thiết kế. Kết quả đạt được cho thấy tính khả thi của việc phân tải xử lý nhận thức và chấp hành lên hai bộ xử lý chuyên biệt, chi phí thấp, làm cơ sở để mở rộng nghiên cứu sau này.

MỤC LỤC

LỜI CẢM ƠN	
LỜI CAM ĐOAN	
TÓM TẮT	
MỤC LỤC	
Chương 1. TỔNG QUAN VỀ XE TỰ HÀNH VÀ HỆ THỐNG HỖ TRỢ LÁI XE THÔNG MINH.....	1
1.1. Lý do chọn đề tài	1
1.2. Các nghiên cứu trong và ngoài nước	2
1.2.1. Tổng quan các nghiên cứu quốc tế về nhận diện và điều khiển thông minh	2
1.2.2. Tình hình nghiên cứu và phát triển mô hình xe tự hành trong nước	3
1.2.3. Đánh giá sự tiến hóa của các thuật toán nhận diện đối tượng (Từ YOLOv4 đến YOLOv8).....	4
1.2.4. Bài học kinh nghiệm và định hướng ứng dụng cho đề tài khóa luận	5
1.3. Mục tiêu nghiên cứu	6
1.4. Đối tượng và Phạm vi nghiên cứu	7
1.5. Phương pháp nghiên cứu	8
1.6. Ý nghĩa khoa học và thực tiễn	9
1.7. Bố cục của đề án.....	10
Chương 2. CƠ SỞ LÝ THUYẾT	11
2.1. Tổng quan về xe tự hành	11
2.1.1. Khái niệm và các cấp độ tự hành.....	11
2.1.2. Kiến trúc ba tầng: nhận thức - quyết định - chấp hành	12
2.2. Nền tảng phần cứng của hệ thống	12
2.2.1. Vi xử lý nhúng Raspberry Pi 4 Model B.....	12
2.2.2. Vi điều khiển ESP32	14
2.2.3. Cảm biến hình ảnh - camera OV5647 góc rộng.....	15
2.2.4. Mạch cầu H L298N và động cơ DC giảm tốc	16
2.2.5. Encoder quadrature và nguyên lý đo tốc độ quay	18
2.2.6. Động cơ servo và cơ cấu lái Ackermann.....	19
2.2.7. Nguồn điện và bộ chuyển đổi DC-DC kiểu buck.....	20
2.3. Kỹ thuật giao tiếp và điều khiển trong hệ thống nhúng	21

2.3.1. Giao tiếp nối tiếp UART	21
2.3.2. Điều chế độ rộng xung PWM và khối LEDC của ESP32	22
2.3.3. Ngắt ngoài và trình phục vụ ngắt	22
2.4. CƠ SỞ LÝ THUYẾT XỬ LÝ ẢNH SỐ	22
2.4.1. Không gian màu và biểu diễn ảnh số	23
2.4.1.1. Không gian màu RGB	23
2.4.1.2. Không gian màu HSV	24
2.4.2. Hiệu chỉnh chói sáng tuyến tính	25
2.4.3. Lọc làm mịn Gaussian	26
2.4.4. Phân ngưỡng thích nghi cục bộ	27
2.4.5. Các phép toán hình thái học	28
2.4.6. Phân tích vùng liên thông và lọc nhiễu theo diện tích	29
2.4.7. Hình học phối cảnh và vùng quan tâm ROI	30
2.4.8. Phân tích đường bao và nhận dạng vạch zebra crossing	31
2.4.9. Lý thuyết Lọc quán tính và Hàm phi tuyến vùng chết	32
2.4.9.1. Hàm phi tuyến vùng chết:.....	32
2.4.9.2. Bộ lọc Trung bình trượt hàm mũ:.....	33
2.5. PHÁT HIỆN ĐỐI TƯỢNG BẰNG HỌC SÂU	34
2.5.1. Mạng nơ-ron tích chập	34
2.5.1.1. Lớp tích chập	34
2.5.1.2. Hàm kích hoạt phi tuyến.....	35
2.5.1.3. Cơ chế giảm mẫu không gian	35
2.5.2. Bài toán phát hiện đối tượng và cơ chế phân loại - định vị đồng thời	36
2.5.3. Kiến trúc YOLOv8n	37
2.5.3.1. Backbone C2f và SPPF	37
2.5.3.2. Neck FPN + PAN	38
2.5.3.3. Đầu dự đoán tách biệt và Anchor-free	38
2.5.4. Huấn luyện mô hình: học chuyển giao và tăng cường dữ liệu	39
2.5.4.1. Học chuyển giao	39
2.5.4.2. Tăng cường dữ liệu.....	40
2.5.5. Các chỉ số đánh giá mô hình	40
2.5.5.1. Intersection over Union	40

2.5.5.2. Precision, Recall và F1-Score	40
2.5.5.3. Mean Average Precision.....	41
2.5.5.4. Thuật toán triệt phi cực đại - NMS.....	41
2.5.6. Tối ưu hóa suy luận trên thiết bị biên.....	42
2.6. Cơ sở lý thuyết điều khiển.....	43
2.6.1. Bộ điều khiển PID và dạng rời rạc	43
2.6.2. Feedforward và kỹ thuật chống tràn tích phân	44
2.7. Mô hình hóa và Ước lượng trạng thái quỹ đạo	44
2.7.1. Thuật toán nội suy điểm nhìn xa	44
2.7.2. Thuật toán tổng hợp quỹ đạo dựa trên Trung bình có trọng số.....	45
2.7.3. Mô hình toán học về Sai số lệch ngang.....	46
2.7.4. Đặc tính nhiễu dao động trong hệ thống thị giác máy tính	47
2.7.5. Kỹ thuật Lọc đột biến biên độ	47
2.7.6. Cơ sở toán học của luật điều khiển bám làn phi tuyến có vùng chết	48
2.8. Lập trình đa luồng trong hệ thống thị giác thời gian thực	49
2.8.1. Bài toán: các tác vụ có nhịp thời gian khác nhau	49
2.8.2. Mô hình sản xuất và tiêu thụ cùng các cơ chế đồng bộ hóa.....	49
2.8.3. Đa nhiệm thời gian thực trên vi điều khiển	50
Chương 3. THIẾT KẾ HỆ THỐNG XE TỰ HÀNH	52
3.1. Yêu cầu hệ thống	52
3.1.1. Yêu cầu chức năng	52
3.1.2. Yêu cầu phi chức năng	53
3.1.2.1. Hiệu năng thời gian thực:	53
3.1.2.2. Độ ổn định giao tiếp:	53
3.1.2.3. Khả năng mở rộng và bảo trì:	53
3.1.2.4. Tương thích phần cứng:	53
3.1.3. Yêu cầu an toàn và độ tin cậy.....	53
3.2. Kiến trúc tổng thể hệ thống	54
3.2.1. Phân tích và lựa chọn kiến trúc	54
3.2.2. Sơ đồ khối tổng thể.....	55
3.2.3. Luồng dữ liệu và giao tiếp giữa các khối	56
3.3. Thiết kế phần cứng	58

3.3.1. Sơ đồ kết nối tổng thể và phân bổ chân.....	58
3.3.2. Khối nguồn điện	60
3.3.3. Khối xử lý trung tâm - Raspberry Pi 4 Model B 4GB	61
3.3.4. Khối điều khiển chấp hành - ESP32.....	62
3.3.5. Khối cảm biến hình ảnh - Camera Module OV5647.....	63
3.3.6. Khối truyền động - L298N, Động cơ DC và Encoder.....	64
3.3.7. Khối cơ cấu lái - Servo MG955	65
3.4. THIẾT KẾ PHẦN MỀM.....	67
3.4.1. Tổng quan Kiến trúc Phần mềm và Cơ chế Đa luồng.....	67
3.4.1.1. Kiến trúc phân tán Master - Slave	67
3.4.1.2. Thiết kế đa luồng và quản lý tài nguyên	67
3.4.2. Khối Nhận thức Không gian: Pipeline Phát hiện Làn đường.....	69
3.4.2.1. Tiền xử lý: ảnh xám và lọc Gaussian	69
3.4.2.2. Tối ưu hóa không gian: Hình học thấu thị và Vùng quan tâm	70
3.4.2.3. Tách vạch: phân ngưỡng thích nghi và lọc nền.....	71
3.4.2.4. Quét cửa sổ trượt và ước lượng tâm làn	71
3.4.3. Khối Ước lượng Trạng thái và Lọc tín hiệu Thời gian	71
3.4.3.1. Ước lượng tâm làn đường và Độ lệch quỹ đạo	71
3.4.3.2. Xử lý nhiễu rung lắc và hiện tượng đứt gãy tín hiệu.....	72
3.4.3.3. Lọc tín hiệu bằng Trung bình trượt hàm mũ	73
3.4.4. Khối Nhận thức Ngữ nghĩa: Mạng nơ-ron Tích chập YOLO	73
3.4.4.1. Xây dựng bộ dữ liệu hạn dạng đối tượng giao thông.....	73
3.4.4.2. Cấu hình và quy trình huấn luyện	75
3.4.4.3. Bài toán phát hiện đối tượng trong môi trường giao thông.....	76
3.4.4.4. Kỹ thuật Tối ưu hóa đường ống Suy luận hệ thống thời gian thực.....	77
3.4.5. Khối Phân xử Hành vi và Lập kế hoạch.....	78
3.4.5.1. Mô hình Máy trạng thái hữu hạn.....	78
3.4.5.2. Cơ chế hợp nhất thông tin	79
3.4.5.3. Logic phân xử ưu tiên.....	80
3.4.6. Khối Điều khiển Động học cấp thấp và Giao tiếp.....	81
3.4.6.1. Điều khiển tốc độ vòng hở và định hướng vòng kín	81
3.4.6.2. Giao thức truyền thông UART và Kỹ thuật tối ưu hóa băng thông	82

3.4.6.3. Thiết kế cơ chế điều khiển chấp hành trên ESP32	82
Chương 4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	85
4.1. Môi trường và điều kiện thử nghiệm.....	85
4.1.1. Sơ hình thử nghiệm.....	85
4.1.2. Điều kiện ánh sáng	86
4.1.3. Cấu hình phần cứng và phần mềm khi thử nghiệm.....	87
4.2. Kết quả lắp ráp phần cứng hoàn chỉnh	88
4.3. Kết quả khối phát hiện làn đường	89
4.3.1. Quy trình xử lý ảnh phát hiện làn	89
4.3.2. Khử nhiễu trong ảnh nhị phân	90
4.3.3. Xác định độ lệch và bám làn qua đoạn thẳng, khúc cua	91
4.3.4. Cơ chế giữ hướng khi tạm mất làn	92
4.3.5. Bộ tham số cấu hình của khối phát hiện làn.....	92
4.3.6. Đánh giá định lượng khối phát hiện làn	93
4.4. Kết quả khối phát hiện biển báo và đèn tín hiệu	94
4.4.1. Phát hiện vạch qua đường (vạch ngựa vằn)	95
4.5. Kết quả khối chấp hành	96
4.5.1. Điều khiển góc lái servo theo cơ cấu Ackermann	96
4.5.2. Điều khiển tốc độ động cơ.....	97
4.5.3. Giao tiếp UART giữa Raspberry Pi và ESP32.....	97
4.6. Kết quả tích hợp và chạy thực tế	97
4.7. Đánh giá chung.....	100
Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	101
5.1. Kết luận.....	101
5.1.1. Những kết quả đã đạt được.....	101
5.1.2. Đối chiếu với mục tiêu đề ra	101
5.2. Hạn chế của đề tài.....	102
5.3. Hướng phát triển.....	102
5.4. Lời kết.....	103
TÀI LIỆU THAM KHẢO.....	104

DANH MỤC HÌNH

Hình 2.1. Vi xử lý Raspberry Pi 4 Model B	13
Hình 2.2. Sơ đồ chân Raspberrypi Pi 4 Model B	14
Hình 2.3. Module vi điều khiển ESP32 DevKitC [12].....	15
Hình 2.4. Module camera OV5647 và đặc trưng ảnh góc rộng [14].....	16
Hình 2.5. Nguyên lý mạch cầu H và module công suất L298N [15]	18
Hình 2.6. Tín hiệu encoder quadrature và nguyên lý giải mã $\times 4$	19
Hình 2.7. Xung PWM servo	20
Hình 2.8. Nguồn pin lipo 12V	21
Hình 2.9. Không gian màu RGB [21].....	23
Hình 2.10. Không gian màu HSV [21].....	25
Hình 2.11. Ảnh trước (trái) và sau (phải) khi đi qua hàm hiệu chỉnh ánh sáng tuyến tính với $\alpha = 2,2$ và $\beta = 50$ [23]	25
Hình 2.12 Nhân Gaussian 7×7 (trái), biên độ cắt ngang (giữa) và tác dụng lọc làm mịn nhiễu ngẫu nhiên trong tín hiệu 1D (phải). $\sigma \approx 1,4$ tương ứng với phạm vi ảnh hưởng ~ 3 pixel mỗi chiều	26
Hình 2.13. Ảnh với các kernel Gaussian khác nhau [24].....	27
Hình 2.14. Kết quả phân ngưỡng thích nghi cục bộ [40]	28
Hình 2.15. Biểu diễn trực quan hiệu ứng của các phép toán hình thái học trong việc xử lý nhiễu biên và lấp đầy các khoảng đứt gãy bên trong đối tượng [26]	29
Hình 2.16. Hiện tượng hội tụ của các đường thẳng song song qua phép chiếu phối cảnh và mặt nạ giới hạn vùng quan tâm	30
Hình 2.17. Minh họa đường bao gốc của đối tượng (trái) và đa giác lồi bao quanh nó (giữa) dùng làm cơ sở tính toán đặc trưng mật độ lấp đầy.....	32
Hình 2.18. Đặc tuyến vào-ra của hàm phi tuyến vùng chết (Khu vực III) kết hợp các giới hạn bão hòa.....	33
Hình 2.19. So sánh tín hiệu chứa dao động nhiễu (xanh) và tín hiệu đầu ra sau khi đi qua bộ lọc quán tính EMA (cam).....	34
Hình 2.20. Đồ thị so sánh đặc tuyến của hàm kích hoạt ReLU và SiLU	35
Hình 2.21. Kiến trúc mạng CNN điển hình.....	36
Hình 2.22. So sánh kiến trúc tổng quát của mô hình phát hiện đối tượng một giai đoạn (YOLO) và hai giai đoạn (R-CNN).....	37
Hình 2.23. Lưu đồ cấu trúc mạng YOLOv8.....	38
Hình 2.24. So sánh cấu trúc Coupled Head (bên trái) của các thế hệ trước và Decoupled Head (bên phải) được ứng dụng trên YOLOv8.....	39
Hình 2.25. Phân loại các kết quả dự đoán (TP, FP, FN) dựa trên ngưỡng IoU trong bài toán phát hiện đối tượng	41
Hình 2.26. (a) Minh họa IoU giữa hộp nhãn thật (xanh lá) và hộp dự đoán (xanh dương); vùng cam là giao nhau. (b) Nhiều hộp chồng lấp trước NMS. (c) Kết quả sau NMS ($\text{IoU_NMS} = 0,45$): chỉ giữ hộp điểm cao nhất.....	42
Hình 2.27. Cấu trúc bộ điều khiển PID vòng kín	44

Hình 2.28. Mô hình đa luồng producer - consumer của phần mềm điều khiển	50
Hình 3.1. Sơ đồ khối tổng thể kiến trúc hệ thống xe tự hành.....	56
Hình 3.2. Sơ đồ luồng dữ liệu và giao tiếp giữa các thành phần.....	58
Hình 3.3. Sơ đồ nguyên lý đầy đủ hệ thống phần cứng xe tự hành.....	60
Hình 3.4. Sơ đồ khối tổng thể.....	60
Hình 3.5. Raspberry Pi 4 Model B , bộ xử lý trung tâm của hệ thống.....	62
Hình 3.6. Vị trí gắn Camera Module OV5647 trên mô hình xe và ví dụ frame thu được .	64
Hình 3.7. Servo MG955 được sử dụng để điều khiển cơ cấu lái	65
Hình 3.8. Hình ảnh thực tế mô hình xe tự hành hoàn chỉnh.....	66
Hình 3.9. Kiến trúc phân tán giữa bộ xử lý trung tâm và bộ điều khiển thực thi.....	67
Hình 3.10. Sơ đồ cơ chế đồng bộ hóa đa luồng giữa các khối xử lý.....	68
Hình 3.11. Pipeline phát hiện làn đường sử dụng xử lý ảnh truyền thống.....	69
Hình 3.12. Vùng quan tâm được triển khai	71
Hình 3.13. Pipeline Tiền xử lý Letterbox và Hậu xử lý Tensor.....	78
Hình 3.14. Sơ đồ chuyển đổi trạng thái của xe	79
Hình 3.15. Lưu đồ thuật toán ra quyết định	81
Hình 3.16. Sơ đồ vòng lặp điều khiển phản hồi kín	84
Hình 4.1. Sa hình thử nghiệm với các đoạn thẳng và khúc cua	86
Hình 4.2. Môi trường thử nghiệm trong phòng.....	87
Hình 4.3. Mô hình xe tự hành sau khi lắp ráp hoàn chỉnh	88
Hình 4.4. Kết quả từng bước trong quy trình phát hiện làn đường	90
Hình 4.5. Hiệu quả bước khử nhiễu trong ảnh nhị phân	91
Hình 4.6. Kết quả bám làn trên đoạn thẳng và trong khúc cua	92
Hình 4.7. Giao diện quan sát trực tuyến khung hình xử lý phục vụ gỡ lỗi	94
Hình 4.8. Kết quả phát hiện biển báo và đèn tín hiệu bằng YOLOv8n	95
Hình 4.9. Kết quả phát hiện vạch qua đường	96
Hình 4.10. Một số khoảnh khắc xe vận hành tích hợp trên sa hình	99

DANH MỤC BẢNG

Bảng 1.1. Các công trình nghiên cứu tiêu biểu về xe tự hành giai đoạn 2024-2025	3
Bảng 2.1. Sáu cấp độ tự hành theo tiêu chuẩn SAE J3016 [9].....	11
Bảng 2.2. Thông số kỹ thuật chính của Raspberry Pi 4 Model B	13
Bảng 2.3. Thông số kỹ thuật chính của vi điều khiển ESP32 [12].....	15
Bảng 2.4. Bảng chân trị điều khiển một kênh cầu H L298N [15].....	17
Bảng 2.5. Vai trò của ba khâu trong bộ điều khiển PID.....	43
Bảng 3.1. So sánh hai phương án kiến trúc hệ thống điều khiển	55
Bảng 3.2. Bảng phân bổ chân kết nối toàn hệ thống	59
Bảng 3.3. Phân bổ timer phần cứng ESP32 trong hệ thống	63
Bảng 3.4. Bảng chân trị điều khiển chiều quay động cơ qua L298N.....	65
Bảng 3.5. Phân bổ tài nguyên CPU và thời gian thực thi kỳ vọng cho từng luồng	68
Bảng 3.6. Tham số cấu hình giới hạn Vùng quan tâm hình thang	70
Bảng 3.7. Tham số cấu hình hệ thống không gian và chuẩn hóa sai số	72
Bảng 3.8. Tham số cấu hình khâu lọc đột biến động lực học	73
Bảng 3.9. Tham số cấu hình khâu lọc thông thấp thời gian thực (EMA)	73
Bảng 3.10. Phân bổ bộ dữ liệu nhận dạng đối tượng giao thông	74
Bảng 3.11. Siêu tham số huấn luyện YOLOv8n và lý do lựa chọn	75
Bảng 3.12. Danh sách lớp đối tượng mục tiêu và ngưỡng vận hành	77
Bảng 3.13. Cấu trúc khung truyền dữ liệu qua UART	82
Bảng 3.14. Tham số cấu hình bộ điều khiển và giới hạn chấp hành	84
Bảng 4.1. Cấu hình phần cứng và phần mềm trong quá trình thử nghiệm.....	87
Bảng 4.2. Các thành phần phần cứng chính trên mô hình hoàn chỉnh	89
Bảng 4.3. Bộ tham số chính của khối phát hiện làn đường.....	92
Bảng 4.4. Đánh giá định tính khả năng bám làn theo từng tình huống.....	93
Bảng 4.5. Sáu lớp đối tượng mà mô hình YOLOv8n nhận diện	94
Bảng 4.6. Kết quả các kịch bản vận hành tích hợp trên sa hình.....	98
Bảng 4.7. Tổng hợp các vấn đề kỹ thuật và giải pháp khắc phục	99
Bảng 5.1. Đối chiếu mức độ hoàn thành so với mục tiêu đề ra.....	101

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Mô tả
AI	Artificial Intelligence
CNN	Convolutional Neural Network
FPS	Frames Per Second
YUV	Luminance and Chrominance Components
YOLO	You Only Look Once
ROI	Region of Interest
UART	Universal Asynchronous Receiver/Transmitter
ONNX	Open Neural Network Exchange
DNN	Deep Neural Network
PWM	Pulse Width Modulation
PID	Proportional–Integral–Derivative
ISR	Interrupt Service Routine
IIR	Infinite Impulse Response
R-CNN	Region-based Convolutional Neural Network
C2f	Cross Stage Partial with Two Convolutions Fast
FPN	Feature Pyramid Network
PAN	Path Aggregation Network
SPPF	Spatial Pyramid Pooling Fast
IoU	Intersection over Union
CIoU	Complete Intersection over Union
DFL	Distribution Focal Loss
NMS	Non-Maximum Suppression
FSM	Finite State Machine

Chương 1. TỔNG QUAN VỀ XE TỰ HÀNH VÀ HỆ THỐNG HỖ TRỢ LÁI XE THÔNG MINH

1.1. Lý do chọn đề tài

Trong bối cảnh Cách mạng Công nghiệp lần thứ tư, sự hội tụ của các công nghệ lõi như trí tuệ nhân tạo, Internet vạn vật, dữ liệu lớn và điện toán đám mây đang định hình lại phương thức vận hành của nhiều lĩnh vực kinh tế - xã hội. Đối với giao thông vận tải, công nghệ xe tự hành không chỉ là một xu hướng công nghệ mà còn là một giải pháp được kỳ vọng cho bài toán an toàn giao thông trên phạm vi toàn cầu. Theo số liệu của Tổ chức Y tế Thế giới, tai nạn giao thông cướp đi sinh mạng của khoảng 1,35 triệu người mỗi năm, trong đó hơn 90% số vụ xuất phát từ lỗi chủ quan của con người như mất tập trung, mệt mỏi, sử dụng chất kích thích hoặc phản xạ chậm trước tình huống bất ngờ [1]. Với khả năng quan sát liên tục và thời gian xử lý tính bằng mili-giây, hệ thống tự lái được nhiều tập đoàn công nghệ như Google, Tesla, Baidu hay NVIDIA xem là một hướng đi khả thi để giảm thiểu các tai nạn có nguyên nhân từ con người.

Tuy tiềm năng ứng dụng lớn, việc nghiên cứu và thử nghiệm xe tự hành trên thực địa đối diện với nhiều rào cản. Rào cản đầu tiên là chi phí: một phương tiện tự hành hoàn chỉnh đòi hỏi hàng loạt cảm biến có giá thành cao như LiDAR 360 độ, radar tầm xa, cùng hệ thống máy tính chuyên dụng hiệu năng cao, khiến tổng chi phí dễ vượt mức trăm nghìn USD - vượt xa khả năng tiếp cận của các nhóm nghiên cứu hàn lâm hay cộng đồng sinh viên. Bên cạnh đó, việc thử nghiệm trực tiếp các thuật toán điều khiển chưa hoàn thiện trên xe thật tiềm ẩn rủi ro về tính mạng và tài sản, nên pháp luật ở hầu hết quốc gia đều thắt chặt việc cấp phép thử nghiệm xe tự lái trên đường bộ công cộng. Vì vậy, việc sử dụng các mô hình xe tự hành cỡ nhỏ với tỉ lệ thu nhỏ 1:10 hay 1:16 đang trở thành một hướng tiếp cận phổ biến trong đào tạo và nghiên cứu - phát triển. Các nền tảng như MIT Racecar hay AWS DeepRacer đã cho thấy mô hình thu nhỏ có thể mô phỏng khá đầy đủ những thách thức của xe tự hành thực tế, từ xử lý ảnh camera, điều khiển động lực học đến định vị và né tránh chướng ngại vật [2] [3].

Tại Việt Nam, sự quan tâm dành cho lĩnh vực robotics và hệ thống tự hành đang tăng lên, nhưng các giải pháp mang tính hệ thống cho xe thu nhỏ vẫn còn một số điểm nghẽn. Phần lớn đồ án của sinh viên thường rơi vào một trong hai hướng: hoặc quá đơn giản khi

chỉ dùng vi điều khiển cơ bản như Arduino với vài cảm biến quang để bám vạch đen, chưa thể hiện được bản chất của xử lý ảnh hiện đại; hoặc dồn toàn bộ tác vụ tính toán nặng như nhận diện bằng AI và điều khiển thời gian thực lên một máy tính nhúng duy nhất. Việc không phân chia kiến trúc hợp lý trong trường hợp thứ hai thường dẫn đến độ trễ đáng kể trong việc ra quyết định điều khiển, làm giảm độ ổn định của hệ thống.

Từ những quan sát đó, đề tài "Xây dựng mô hình hỗ trợ lái thông minh cho xe tự hành cỡ nhỏ: bám làn, và nhận diện tín hiệu giao thông" được lựa chọn nhằm xây dựng một kiến trúc hệ thống nhúng phân tán rõ rệt: Raspberry Pi 4 đóng vai trò bộ xử lý trung tâm cho thị giác máy tính và trí tuệ nhân tạo, còn vi điều khiển ESP32 đảm nhiệm vòng điều khiển cơ cấu chấp hành theo thời gian thực. Hướng tiếp cận này giúp người thực hiện trực tiếp làm chủ các thuật toán lõi của một hệ thống hỗ trợ lái xe ở quy mô thu nhỏ, đồng thời đề xuất một kiến trúc phân tải hợp lý giữa nhận thức - lập kế hoạch và điều khiển trên nền tảng phần cứng có chi phí thấp.

1.2. Các nghiên cứu trong và ngoài nước

1.2.1. Tổng quan các nghiên cứu quốc tế về nhận diện và điều khiển thông minh

Việc kế thừa các thành tựu nghiên cứu quốc tế là nền tảng để giải quyết bài toán tối ưu hóa thuật toán trên các nền tảng phần cứng có tài nguyên hạn chế. Đối với các hệ thống xe tự hành cấp độ prototype, thách thức không chỉ nằm ở độ chính xác của mô hình học sâu mà còn ở việc cân bằng giữa tốc độ tính toán và mức tiêu thụ năng lượng. Các nghiên cứu quốc tế gần đây có xu hướng chuyển từ các hệ thống máy chủ GPU sang các kiến trúc mạng nơ-ron tích chập lightweight được tinh chỉnh cho kiến trúc ARM như Raspberry Pi. Việc khảo sát các phương pháp tiếp cận, từ nhận diện làn đường bằng OpenCV truyền thống đến các mô hình End-to-End, cung cấp tham chiếu hữu ích cho việc xây dựng hệ thống nhúng thời gian thực.

Bảng 1.1 đối chiếu một số công trình nghiên cứu tiêu biểu được công bố trong giai đoạn 2024-2025:

Bảng 1.1. Các công trình nghiên cứu tiêu biểu về xe tự hành giai đoạn 2024-2025

Tác giả/Năm	Phương pháp cốt lõi	Kết quả/Đóng góp chính
Yashraj S. Dube và cộng sự (2025) [4]	Kết hợp YOLOv4-tiny (tối ưu cho CPU) và YOLOv5; OpenCV cho nhận diện làn đường; CNN phân loại đèn giao thông.	Triển khai trên Raspberry Pi 3B+, đạt 8.2 FPS cho nhận diện biển báo. Tích hợp cảm biến siêu âm và dashboard Streamlit để giám sát thời gian thực.
Binaya Dhakal và cộng sự (2025) [5]	Kết hợp YOLOv8 để phát hiện vật thể và mô hình End-to-End cho dự đoán góc lái.	Sử dụng Raspberry Pi 4 (2GB). Chuyển đổi sang định dạng YUV cho steering và RGB cho detection để giảm tải tính toán, đạt khả năng suy luận gần thời gian thực.
Khaldaa Alawaji và cộng sự (2024) [6]	Học đa nhiệm kết hợp YOLOv7 làm module trích xuất vùng đặc trưng.	Đạt độ chính xác 99,07%. YOLOv7 đóng vai trò bộ tiền xử lý trước khi thực hiện phân loại đa nhiệm tại Riyadh.

Một xu hướng đáng chú ý trong nghiên cứu của Alawaji (2024) là sự chuyển dịch từ Single-Task sang Multi-Task Learning. Cơ chế chia sẻ biểu diễn giữa các nhiệm vụ liên quan - như nhận diện biển báo và đèn giao thông - giúp giảm bớt vấn đề thừa dữ liệu và cho phép các nền tảng CPU hạn chế thực hiện đồng thời nhiều tác vụ với chi phí năng lượng gần tương đương một tác vụ đơn lẻ. Việc dùng YOLOv7 làm mô-đun ROI giúp cô lập vùng dữ liệu quan trọng trước khi phân loại, góp phần vào độ chính xác 99,07% ghi nhận được.

Những kết quả về kiến trúc mạng và chiến lược xử lý dữ liệu này là tham chiếu hữu ích cho việc ứng dụng và bản địa hóa mô hình xe tự hành trong bối cảnh nghiên cứu tại Việt Nam.

1.2.2. Tình hình nghiên cứu và phát triển mô hình xe tự hành trong nước

Tại Việt Nam, các trường đại học kỹ thuật hàng đầu, tiêu biểu là Trường Đại học Sư phạm Kỹ thuật TP.HCM, đã bắt đầu tiếp cận và làm chủ các công nghệ xe tự hành thông qua các đề tài nghiên cứu. Sự phát triển này tập trung vào việc thu hẹp khoảng cách giữa lý thuyết AI quốc tế và thực tiễn hạ tầng nhúng nội địa.

Đáng chú ý là đồ án tốt nghiệp của Văn Huy Du và Trần Văn Đạt (2024) với đề tài "Nghiên cứu và phát triển mô hình xe tự hành ứng dụng AI" [7]. Các tác giả đã thành công trong việc tích hợp các thuật toán thị giác máy tính vào môi trường thực nghiệm, cho thấy khả năng làm chủ các chuỗi xử lý từ thu nhận hình ảnh đến điều khiển chấp hành. Tuy nhiên, qua phân tích sâu, có thể thấy các nghiên cứu trong nước thường tập trung vào 3 hướng tiếp cận chủ đạo:

Tối ưu hóa chi phí phần cứng: Tận dụng các linh kiện phổ thông như động cơ BO, mạch cầu H L298N hoặc driver MDD20Amp kết hợp với Raspberry Pi/ESP32 để xây dựng các mô hình prototype với ngân sách thấp.

Thích nghi hạ tầng đặc thù: Tinh chỉnh tập dữ liệu để nhận diện hệ thống biển báo và vạch kẻ đường theo quy chuẩn giao thông đường bộ Việt Nam [8].

Tích hợp hệ thống nhúng thời gian thực: Tập trung vào việc giảm độ trễ trong quá trình suy luận trên CPU để điều khiển xe ổn định ở tốc độ thấp trong môi trường mô phỏng.

Đánh giá khoảng cách: các nghiên cứu trong nước như của Văn Huy Du và Trần Văn Đạt (2024) đã đạt một số kết quả về mô hình vật lý ổn định, nhưng vẫn còn hạn chế về độ trễ suy luận khi chuyển từ môi trường mô phỏng sang các kịch bản thực tế phức tạp hơn. Các kỹ thuật như học đa nhiệm hay tối ưu hóa sâu cho suy luận trên CPU vẫn chưa phổ biến, khiến hiệu suất hệ thống bị ảnh hưởng khi đối mặt với các tác vụ tính toán song song cường độ cao.

1.2.3. Đánh giá sự tiến hóa của các thuật toán nhận diện đối tượng (Từ YOLOv4 đến YOLOv8)

Trong thị giác máy tính cho xe tự hành, họ thuật toán YOLO đã trở thành một lựa chọn phổ biến nhờ kiến trúc một giai đoạn, cho phép tối ưu đồng thời tốc độ và độ chính xác. Sự khác biệt giữa các phiên bản phản ánh các hướng cải tiến khác nhau cho hệ thống nhúng:

- YOLOv4-tiny (Dube và cộng sự, 2025) [4]: phiên bản rút gọn dựa trên CSPDarknet53, cho tốc độ phản hồi ổn định trên Raspberry Pi 3B+, phù hợp cho hệ thống có tài nguyên CPU rất hạn chế.

- YOLOv7 (Alawaji và cộng sự, 2024) [6]: cải tiến cấu trúc backbone E-ELAN, cho phép trích xuất ROI hiệu quả; trong nghiên cứu của Alawaji, YOLOv7 đóng vai trò bộ tiền xử lý trước khi thực hiện phân loại đa nhiệm.
- YOLOv8 (Dhakal và cộng sự, 2025) [5]: chuyển sang kiến trúc anchor-free, dự đoán trực tiếp tâm vật thể thay vì phụ thuộc các khung neo định trước, giúp xử lý tốt hơn các đối tượng có tỉ lệ khung hình biến thiên và cải thiện tốc độ suy luận trên Raspberry Pi 4.

Sự chuyển dịch từ kiến trúc dựa trên khung neo sang anchor-free, cùng với việc tích hợp học đa nhiệm, đã giúp các mô hình phát hiện đối tượng triển khai khả thi hơn trên thiết bị không có GPU chuyên dụng.

1.2.4. Bài học kinh nghiệm và định hướng ứng dụng cho đề tài khóa luận

Việc khảo sát các nghiên cứu trên định hình một số lựa chọn kỹ thuật cụ thể cho khóa luận này, đồng thời cũng cho thấy những điểm cần điều chỉnh để phù hợp với phạm vi và tài nguyên phần cứng thực tế của đề tài.

- Lựa chọn phần cứng và kiến trúc phân tán: Raspberry Pi 4 được chọn làm bộ xử lý trung tâm nhờ lợi thế về băng thông dữ liệu khi chạy các mô hình YOLO thế hệ mới. Khác với một số nghiên cứu dồn toàn bộ xử lý lên một board duy nhất, đề tài tách riêng tầng nhận thức - lập kế hoạch khỏi tầng chấp hành thời gian thực, giao tiếp một chiều qua UART, nhằm tránh tình trạng vòng điều khiển động cơ bị nghẽn bởi thời gian xử lý không cố định của suy luận AI.
- Lựa chọn kiến trúc xử lý ảnh phân tầng: thay vì hướng đến một mô hình học sâu End-to-End như một số nghiên cứu quốc tế, đề tài giữ kiến trúc phân tầng: băm lán bằng xử lý ảnh cấu trúc cổ điển, phát hiện đối tượng bằng mạng YOLO riêng, và phân xử hành vi bằng máy trạng thái hữu hạn. Lựa chọn này phù hợp hơn với quy mô dữ liệu tự thu thập và tài nguyên tính toán của Raspberry Pi 4.
- Tối ưu hóa suy luận cho CPU nhúng: một bài học kỹ thuật quan trọng là việc chuyển mô hình đã huấn luyện sang đồ thị tính toán tĩnh ONNX và nạp bằng mô-đun DNN của OpenCV, thay vì phụ thuộc trực tiếp vào môi trường huấn luyện ban đầu. Cách làm này loại bỏ các phụ thuộc động trong đồ thị tính toán và cho

phép suy luận hoàn toàn trên CPU của Raspberry Pi mà không cần cài đặt các thư viện huấn luyện công kênh.

- Chiến lược cảm biến: do giới hạn về chi phí và độ phức tạp tích hợp, đề tài không trang bị cảm biến khoảng cách chuyên dụng như một số nghiên cứu đối chiếu ở Bảng 1.1. Thay vào đó, nhận thức về vật cản và tình huống giao thông được thực hiện hoàn toàn qua camera quang học duy nhất, kết hợp với các cơ chế dự phòng an toàn ở tầng phần mềm để bù trừ cho việc không có cảm biến đo khoảng cách độc lập. Đây là một đánh đổi được ghi nhận rõ trong phạm vi nghiên cứu ở Mục 1.4.

Khóa luận này kế thừa định hướng kiến trúc YOLO và OpenCV từ các nghiên cứu đã khảo sát, đồng thời tập trung tối ưu hóa tài nguyên phần cứng hạn chế để xây dựng một hệ thống xe tự hành có khả năng vận hành thực tế trên sa hình thử nghiệm.

1.3. Mục tiêu nghiên cứu

Mục tiêu bao trùm của đề tài là hoàn thiện từ khâu thiết kế, lắp ráp phần cứng đến xây dựng và triển khai các thuật toán điều khiển - nhận thức, nhằm tạo ra một phương tiện tự hành thu nhỏ có khả năng tự định hướng an toàn trên sa hình giả lập, tương đương cấp độ tự động hóa một phần (Level 2) theo tiêu chuẩn SAE J3016 [9]. Để hiện thực hóa mục tiêu tổng quát này, nội dung nghiên cứu được triển khai theo ba trụ cột chính:

Về hệ thống vật lý: Hoàn thiện cấu hình xe truyền động cầu sau với cơ cấu lái bằng động cơ servo; thiết lập kênh giao tiếp UART ổn định theo kiến trúc phân tán Master - Slave giữa Raspberry Pi 4 và ESP32. Đồng thời, giải quyết bài toán cấp nguồn cách ly giữa hệ thống công suất động cơ và các board mạch logic, tránh nhiễu và sụt áp ảnh hưởng đến tín hiệu điều khiển.

Về thuật toán điều khiển: Xây dựng và liên kết thành công ba mô-đun thành phần, ánh xạ trực tiếp lên kiến trúc phần mềm đa luồng của hệ thống:

- Mô-đun bám làn: trích xuất quỹ đạo làn đường bằng chuỗi xử lý ảnh cấu trúc - chuyển ảnh xám và lọc Gauss, phân ngưỡng thích nghi cục bộ để tách vạch, lọc nền và phân tích vùng liên thông để khử nhiễu, rồi quét dải ngang theo cửa sổ trượt trong vùng quan tâm hình thang. Sai lệch quỹ đạo ngang được chuyển hóa

thành góc lái qua một hàm truyền phi tuyến có vùng chết, sau đó làm mượt bằng bộ lọc trung bình trượt hàm mũ trước khi truyền xuống cơ cấu chấp hành.

- Mô-đun nhận thức ngữ nghĩa: sử dụng mạng YOLOv8n để phát hiện và phân loại sáu nhóm đối tượng giao thông trong môi trường thử nghiệm - biển báo dừng, biển cấm đi ngược chiều, đèn đỏ, đèn vàng, đèn xanh và người đi bộ. Kết quả nhận dạng là đầu vào cho một máy trạng thái hữu hạn quy định hành vi tương ứng cho xe: dừng có thời hạn, dừng vĩnh viễn, giảm tốc hoặc vận hành bình thường.
- Mô-đun điều khiển động học cấp thấp: thực thi trên ESP32 ở chu kỳ cố định 50 Hz, bao gồm bộ điều khiển PI cho đồng bộ tốc độ hai bánh sau dựa trên phản hồi từ cảm biến tốc độ, kết hợp vi sai điện tử khi vào cua và cơ chế giám sát mất kết nối để tự động đưa xe về trạng thái dừng an toàn khi mất tín hiệu điều khiển.

Về tiêu chuẩn hiệu suất: Hệ thống được thiết kế để duy trì các chỉ tiêu thời gian thực sau, phù hợp với năng lực tính toán của Raspberry Pi 4 không có đơn vị xử lý đồ họa chuyên dụng:

- Luồng xử lý ảnh bám làn duy trì tần số khoảng 15-20 khung hình/giây;
- Luồng nhận dạng đối tượng bằng YOLOv8n vận hành bất đồng bộ ở tần số thấp hơn, không làm chậm luồng bám làn nhờ kiến trúc đa luồng độc lập;
- Luồng hợp nhất và truyền lệnh điều khiển hoạt động ở tần số 20 Hz; vòng điều khiển động học trên ESP32 hoạt động ở tần số cố định 50 Hz;
- Ngưỡng tin cậy tối thiểu cho một phát hiện đối tượng hợp lệ là 0,25, kết hợp ngưỡng diện tích khung bao tối thiểu nhằm loại bỏ các phát hiện giả ở khoảng cách quá xa.

Phương tiện hoàn chỉnh có kích thước thu nhỏ, phù hợp vận hành trên sa hình mô phỏng trong nhà, hướng đến năng lực tự định hướng an toàn tương đương cấp độ tự lái tự động một phần - Level 2 theo chuẩn SAE đã đề cập ở trên.

1.4. Đối tượng và Phạm vi nghiên cứu

Đối tượng nghiên cứu của đề án bao gồm hai nhóm: thiết bị vật lý và tri thức phần mềm cốt lõi. Đối với thiết bị vật lý, nghiên cứu khai thác năng lực của máy tính bo mạch đơn Raspberry Pi 4 chạy hệ điều hành Linux, kết hợp với vi điều khiển ESP32 cho vòng điều khiển thời gian thực. Hệ thống cảm biến trung tâm bao gồm một camera quang học

duy nhất phục vụ cho cả hai nhánh nhận thức, cùng cảm biến tốc độ kiểu encoder gắn trên bánh xe phục vụ vòng phản hồi tốc độ. Đề tài không trang bị cảm biến đo khoảng cách chuyên dụng như siêu âm, LiDAR hay radar; mọi nhận thức về vật cản và tín hiệu giao thông được thực hiện hoàn toàn bằng thị giác máy tính.

Đối với tri thức phần mềm, đề tài tập trung làm chủ thư viện thị giác máy tính mã nguồn mở OpenCV cho chuỗi xử lý ảnh cổ điển (ảnh xám, lọc Gaussian, phân ngưỡng thích nghi, phân tích vùng liên thông, quét cửa sổ trượt), đi sâu vào cấu trúc mạng nơ-ron tích chập và kiến trúc YOLOv8n - biến thể nhỏ nhất của họ YOLOv8, được huấn luyện theo phương pháp học chuyển giao từ trọng số COCO và chuyển đổi sang định dạng ONNX để suy luận trên CPU qua mô-đun DNN của OpenCV. Đề tài cũng khai thác lý thuyết điều khiển tự động cho khâu đồng bộ tốc độ động cơ với bộ điều khiển PI với khâu bù truyền thẳng và vi sai điện tử và lý thuyết xử lý tín hiệu số cho khâu làm mượt tín hiệu lái với bộ lọc trung bình trượt hàm mũ.

Nhằm đảm bảo tính khả thi trong khung thời gian một học kỳ và bám sát ngân sách sinh viên, phạm vi nghiên cứu được giới hạn rõ ràng như sau. Mô hình xe được thiết kế để hoạt động trong môi trường sa hình tự dựng trong nhà, dưới điều kiện đèn chiếu sáng nhân tạo ổn định, tránh các nguồn sáng chói mạnh hoặc khu vực đổ bóng có độ tương phản quá lớn. Bề mặt đường đua được mô phỏng với độ phẳng và ma sát không đổi, giới hạn bởi các đường kẻ viền nổi bật trên nền tối của sa hình.

Khả năng tín hiệu giao thông của hệ thống giới hạn trong sáu nhóm đối tượng tĩnh đã huấn luyện cho mạng YOLOv8n gồm biển báo dừng, biển cấm đi ngược chiều, đèn đỏ, đèn vàng, đèn xanh, người đi bộ, không xử lý các vật cản hay hành vi chuyển động bất quy tắc nằm ngoài tập nhãn này. Phạm vi nhận diện biển báo và đèn tín hiệu được giới hạn ở khoảng cách tiệm cận trực diện, phù hợp với góc nhìn và độ phân giải của camera gắn trên xe. Không gian bài toán cũng loại trừ hoàn toàn các yêu cầu tích hợp định vị vệ tinh GPS hoặc tái tạo bản đồ phục vụ di chuyển tự do trên diện rộng.

1.5. Phương pháp nghiên cứu

Để thực hiện đề tài và giải quyết các mục tiêu đã đề ra, quy trình nghiên cứu được triển khai theo các giai đoạn từ lý thuyết đến thực nghiệm thực địa.

Giai đoạn đầu là tổng hợp và nghiên cứu tài liệu, khảo sát các công trình khoa học và báo cáo kỹ thuật về xe tự hành cũng như các kiến trúc mạng học sâu phù hợp với phần cứng nhúng. Giai đoạn này xây dựng nền tảng lý thuyết về xử lý ảnh, hệ thống nhúng và lý thuyết điều khiển, đồng thời định hình phương pháp triển khai mô hình học sâu trên tài nguyên phần cứng hạn chế. Tiếp theo là giai đoạn phân tích và thiết kế hệ thống, trong đó các yêu cầu về tốc độ đáp ứng, khả năng xử lý thời gian thực và kiến trúc phân tán Master - Slave được cụ thể hóa thành sơ đồ khối, sơ đồ nguyên lý và lưu đồ giải thuật cho từng mô-đun chức năng.

Sau khi hoàn tất thiết kế, đề tài chuyển sang giai đoạn xây dựng và triển khai thực tế: tích hợp phần cứng mô hình, thiết lập môi trường lập trình trên Raspberry Pi 4 và ESP32, thu thập dữ liệu ảnh từ camera trên xe, huấn luyện mạng YOLOv8n theo phương pháp học chuyển giao từ trọng số COCO, rồi chuyển đổi mô hình sang định dạng ONNX để suy luận trên CPU. Cuối cùng, công tác thực nghiệm và đánh giá được thực hiện trên sa hình thử nghiệm để kiểm chứng hiệu năng. Dựa trên dữ liệu thu thập được trong quá trình chạy thử, hệ thống được phân tích định lượng về độ chính xác nhận diện, tính ổn định của quỹ đạo lái và tốc độ xử lý, làm cơ sở để tinh chỉnh tham số và tối ưu khả năng vận hành của xe trong môi trường thực tế.

1.6. Ý nghĩa khoa học và thực tiễn

Việc hoàn thành các yêu cầu của đề án mang lại đóng góp trên cả hai phương diện học thuật và kỹ thuật ứng dụng. Về mặt học thuật, đề tài là một minh chứng cụ thể cho tính phù hợp của mô hình xử lý biên kết hợp phân tải xử lý giữa hai bộ xử lý chuyên biệt trong lĩnh vực robot di động. Việc triển khai thành công một mạng nơ-ron YOLOv8n trên phần cứng có tài nguyên hạn chế, thông qua chuyển đổi ONNX và suy luận trên CPU, củng cố tính khả thi của xu hướng AI cho các nền tảng nhỏ trong các bài toán nhận dạng thời gian thực.

Về mặt thực tiễn, mô hình xe sau khi hoàn thiện có thể là một giáo cụ trực quan cho các môn học thuộc ngành Điện tử - Tự động hóa, Công nghệ kỹ thuật máy tính và Trí tuệ nhân tạo, giúp người học tiếp cận trực tiếp với mã nguồn và kiến trúc hệ thống thực tế. Bên cạnh mục đích giáo dục, kinh nghiệm xây dựng kiến trúc nhận thức - lập kế hoạch - điều khiển và kỹ thuật tối ưu suy luận cho phần cứng nhúng trong đề tài cũng là tiền đề có thể mở rộng

cho các hệ thống xe vận chuyển tự động quy mô lớn hơn trong môi trường kho bãi và logistic.

1.7. Bố cục của đồ án

Đồ án tốt nghiệp được cấu trúc thành năm chương với nội dung như sau:

- Chương 1: Tổng quan - trình bày lý do chọn đề tài, khảo sát hiện trạng công nghệ trong và ngoài nước, xác định mục tiêu, đối tượng, phạm vi nghiên cứu và phương pháp thực hiện.
- Chương 2: Cơ sở lý thuyết - hệ thống hóa các nền tảng lý thuyết chính: kiến trúc hệ thống xe tự hành theo tiêu chuẩn SAE J3016; nền tảng phần cứng nhúng (Raspberry Pi 4, ESP32, camera, mạch cầu H, servo, nguồn); kỹ thuật giao tiếp và điều khiển trong hệ thống nhúng (UART, PWM, ngắt ngoại); cơ sở xử lý ảnh phát hiện làn đường; lý thuyết phát hiện đối tượng bằng mạng YOLO; cơ sở lý thuyết điều khiển PID; mô hình hóa và ước lượng trạng thái quỹ đạo; và lập trình đa luồng cho hệ thống thị giác thời gian thực.
- Chương 3: Phân tích và Thiết kế hệ thống - mô tả chi tiết thiết kế cơ khí, sơ đồ khối phần cứng, sơ đồ nguyên lý mạch điện, và đặc biệt là thiết kế phần mềm: kiến trúc phân tán Master - Slave và cơ chế đa luồng, pipeline phát hiện làn đường, khối ước lượng trạng thái và lọc tín hiệu thời gian, khối nhận thức ngữ nghĩa bằng YOLO, khối phân xử hành vi bằng máy trạng thái hữu hạn, và khối điều khiển động học cấp thấp cùng giao tiếp UART giữa Raspberry Pi 4 và ESP32.
- Chương 4: Thi công và Kết quả thực nghiệm - trình bày quá trình lắp ráp thực tế, quá trình huấn luyện mô hình YOLOv8n, kết quả chạy thử nghiệm trên sa hình và các biểu đồ đánh giá sai số, độ chính xác nhận dạng.
- Chương 5: Kết luận và Hướng phát triển - tóm tắt các kết quả đạt được so với mục tiêu ban đầu, phân tích các hạn chế còn tồn tại và đề xuất hướng cải tiến.

Chương 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về xe tự hành

2.1.1. Khái niệm và các cấp độ tự hành

Xe tự hành là phương tiện có khả năng cảm nhận môi trường xung quanh và di chuyển an toàn với rất ít hoặc hoàn toàn không có sự can thiệp của con người. Để làm được điều đó, xe phải giải quyết đồng thời ba bài toán: nhận thức môi trường - xung quanh có gì, định vị và quyết định - xe đang ở đâu, cần đi thế nào, và điều khiển chấp hành - biến quyết định thành tác động vật lý lên lái, ga, phanh.

Hiệp hội Kỹ sư Ô tô Hoa Kỳ, tiêu chuẩn J3016 [9], phân loại mức độ tự động hóa phương tiện thành sáu cấp độ từ 0 đến 5. Bảng 2.1 tóm tắt sáu cấp độ này.

Bảng 2.1. Sáu cấp độ tự hành theo tiêu chuẩn SAE J3016 [9]

Cấp độ	Tên gọi	Đặc điểm chính
Cấp 0	Không tự động	Người lái thực hiện toàn bộ; xe chỉ có thể cảnh báo.
Cấp 1	Hỗ trợ người lái	Xe hỗ trợ một chức năng đơn lẻ; giữ làn hoặc ga tự động.
Cấp 2	Tự động hóa một phần	Xe đồng thời điều khiển lái và ga/phanh; người lái phải giám sát liên tục.
Cấp 3	Tự động hóa có điều kiện	Xe tự lái trong điều kiện xác định; người lái can thiệp khi được yêu cầu.
Cấp 4	Tự động hóa cao	Xe tự lái hoàn toàn trong vùng vận hành thiết kế, không cần người giám sát.
Cấp 5	Tự hành hoàn toàn	Xe tự lái trong mọi điều kiện mà con người có thể lái.

Mô hình xe trong đề tài thuộc phạm vi nghiên cứu học thuật quy mô thu nhỏ: tự bám làn đường, tự phát hiện vạch qua đường, biển báo, đèn tín hiệu và người đi bộ trên sa hình trong nhà, tương ứng về mặt chức năng với cấp độ 2. Dù quy mô nhỏ, mô hình vẫn giải quyết đầy đủ chuỗi nhận thức - quyết định - chấp hành như một hệ thống thật, nên cơ sở lý thuyết trình bày dưới đây mang tính tổng quát.

2.1.2. Kiến trúc ba tầng: nhận thức - quyết định - chấp hành

Hầu hết các hệ thống xe tự hành hiện đại đều được tổ chức theo kiến trúc phân tầng gồm ba tầng chức năng nối tiếp [10]:

Tầng nhận thức: thu nhận dữ liệu từ cảm biến từ camera, LiDAR, radar, siêu âm... và trích xuất thông tin có ý nghĩa: vị trí làn đường, vật cản, biển báo, đèn tín hiệu.

Tầng quyết định: tổng hợp thông tin nhận thức thành lệnh điều khiển mức cao. từ độ lệch quỹ đạo suy ra lệnh lái, từ đối tượng nhận diện được suy ra mức tốc độ phù hợp.

Tầng chấp hành: biến lệnh mức cao thành tín hiệu điện điều khiển cơ cấu vật lý - động cơ, servo.

Ưu điểm căn bản của kiến trúc phân tầng là tách bạch trách nhiệm: mỗi tầng có chu kỳ thời gian và độ phức tạp tính toán rất khác nhau, suy luận học sâu có thể mất hàng trăm mili-giây, trong khi vòng điều khiển động cơ cấp thấp cần chạy đều đặn ở chu kỳ vài chục mili-giây, nên việc phân tách cho phép mỗi tầng chạy trên nền tảng tính toán phù hợp nhất với nó. Đây chính là cơ sở lý luận cho kiến trúc phân tán giữa bộ xử lý trung tâm và vi điều khiển chấp hành được lựa chọn ở Chương 3.

2.2. Nền tảng phần cứng của hệ thống

2.2.1. Vi xử lý nhúng Raspberry Pi 4 Model B

Raspberry Pi 4 Model B là máy tính bo mạch đơn thể hệ thứ tư do Raspberry Pi Foundation phát triển, ra mắt năm 2019. Thiết bị được trang bị SoC Broadcom BCM2711 gồm bốn nhân ARM Cortex-A72 64-bit xung nhịp 1,5-1,8 GHz, RAM LPDDR4-3200 với các tùy chọn 2/4/8 GB, GPU VideoCore VI, cùng hệ thống ngoại vi phong phú: cổng camera CSI-2 15 chân, header GPIO 40 chân mức logic 3,3 V với các bus UART, I2C, SPI, hai cổng USB 3.0, Gigabit Ethernet và WiFi hai băng tần [11].

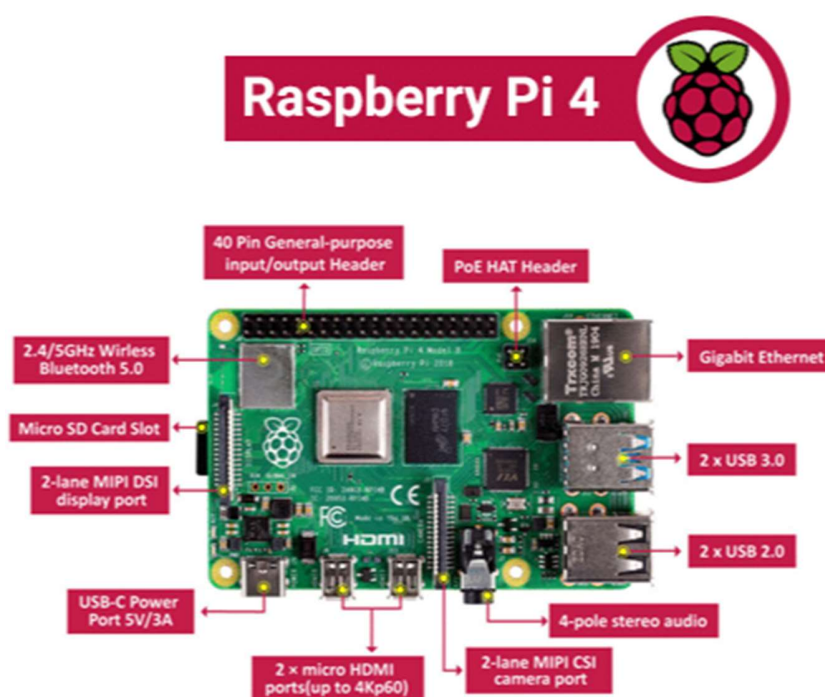
Khác với vi điều khiển, Raspberry Pi chạy hệ điều hành Linux đầy đủ (Raspberry Pi OS, nền Debian), cho phép sử dụng trực tiếp các thư viện tính toán hiệu năng cao biên dịch sẵn cho kiến trúc ARM, cùng lớp trừu tượng chuẩn của nhân Linux cho thiết bị thu hình (V4L2), nhờ đó tận dụng được cả bốn nhân CPU cho các tác vụ xử lý song song.

Điểm mạnh quyết định khiến Raspberry Pi 4 được chọn làm bộ xử lý trung tâm là sự cân bằng giữa năng lực tính toán: đủ để chạy đồng thời một pipeline xử lý ảnh cổ điển và một mạng nơ-ron suy luận trên CPU (số liệu cụ thể tại Mục 2.7/Bảng 3.5); hệ sinh thái phần

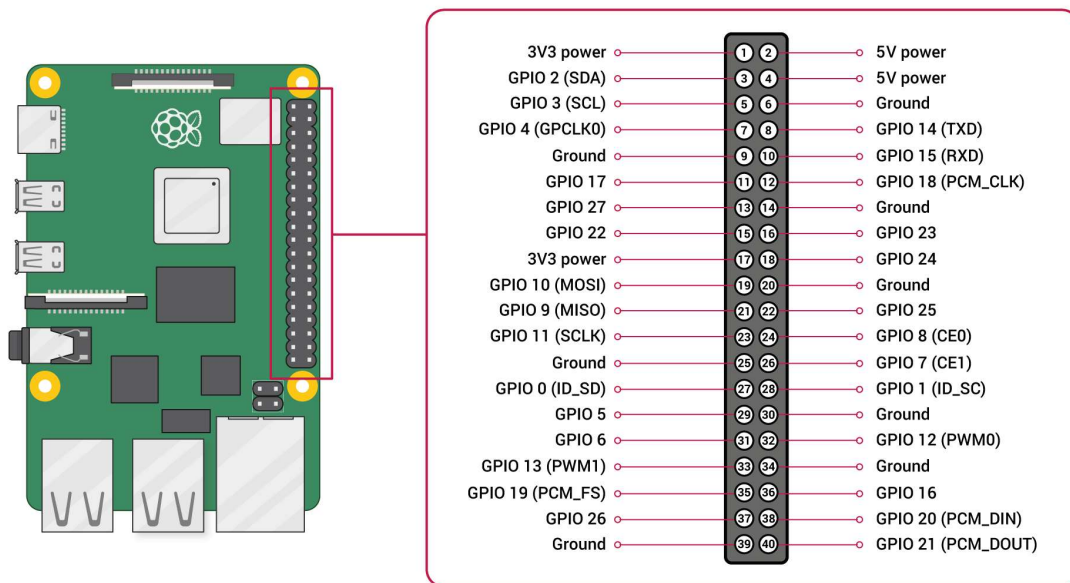
mềm trường thành, và mức tiêu thụ điện vừa phải, phù hợp nguồn pin di động. Hạn chế chính của nền tảng là Linux không phải hệ điều hành thời gian thực: thời điểm thực thi một tiến trình có thể bị trễ không xác định do bộ lập lịch, nên các tác vụ cần định thời chính xác (PWM động cơ, servo) không giao cho Raspberry Pi mà cho ESP32.

Bảng 2.2. Thông số kỹ thuật chính của Raspberry Pi 4 Model B

Thông số	Giá trị
SoC	Broadcom BCM2711, 4 nhân Cortex-A72 @ 1,5 GHz, 64-bit
RAM	LPDDR4-3200 4GB
GPU	VideoCore VI
Giao diện camera	MIPI CSI-2, 15 chân
GPIO	40 chân, mức logic 3,3V
Nguồn	5V/3A qua USB-C hoặc chân GPIO 5 V
Hệ điều hành	Raspberry Pi OS (Linux Debian), 64-bit



Hình 2.1. Vi xử lý Raspberry Pi 4 Model B



Hình 2.2. Sơ đồ chân Raspberrypi Pi 4 Model B

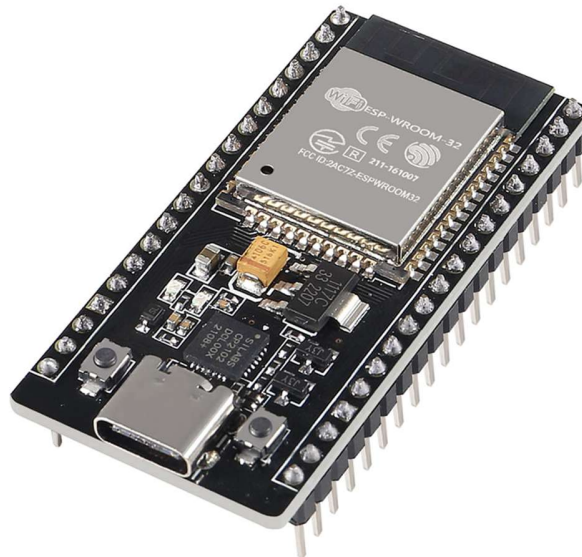
2.2.2. Vi điều khiển ESP32

ESP32 là dòng vi điều khiển 32-bit do Espressif Systems phát triển, tích hợp hai nhân Xtensa LX6 xung nhịp tối đa 240 MHz, 520 KB SRAM, cùng khối thu phát WiFi 802.11 b/g/n và Bluetooth trên một chip duy nhất. Bộ ngoại vi của ESP32 đặc biệt phù hợp với bài toán điều khiển động cơ: 16 kênh PWM phân cứng thuộc khối LEDC, 3 bộ UART, các bộ định thời 64-bit, và khả năng gán ngắt ngoài cho hầu hết các chân GPIO [12].

Khác biệt bản chất giữa vi điều khiển dạng này và vi xử lý nhúng chạy hệ điều hành đầy đủ nằm ở mô hình thực thi: ESP-IDF cung cấp một bản phân phối FreeRTOS, hệ điều hành thời gian thực hỗ trợ đa nhiệm ưu tiên trên cả hai nhân vật lý, nhưng không có bộ lập lịch chia sẻ thời gian hay quản lý bộ nhớ ảo phức tạp như Linux. Nhờ đặc tính này, độ trễ phản ứng với sự kiện ngắt duy trì ổn định ở mức micro-giây, và chu kỳ của một tác vụ định kỳ có thể giữ đều đặn với sai số rất nhỏ - yếu tố then chốt cho các vòng điều khiển đòi hỏi tính thời gian thực nghiêm ngặt [13].

Bảng 2.3. Thông số kỹ thuật chính của vi điều khiển ESP32 [12]

Thông số	Giá trị
CPU	2 nhân Xtensa LX6 32-bit, tối đa 240 MHz
SRAM / Flash	520 KB SRAM nội / 4 MB Flash ngoài (module DevKitC)
PWM	16 kênh LEDC, độ phân giải tới 16-bit; 4 timer phần cứng dùng chung
UART	3 bộ, hỗ trợ tốc độ baud tùy chỉnh
Kết nối	WiFi 802.11 b/g/n 2,4 GHz; Bluetooth 4.2/BLE
GPIO	34 chân đa năng, mức logic 3,3V, hỗ trợ ngắt ngoài

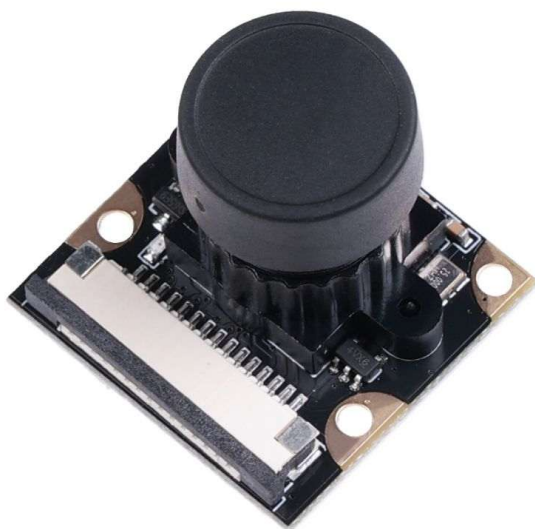


Hình 2.3. Module vi điều khiển ESP32 DevKitC [12]

2.2.3. Cảm biến hình ảnh - camera OV5647 góc rộng

Camera là cảm biến duy nhất cung cấp thông tin nhận thức cho hệ thống, do đó đặc tính của nó chi phối trực tiếp chất lượng của toàn bộ pipeline thị giác. Đề tài sử dụng module camera dùng cảm biến OmniVision OV5647, cảm biến CMOS 5 megapixel (2592×1944) từng được dùng trong Camera Module v1 chính thức của Raspberry Pi, phiên bản ống kính góc rộng với trường nhìn khoảng 160°. Module kết nối với Raspberry Pi qua cổng CSI 15 chân theo giao thức MIPI CSI-2, băng thông đủ truyền ảnh độ phân giải cao mà không chiếm CPU [14].

Hai đặc tính quang học của cảm biến góc rộng cần được lưu ý khi xây dựng thuật toán thị giác phía sau. Thứ nhất, trường nhìn càng rộng thì méo dạng thùng ở vùng rìa ảnh càng rõ rệt các đường thẳng trong không gian thực không còn giữ được tính thẳng khi chiếu lên vùng biên của ảnh, gây sai số cho các thuật toán giả định đối tượng là đường thẳng như biến đổi Hough. Thứ hai, cảm biến CMOS dân dụng mặc định vận hành ở chế độ phơi sáng tự động: bộ điều khiển phơi sáng liên tục điều chỉnh thời gian mở trập và độ lợi tương tự theo độ sáng trung bình của khung hình, khiến độ sáng toàn khung dao động giữa các frame khi điều kiện chiếu sáng nền thay đổi cục bộ, ví dụ khi xe đi qua vùng bóng đổ làm các ngưỡng phân đoạn màu cố định (trình bày ở Mục 2.4) kém ổn định theo thời gian. Để hạn chế ảnh hưởng này, các hệ thống thị giác nhúng thường chọn một trong hai hướng: khóa cố định thời gian phơi sáng và độ lợi ở tầng điều khiển cảm biến qua giao diện chuẩn V4L2, hoặc giữ chế độ tự động và bù trừ độ sáng ở tầng xử lý phần mềm bằng các kỹ thuật trình bày tại Mục 2.4.



Hình 2.4. Module camera OV5647 và đặc trưng ảnh góc rộng [14]

2.2.4. Mạch cầu H L298N và động cơ DC giảm tốc

Động cơ điện một chiều chuyển đổi điện năng thành cơ năng quay; chiều quay phụ thuộc chiều dòng điện qua cuộn dây và tốc độ tỷ lệ thuận với điện áp trung bình đặt vào.

Để một vi điều khiển 3,3V có thể điều khiển động cơ 12V dòng vài ampe, cần một tầng công suất trung gian, phổ biến nhất là mạch cầu H.

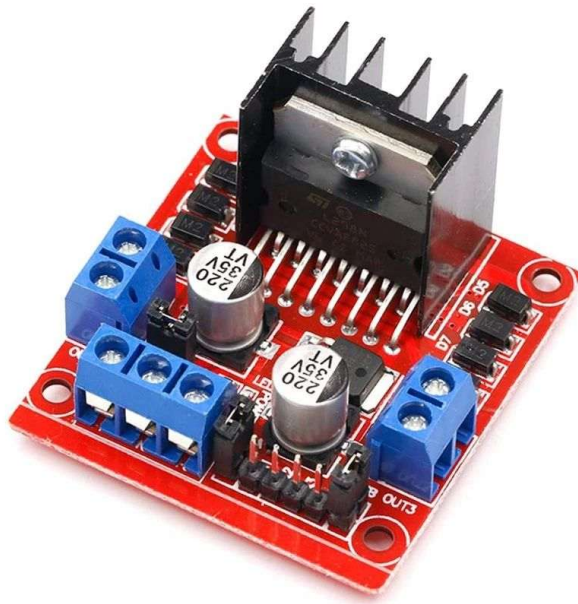
Cầu H gồm bốn khóa bán dẫn bố trí theo hình chữ H với động cơ nằm trên thanh ngang. Đóng cặp khóa chéo nhau cho dòng chảy qua động cơ theo một chiều; đóng cặp chéo còn lại đảo chiều dòng và do đó đảo chiều quay; đóng cả hai khóa thấp (hoặc mở tất cả) tạo trạng thái phanh hoặc thả trôi. Tốc độ được điều chỉnh bằng cách băm xung chân cho phép: điện áp trung bình đặt lên động cơ tỷ lệ với hệ số chu trình của xung.

L298N của STMicroelectronics là vi mạch cầu H kép kinh điển, mỗi kênh chịu dòng liên tục 2A, đỉnh 3A, điện áp tải 5-46V, tầng logic tương thích TTL. Mỗi vi mạch tích hợp hai kênh cầu H độc lập, cho phép điều khiển đồng thời hai động cơ DC từ một IC duy nhất. Trạng thái hai chân IN của mỗi kênh quyết định chiều quay theo bảng chân trị 2.4; chân cho phép của mỗi kênh nhận tín hiệu PWM để điều chỉnh tốc độ theo hệ số chu trình. Module L298N thương mại tích hợp sẵn tám diode hoàn năng lượng bảo vệ vi mạch khỏi sức điện động ngược sinh ra khi cuộn dây động cơ chuyển mạch, đặc tính quan trọng giúp cách ly nhiễu giữa mạch công suất và mạch logic [15].

Bảng 2.4. Bảng chân trị điều khiển một kênh cầu H L298N [15]

IN1 (IN3)	IN2 (IN4)	ENA (ENB)	Trạng thái động cơ
HIGH	LOW	PWM	Quay thuận, tốc độ theo duty cycle
LOW	HIGH	PWM	Quay ngược, tốc độ theo duty cycle
LOW	LOW	X	Dừng (thả trôi/phanh nhẹ)
HIGH	HIGH	X	Phanh nhanh

Động cơ giảm tốc DC kết hợp một động cơ DC thông thường với hộp số bánh răng gắn liền trực. Hộp số làm giảm tốc độ quay đầu ra xuống còn vài trăm vòng/phút, đồng thời khuếch đại mô-men theo tỉ lệ truyền của bộ bánh răng, đánh đổi tốc độ quay lấy lực kéo, phù hợp với các ứng dụng cần di chuyển ở tốc độ thấp nhưng ổn định và có khả năng kiểm soát tốt.

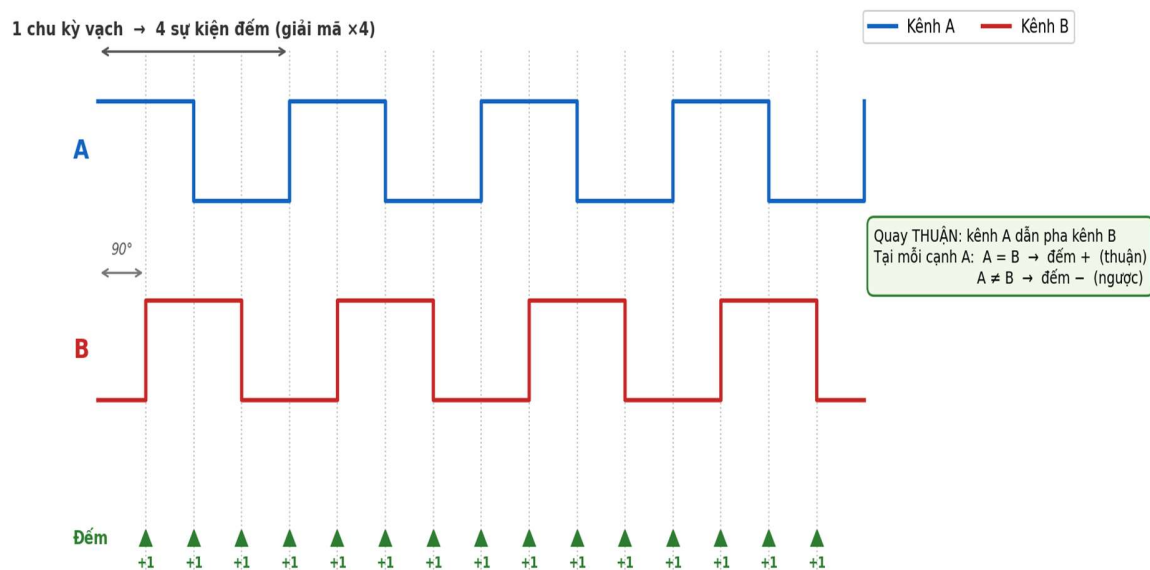


Hình 2.5. Nguyên lý mạch cầu H và module công suất L298N [15]

2.2.5. Encoder quadrature và nguyên lý đo tốc độ quay

Muốn điều khiển kín tốc độ động cơ, hệ thống phải đo được tốc độ thực tế. Cảm biến thông dụng nhất cho mục đích này là encoder quay tăng dần: một đĩa khắc vạch gắn trên trục động cơ quét qua cặp cảm biến quang hoặc Hall, sinh ra chuỗi xung vuông có tần số tỷ lệ thuận với tốc độ quay.

Encoder hai kênh bố trí hai cảm biến A và B lệch nhau 90° điện. Nhờ độ lệch pha này, ngoài tốc độ còn suy ra được chiều quay: nếu kênh A sớm pha hơn B thì trục quay thuận, ngược lại là quay ngược. Về mặt logic, tại mỗi sườn (cạnh lên hoặc xuống) của một kênh, so sánh trạng thái tức thời của hai kênh: $A = B$ ứng với một chiều, $A \neq B$ ứng với chiều kia. Nếu đăng ký ngắt trên cả hai sườn - chế độ CHANGE của cả hai kênh, mỗi chu kỳ vạch cho 4 sự kiện đếm, độ phân giải gấp 4 lần so với đếm một sườn một kênh (giải mã $\times 4$) [10].



Hình 2.6. Tín hiệu encoder quadrature và nguyên lý giải mã $\times 4$

2.2.6. Động cơ servo và cơ cấu lái Ackermann

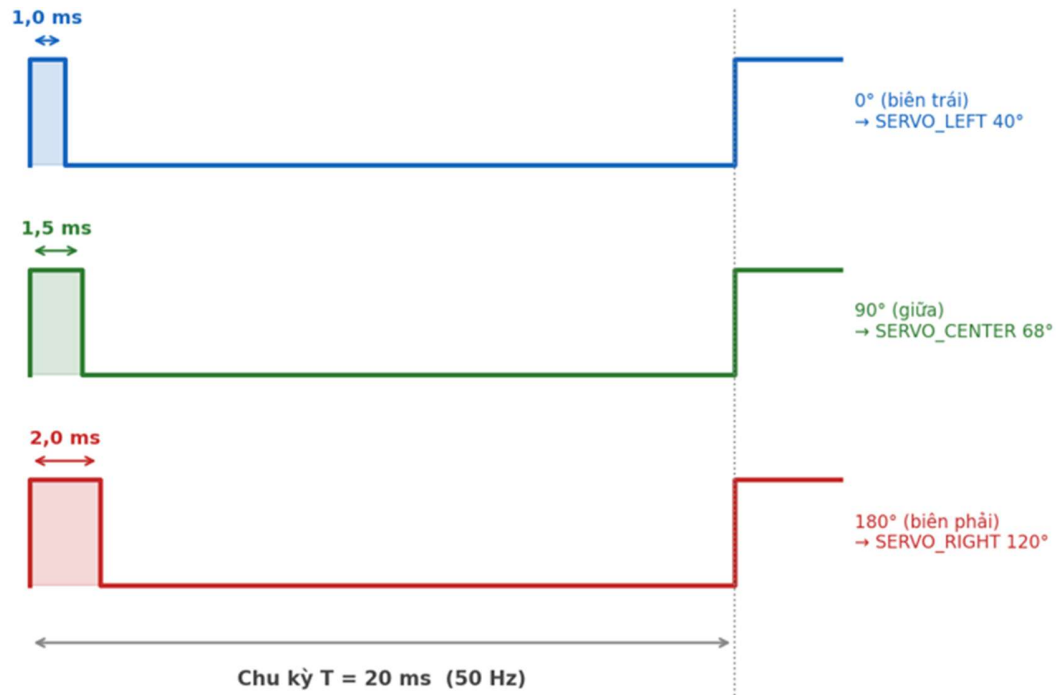
Servo RC là cụm cơ điện tích hợp gồm động cơ DC nhỏ, hộp giảm tốc và mạch điều khiển vị trí phản hồi bằng chiết áp. Khác với động cơ DC thường điều khiển tốc độ, servo điều khiển góc tuyệt đối của trục ra: mạch nội so sánh góc thực đọc từ chiết áp với góc đặt và tự cấp nguồn cho động cơ nội quay đến đúng vị trí, bản thân servo đã là một vòng điều khiển phản hồi âm hoàn chỉnh ở cấp linh kiện.

Giao thức điều khiển servo là PWM chu kỳ 20 ms (50 Hz), trong đó độ rộng xung 1-2 ms mã hóa góc đặt: 1 ms ứng với một biên hành trình, 1,5 ms ứng với vị trí giữa, 2 ms ứng với biên còn lại. Đề tài sử dụng servo MG996R/MG955 bánh răng kim loại, mô-men giữ khoảng 10-13 kg·cm, đủ khỏe để xoay cơ cấu lái khi bánh xe ma sát với mặt sa hình [16].

Về cơ khí, xe sử dụng cơ cấu lái Ackermann thu nhỏ: hai bánh trước xoay quanh trụ đứng riêng, liên kết với nhau và với trục servo qua các thanh kéo hình thang. Hình học Ackermann được thiết kế sao cho khi vào cua, đường vuông góc của cả bốn bánh đồng quy tại một tâm quay tức thời, bánh trong cua xoay góc lớn hơn bánh ngoài, nhờ đó các bánh lăn không trượt ngang, giảm mài mòn và giữ quỹ đạo ổn định [10].

Một hệ quả thực tiễn quan trọng của cơ cấu này: do dung sai lắp ráp của các thanh kéo, vị trí trung tâm thực tế của servo hiếm khi trùng 90° lý thuyết, và hành trình hai phía

có thể không hoàn toàn đối xứng. Vì vậy các giới hạn góc lái trong thực tế cần được hiệu chỉnh thực nghiệm theo từng mô hình cơ khí cụ thể, thay vì lấy theo thông số danh định của servo.



Hình 2.7. Xung PWM servo

2.2.7. Nguồn điện và bộ chuyển đổi DC-DC kiểu buck

Một hệ thống xe tự hành thường có nhiều tải tiêu thụ điện với yêu cầu điện áp và đặc tính dòng khác nhau: động cơ DC cần dòng lớn và sinh nhiều khi khởi động/đảo chiều, trong khi máy tính nhúng và vi điều khiển cần điện áp thấp nhưng phải ổn định tuyệt đối, không chấp nhận sụt áp tức thời. Khi nguồn năng lượng chung là pin Li-ion/LiPo, hệ thống cần bộ chuyển đổi DC-DC giảm áp kiểu buck để tạo ra các đường điện áp thấp hơn từ điện áp pin.

Bộ buck hoạt động theo nguyên lý đóng cắt: khóa bán dẫn băm điện áp vào ở tần số hàng trăm kHz, cuộn cảm và tụ lọc lấy trung bình thành điện áp ra thấp hơn, ổn định bằng vòng phản hồi điều chỉnh duty cycle. Khác với ổn áp tuyến tính như 7805 tiêu tán phần điện áp dư thành nhiệt, hiệu suất chỉ khoảng 40% khi hạ áp lớn, bộ buck đạt hiệu suất 85-95%, hầu như không sinh nhiệt và không lãng phí năng lượng pin, quan trọng với thiết bị chạy pin [17].

Nguyên tắc thiết kế quan trọng thứ hai là tách tải: nhiều và sụt áp từ động cơ nếu dùng chung đường nguồn với máy tính nhưng có thể khiến máy tính khởi động lại giữa chừng. Vì vậy mỗi nhóm tải nên dùng một bộ buck riêng, chỉ chung điểm đất để giữ mức tham chiếu tín hiệu đồng nhất giữa các khối.



Hình 2.8. Nguồn pin lipo 12V

2.3. Kỹ thuật giao tiếp và điều khiển trong hệ thống nhúng

2.3.1. Giao tiếp nối tiếp UART

UART là giao thức truyền nối tiếp bất đồng bộ lâu đời và phổ biến trong điện tử nhúng. Hai thiết bị nối bằng hai dây chéo với TX bên này vào RX bên kia cùng dây đất chung, không cần dây xung nhịp, cả hai phía tự đồng bộ theo bit Start của mỗi khung. Khung chuẩn 8N1 gồm 1 bit Start, 8 bit dữ liệu, 1 bit Stop, tổng 10 bit cho mỗi byte hữu ích; ở tốc độ 115.200 baud, thông lượng đạt khoảng 11.520 byte/giây, dư băng thông rất lớn so với một lệnh điều khiển vài ký tự gửi vài lần mỗi giây [18].

Giao thức dạng văn bản ASCII có ký tự kết thúc dòng thường được ưu tiên hơn gói nhị phân trong các ứng dụng gỡ lỗi thường xuyên: dễ quan sát bằng mắt, tự đồng bộ lại được sau khi mất byte, chỉ cần chờ ký tự xuống dòng kế tiếp, và phía nhận kiểm tra hợp lệ đơn giản trước khi xử lý. Trên Linux, cổng UART vật lý được trừu tượng hóa thành một tệp thiết bị và truy cập ở chế độ thô, không qua xử lý ký tự của terminal, để dữ liệu truyền trong suốt. Trên các vi điều khiển có nhiều bộ UART vật lý, nên dành riêng một bộ cho kênh lệnh, tách khỏi bộ dùng để nạp chương trình và in log, để hai chức năng không xung đột [18].

2.3.2. Điều chế độ rộng xung PWM và khối LEDC của ESP32

PWM điều khiển công suất bằng đóng cắt nhanh: tín hiệu chỉ có hai mức 0/1 nhưng tỷ lệ thời gian ở mức cao trong một chu kỳ quyết định giá trị trung bình tải nhận được:

$$V_{tb} = D \times V_{nguồn}$$

Với động cơ DC, quán tính cơ và điện cảm đủ lớn để tần số PWM vài kHz làm phẳng dòng điện, động cơ quay êm; tần số quá thấp gây rung/hú, quá cao tăng tổn hao chuyển mạch [18].

Trên ESP32, PWM được tạo bằng khối phần cứng LEDC, nhiều kênh chia sẻ chung một số bộ định thời hạn chế. Khi đồng thời điều khiển động cơ bằng PWM tần số cao và servo bằng PWM 50Hz cố định, cần cấp phát timer tường minh cho từng nhóm tần số, nếu để tự động, hai miền tần số khác nhau có thể bị gán trùng timer, gây giật servo hoặc sai tốc độ động cơ [19].

2.3.3. Ngắt ngoài và trình phục vụ ngắt

Ngắt cho phép CPU tạm dừng chương trình chính khi có sự kiện sườn tín hiệu, byte UART đến, timer tràn để chạy một đoạn mã ngắn rồi quay về đúng chỗ dừng, đảm bảo không bỏ sót sự kiện ngắn, khác với polling vốn tốn CPU liên tục và có thể đếm sót các xung tồn tại rất ngắn. Tín hiệu encoder là ví dụ điển hình buộc phải dùng ngắt: ở tốc độ cao, mỗi kênh có thể sinh hàng nghìn sườn xung mỗi giây, mỗi xung chỉ tồn tại vài trăm micro-giây [18].

Nguyên tắc viết ISR là càng ngắn càng tốt: chỉ đọc trạng thái và cập nhật biến đếm, tuyệt đối không gọi hàm chậm như in log, cấp phát bộ nhớ. Trên các vi điều khiển có cả vùng nhớ Flash và RAM thực thi nhanh, ISR nên đặt ở vùng RAM để không bị trễ khi Flash đang bận phục vụ tác vụ khác [13].

2.4. CƠ SỞ LÝ THUYẾT XỬ LÝ ẢNH SỐ

Phát hiện làn đường bằng thị giác máy tính thường áp dụng một chuỗi biến đổi hình ảnh tuần tự, từ hiệu chỉnh độ sáng đến phân tích hình học, mỗi bước được lựa chọn dựa trên đặc điểm vật lý của vạch kẻ đường có độ phản xạ cao, thường là màu trắng hoặc vàng so với nền đường tối hơn. Phần này trình bày cơ sở lý thuyết của từng kỹ thuật theo đúng thứ tự xử lý trong chuỗi: chuyển sang ảnh xám, lọc nhiễu, phân ngưỡng thích nghi cục bộ, các phép toán hình thái học, phân tích vùng liên thông, và quét dải ngang theo cửa sổ trượt [20].

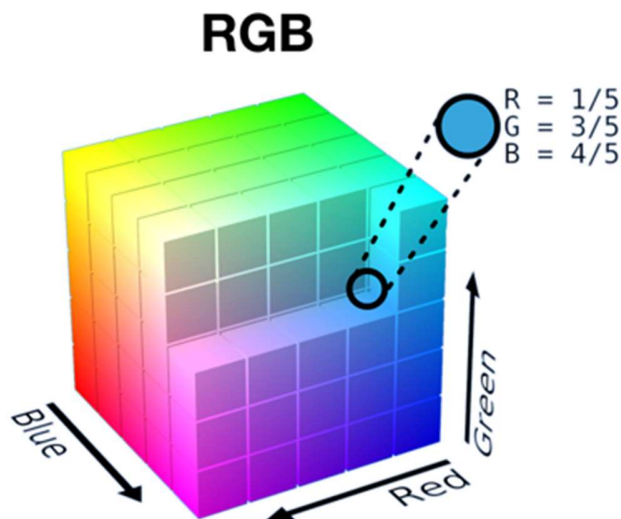
2.4.1. Không gian màu và biểu diễn ảnh số

Ảnh số được lưu trong bộ nhớ máy tính dưới dạng một ma trận ba chiều kích thước $H \times W \times C$, trong đó H là chiều cao, W là chiều rộng và C là số kênh màu. Trong các ví dụ tính toán ở phần này, độ phân giải 320×240 pixel tức 76.800 điểm ảnh mỗi khung hình, được dùng làm cơ sở minh họa, một lựa chọn thể hiện sự cân bằng phổ biến giữa độ chi tiết cần thiết để nhận dạng vạch kẻ và tốc độ xử lý khả thi trên phần cứng nhúng.

2.4.1.1. Không gian màu RGB

Trong không gian RGB, mỗi pixel được mã hoá bởi ba giá trị nguyên không dấu 8 bit $(R, G, B) \in [0, 255]^3$, biểu thị cường độ của ba thành phần ánh sáng cơ bản: đỏ, lục và lam. Mô hình này phản ánh trực tiếp cách cảm biến quang học CCD/CMOS thu nhận photon ánh sáng tới.

Hạn chế của RGB khi áp dụng cho bài toán phân loại màu nằm ở sự phụ thuộc lẫn nhau giữa thông tin độ chói và thông tin màu sắc: khi cường độ ánh sáng môi trường thay đổi, cả ba giá trị R, G, B của cùng một vật thể biến đổi đồng thời theo tỉ lệ không tuyến tính. Một vạch sơn trắng dưới bóng râm và dưới ánh sáng trực tiếp tạo ra hai bộ ba giá trị khác biệt rõ rệt, mặc dù bản chất vật lý của vật thể không thay đổi. Vì lý do này, việc đặt ngưỡng phân loại màu cố định trực tiếp trên không gian RGB thường không ổn định khi điều kiện chiếu sáng thực tế biến động [20].



Hình 2.9. Không gian màu RGB [21]

2.4.1.2. Không gian màu HSV

Không gian màu HSV tách riêng thông tin cường độ sáng (V) khỏi bản chất màu sắc (H) và độ tinh khiết của màu (S), phù hợp hơn với cách con người phân biệt màu sắc trong điều kiện sáng thay đổi. Gọi $R', G', B' \in [0, 1]$ là các giá trị RGB đã chuẩn hoá, và đặt $C_{\max} = \max(R', G', B')$, $C_{\min} = \min(R', G', B')$, $\Delta = C_{\max} - C_{\min}$. Phép biến đổi song ánh từ RGB sang HSV được xác định như sau:

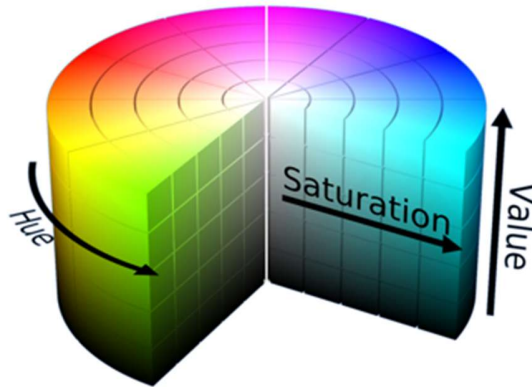
$$V = C_{\max}$$
$$S = \begin{cases} 0, & \text{nếu } C_{\max} = 0 \\ \frac{\Delta}{C_{\max}}, & \text{nếu } C_{\max} \neq 0 \end{cases} \quad [20]$$

Thành phần Hue được xác định theo ba trường hợp tùy vào kênh nào đạt giá trị $C_{\max}$:

$$H = \begin{cases} 0^\circ, & \text{nếu } \Delta = 0 \\ 60^\circ \times \left(\frac{G' - B'}{\Delta} \bmod 6 \right), & \text{nếu } C_{\max} = R' \\ 60^\circ \times \left(\frac{B' - R'}{\Delta} + 2 \right), & \text{nếu } C_{\max} = G' \\ 60^\circ \times \left(\frac{R' - G'}{\Delta} + 4 \right), & \text{nếu } C_{\max} = B' \end{cases} \quad [20]$$

Trong thư viện OpenCV, để dữ liệu nằm gọn trong một byte, các kênh S và V được ánh xạ về dải $[0, 255]$, còn kênh H, vốn có dải gốc từ 0° đến 360° , được chia đôi để nằm trong $[0, 180]$ [20].

Vạch sơn trắng phản xạ gần như toàn bộ quang phổ ánh sáng nhìn thấy, do đó trong không gian HSV nó tương ứng với độ bão hoà thấp và độ sáng cao, gần như độc lập với giá trị Hue. Đây là lý do không gian HSV thường được ưu tiên hơn RGB khi cần phân ngưỡng các đối tượng có màu trắng hoặc gần trắng: hai thuộc tính S thấp và V cao ổn định hơn nhiều so với bộ ba RGB khi ánh sáng môi trường thay đổi [22].



Hình 2.10. Không gian màu HSV [21]

2.4.2. Hiệu chỉnh chói sáng tuyến tính

Vạch kẻ ở vùng khuất bóng hoặc xa nguồn sáng thường bị suy giảm cường độ, gây khó khăn cho bước phân ngưỡng màu phía sau. Các kỹ thuật hiệu chỉnh phi tuyến như Gamma hay CLAHE cho chất lượng ảnh tốt nhưng đòi hỏi nhiều chu kỳ tính toán hơn, có thể ảnh hưởng đến tốc độ xử lý trên phần cứng nhúng có tài nguyên hạn chế. Trong các ứng dụng thời gian thực, một phép biến đổi điểm ảnh tuyến tính đơn giản hơn thường được ưu tiên:

$$g(x, y) = \text{saturate}(\alpha \cdot f(x, y) + \beta) \quad [23]$$

Trong đó $\alpha > 0$ là hệ số khuếch đại điều khiển độ tương phản, β là độ lệch điều khiển độ sáng tổng thể, và hàm *saturate* giới hạn giá trị đầu ra trong dải $[0, 255]$ để tránh tràn số khi lưu ở dạng 8 bit [23].



Hình 2.11. Ảnh trước (trái) và sau (phải) khi đi qua hàm hiệu chỉnh ánh sáng tuyến tính với $\alpha = 2,2$ và $\beta = 50$ [23]

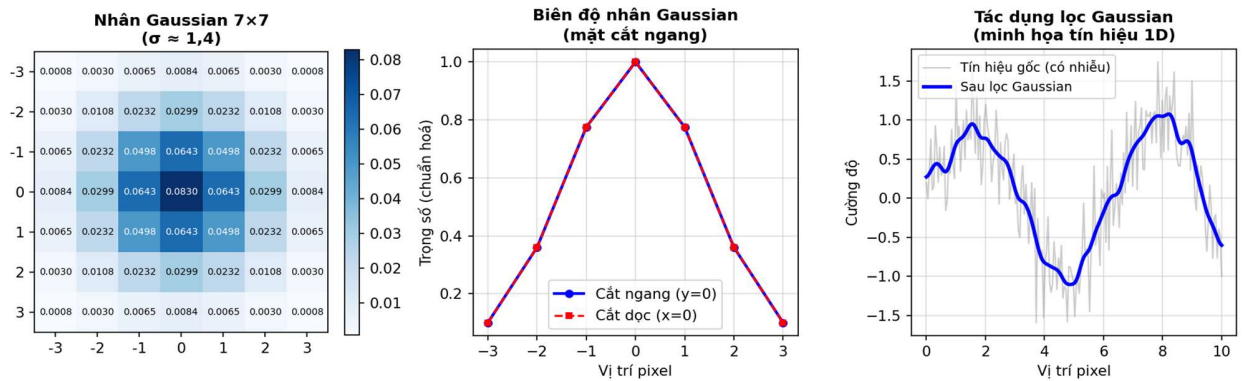
2.4.3. Lọc làm mịn Gaussian

Trước khi trích xuất đặc trưng hình học, ảnh cần được lọc nhiễu để tránh phát hiện biên giả do nhiễu cảm biến. Lọc Gaussian là kỹ thuật làm mịn tuyến tính phổ biến, thực hiện phép tích chập giữa ảnh gốc và một nhân Gaussian hai chiều:

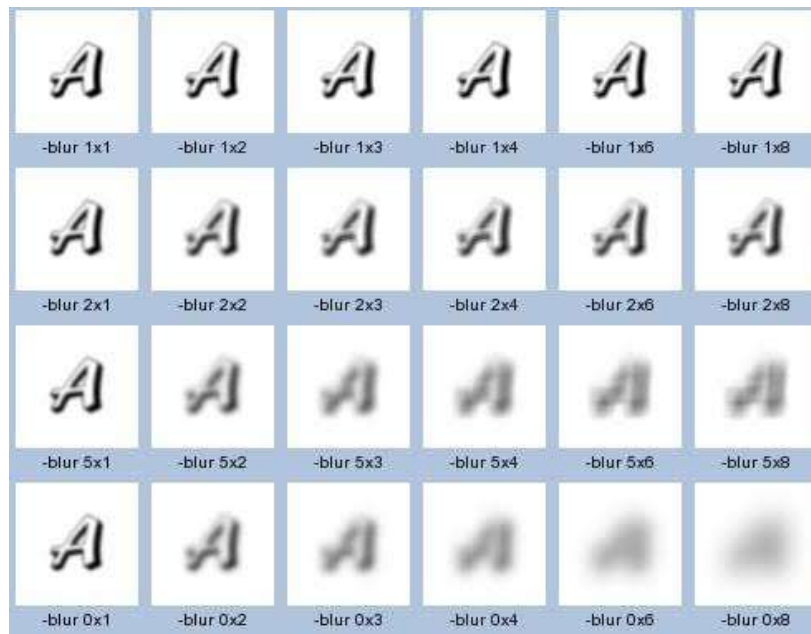
$$G(x, y; \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad [20]$$

trong đó σ là độ lệch chuẩn, quyết định mức độ làm mịn. Nhân rời rạc kích thước $(2k + 1) \times (2k + 1)$ thu được bằng cách lấy mẫu $G(x, y)$ tại các vị trí nguyên rồi chuẩn hoá tổng trọng số về 1.

Một ưu điểm của lọc Gaussian khi triển khai trên phần cứng nhúng là tính tách biệt: nhân hai chiều có thể phân rã thành tích của hai vector một chiều, cho phép thực hiện lần lượt hai phép tích chập 1D theo hàng và theo cột thay vì một phép tích chập 2D, giảm độ phức tạp tính toán từ $O(k^2)$ xuống $O(2k)$ phép nhân cho mỗi pixel [20].



Hình 2.12 Nhân Gaussian 7x7 (trái), biên độ cắt ngang (giữa) và tác dụng lọc làm mịn nhiễu ngẫu nhiên trong tín hiệu 1D (phải). $\sigma \approx 1,4$ tương ứng với phạm vi ảnh hưởng ~ 3 pixel mỗi chiều



Hình 2.13. Ảnh với các kernel Gaussion khác nhau [24]

2.4.4. Phân ngưỡng thích nghi cục bộ

Để tách vạch lằn ra khỏi ảnh xám, đề tài sử dụng phép phân ngưỡng thích nghi cục bộ thay cho phân ngưỡng cố định. Lý do là độ sáng trên mặt đường không đồng đều giữa các vùng trong khung hình; một ngưỡng cố định chung cho toàn ảnh sẽ hoặc bỏ sót vạch ở vùng tối, hoặc nhận dư ở vùng sáng. Phân ngưỡng thích nghi khắc phục bằng cách tính riêng một ngưỡng cho mỗi điểm ảnh dựa trên độ sáng trung bình của vùng lân cận quanh nó.

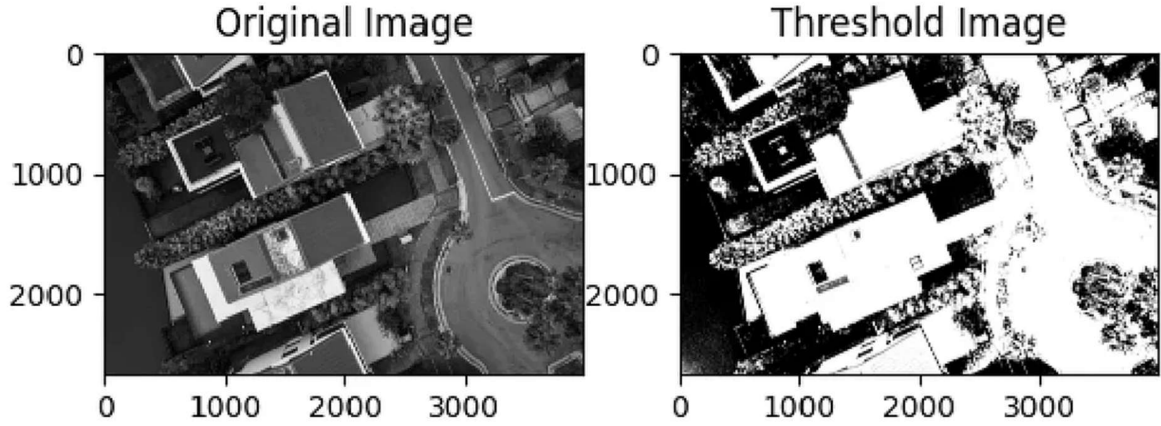
Gọi $I(x, y)$ là độ sáng của điểm ảnh và $\mu(x, y)$ là độ sáng trung bình trong ô cục bộ kích thước $b \times b$ bao quanh điểm đó. Ngưỡng cục bộ và quy tắc phân loại được xác định như sau:

$$T(x, y) = \mu(x, y) - C; \quad D(x, y) = 255 \text{ nếu } I(x, y) > T(x, y), \text{ ngược lại } = 0$$

với C là hằng số điều chỉnh. Khi chọn C mang giá trị âm, ngưỡng T được nâng cao hơn mức trung bình cục bộ, do đó chỉ những điểm ảnh sáng hơn hẳn vùng xung quanh, chính là vạch lằn mới được giữ lại, còn nền đường bị loại. Đề tài dùng ô cục bộ 41×41 lớn hơn bề rộng vạch và $C = -15$. Kết quả là ảnh nhị phân làm nổi bật các vùng sáng ứng với vạch lằn, bền vững với sự thay đổi độ sáng giữa các vùng trong khung hình.

Ngoài ra, để loại các vùng sáng không nằm trên mặt đường, đề tài tạo thêm một mặt nạ vùng tối ứng với phần mặt đường bằng phân ngưỡng nghịch đảo rồi lấy phần giao với

mặt nạ vạch. Chỉ những vùng sáng nằm bên trong mặt đường mới được giữ lại, nhờ vậy mặt nạ làn thu được sạch hơn.



Hình 2.14. Kết quả phân ngưỡng thích nghi cục bộ [40]

2.4.5. Các phép toán hình thái học

Mặt nạ nhị phân sau phân ngưỡng thường không hoàn hảo: xuất hiện các đốm nhiễu nhỏ trên nền đen, hoặc các khoảng đứt gãy bên trong vùng vạch sơn liên tục. Hình thái học nhị phân là tập các phép biến đổi dựa trên lý thuyết tập hợp, sử dụng một phần tử cấu trúc B để thao tác trên hình dạng của đối tượng trong ảnh.

Gọi A là tập các điểm ảnh mang giá trị 1 trong không gian $\mathbb{Z}^2$. Phép giãn nở $A \oplus B$ mở rộng vùng biên của đối tượng: một điểm thuộc kết quả nếu phần tử cấu trúc đặt tại vị trí đó giao với A .

$$A \oplus B = \{z \mid \hat{B}_z \cap A \neq \emptyset\}$$

Ngược lại, phép co $A \ominus B$ thu hẹp vùng biên: một điểm thuộc kết quả chỉ khi toàn bộ phần tử cấu trúc nằm trọn trong A :

$$A \ominus B = \{z \mid (B)_z \subseteq A\}$$

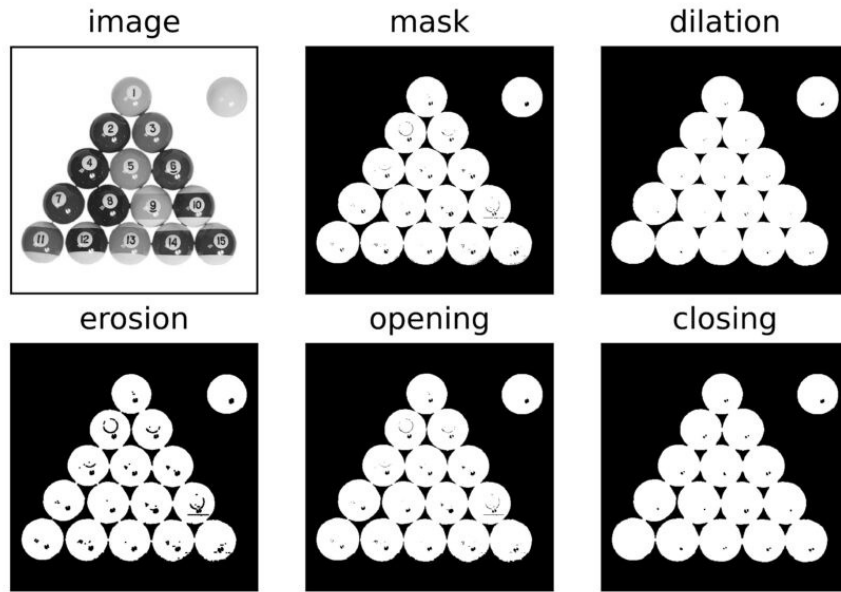
Từ hai phép toán gốc này, hai phép toán phái sinh được sử dụng để xử lý các dạng nhiễu khác nhau mà không làm thay đổi đáng kể diện tích tổng thể của đối tượng. Phép mở $A \circ B$ là phép co theo sau bởi giãn nở:

$$A \circ B = (A \ominus B) \oplus B$$

có tác dụng cắt các eo hẹp và loại bỏ các đốm nhiễu nhỏ nằm rời rạc bên ngoài đối tượng chính. Phép đóng $A \bullet B$ là phép giãn nở theo sau bởi co:

$$A \bullet B = (A \oplus B) \ominus B$$

có tác dụng lấp đầy các khoảng hở nhỏ và hàn gắn các đoạn vạch bị đứt gãy bên trong đối tượng [20].



Hình 2.15. Biểu diễn trực quan hiệu ứng của các phép toán hình thái học trong việc xử lý nhiễu biên và lấp đầy các khoảng đứt gãy bên trong đối tượng [26]

2.4.6. Phân tích vùng liên thông và lọc nhiễu theo diện tích

Sau bước phân ngưỡng và làm sạch bằng hình thái học, ảnh nhị phân được phân tích thành các vùng liên thông, mỗi vùng là một cụm điểm trắng nối liền nhau. Với mỗi vùng, hệ thống tính diện tích số điểm ảnh thuộc vùng.

Những vùng có diện tích nhỏ hơn một ngưỡng cho trước được xem là nhiễu và bị xóa; chỉ giữ lại các vùng đủ lớn tương ứng với vạch lằn. Bước này loại bỏ các đốm trắng lẻ tẻ còn sót, để việc xác định tâm lằn ở bước sau không bị đánh lừa bởi nhiễu. So với phương pháp dò biên rồi ghép đoạn thẳng, cách lọc theo vùng liên thông đơn giản và ít nhạy nhiễu hơn trên ảnh nhị phân đã tách vạch.

2.4.7. Hình học phối cảnh và vùng quan tâm ROI

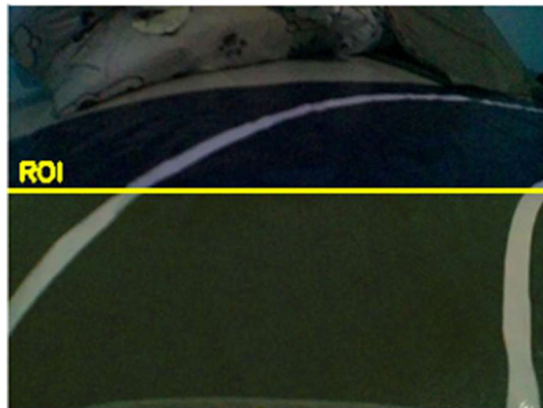
Camera được gắn cố định trên khung xe, hướng về phía trước với một góc nghiêng xuống so với phương ngang. Theo mô hình camera lỗ kim, phép chiếu phối cảnh ánh xạ tọa độ không gian ba chiều xuống mặt phẳng ảnh hai chiều. Hệ quả hình học của phép chiếu này là hai vạch giới hạn làn đường vốn song song trong không gian thực tế hội tụ về một điểm tụ nằm trên đường chân trời ảo [25].

Trong một khung hình đầy đủ, nửa trên thường chứa hậu cảnh (tường, vật thể ở xa, thiết bị ngoài mặt đường) không mang thông tin về quỹ đạo di chuyển. Phần mặt đường hữu ích nằm tập trung ở nửa dưới của khung hình, đặc biệt khi camera có góc nhìn rộng và được lắp đặt đủ cao so với mặt đường.

Kỹ thuật Vùng quan tâm giới hạn không gian xử lý vào đúng vùng mang thông tin hữu ích bằng cách loại bỏ toàn bộ phần ảnh phía trên một ngưỡng chiều cao y_{top} . Kết quả là một vùng hình chữ nhật kéo dài toàn bộ chiều rộng W và chiều cao từ y_{top} đến mép dưới của ảnh:

$$A_{ROI} = W \times (H - y_{top})$$

Diện tích này nhỏ hơn diện tích toàn ảnh ($W \times H$) theo tỉ lệ tương ứng với giá trị y_{top} , giúp giảm mật độ điểm ảnh đầu vào cho các bước xử lý phía sau. Không giống với ROI hình thang thường dùng để bù trừ hiệu ứng hội tụ phối cảnh, ROI hình chữ nhật được áp dụng khi ống kính góc rộng đã bao phủ cả hai vạch làn trong toàn bộ chiều rộng ảnh ở mọi khoảng cách thuộc vùng xử lý, khiến việc thu hẹp cạnh bên không mang lại lợi ích thực tế [25].



Hình 2.16. Hiện tượng hội tụ của các đường thẳng song song qua phép chiếu phối cảnh và mặt nạ giới hạn vùng quan tâm

2.4.8. Phân tích đường bao và nhận dạng vạch đi bộ

Vạch đi bộ bao gồm nhiều dải sơn trắng ngang được xếp song song với nhau. Cấu trúc hình học này đi ngược lại với giả thiết "làn đường là các đoạn thẳng dài liên tục" mà biến đổi Hough xác suất thường khai thác. Do đó, phương pháp phân tích đường bao được áp dụng riêng biệt nhằm giải quyết bài toán nhận diện đối tượng này. [30]

Thuật toán trích xuất đường bao (Suzuki, 1985) hoạt động bằng cách quét ma trận ảnh nhị phân để dò tìm các biên giới khép kín phân tách giữa vùng đối tượng với pixel có giá trị 1 và vùng nền với pixel có giá trị 0. Với mỗi đường bao được trích xuất, hệ thống tiến hành tính toán ba đặc trưng hình học cốt lõi để phân loại:

Thứ nhất, diện tích bề mặt của đối tượng được đánh giá nhằm loại bỏ các đường bao có kích thước quá nhỏ, vốn thường phát sinh từ nhiễu cảm biến hoặc gợn sóng trên mặt đường.

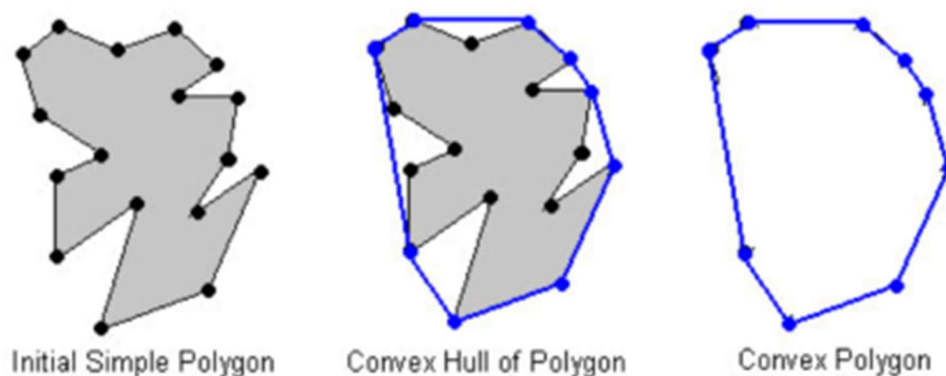
Thứ hai, tỉ lệ khung hình, ký hiệu là AR , được xác định dựa trên kích thước của hình chữ nhật giới hạn bao quanh đối tượng với chiều rộng W và chiều cao H :

$$AR = \frac{W}{H}$$

Dưới góc nhìn của phép chiếu phối cảnh, một dải sơn của vạch đi bộ thường xuất hiện dưới dạng các dải ngang dẹt, do đó AR của chúng có giá trị lớn hơn 1. Đặc điểm này tạo ra sự khác biệt hình thái rất rõ rệt so với các vạch phân làn dọc vốn có AR nhỏ hơn 1.

Thứ ba, mật độ lấp đầy được định nghĩa là tỉ số giữa diện tích thực tế của đường bao và diện tích của hình chữ nhật giới hạn hoặc đa giác lồi bao quanh nó. Vạch đi bộ mang hình dáng của một hình chữ nhật vuông vức, do đó mật độ lấp đầy của nó sẽ tiến tới 1. Ngược lại, các đối tượng nhiễu trong môi trường thường có dạng đường cong hoặc méo mó, dẫn đến tỉ lệ lấp đầy thấp.

Khi số lượng đường bao thỏa mãn đồng thời các tiêu chí hình học trên đạt đến một ngưỡng xác định và có phân bố tập trung theo phương ngang trong vùng quan tâm, sự xuất hiện của vạch đi bộ phía trước được xác nhận. Đây là một tín hiệu hữu ích cho việc điều chỉnh hành vi vận hành theo ngữ cảnh giao thông, đồng thời có thể kết hợp với các tín hiệu nhận thức khác để tăng độ tin cậy tổng thể của hệ thống.



Hình 2.17. Minh họa đường bao gốc của đối tượng (trái) và đa giác lồi bao quanh nó (giữa) dùng làm cơ sở tính toán đặc trưng mật độ lấp đầy

2.4.9. Lý thuyết Lọc quán tính và Hàm phi tuyến vùng chết

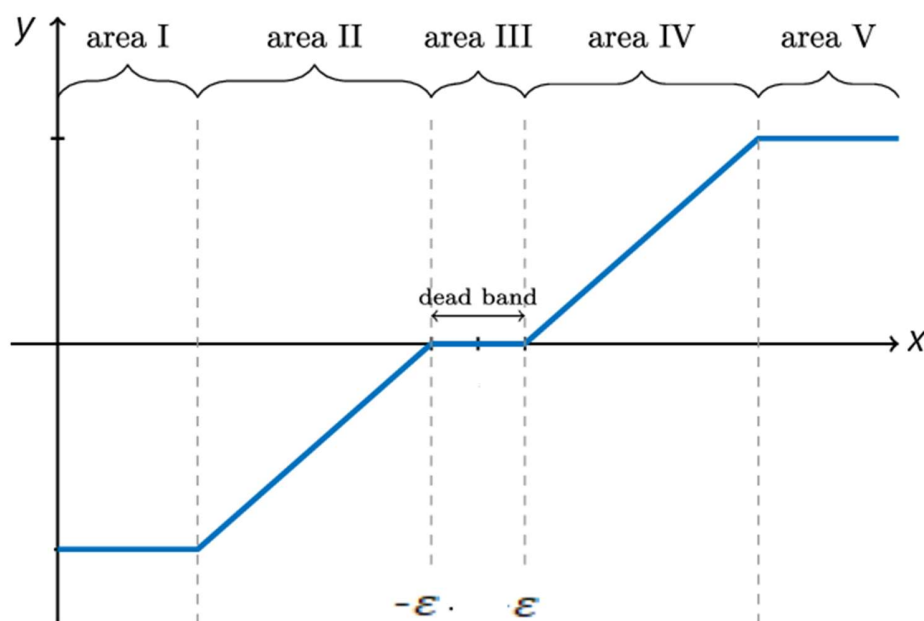
Tín hiệu đầu ra của chuỗi xử lý thị giác máy tính như độ lệch tâm làn đường, góc lái được trích xuất theo từng khung hình rời rạc, do đó luôn chứa các dao động nhiễu tần số cao hoặc sai số đo lường ngẫu nhiên. Để đảm bảo cơ cấu lái hoạt động ổn định và mượt mà, tín hiệu thô cần đi qua các khâu xử lý tín hiệu động lực học trước khi chuyển thành lệnh điều khiển cơ khí.

2.4.9.1. Hàm phi tuyến vùng chết:

Hàm vùng chết là một khâu phi tuyến được ứng dụng rộng rãi trong lý thuyết điều khiển tự động nhằm ngăn chặn hiện tượng dao động liên tục khi sai lệch quỹ đạo quanh tâm rất nhỏ. Cho tín hiệu đầu vào x (độ lệch tâm) và ngưỡng vùng chết ε , tín hiệu đầu ra y được biểu diễn theo hàm toán học sau:

$$y = \begin{cases} 0 & \text{nếu } |x| \leq \varepsilon \\ x & \text{nếu } |x| > \varepsilon \end{cases}$$

Về mặt vật lý, khi độ lệch quỹ đạo nằm trong phạm vi cho phép ($|x| \leq \varepsilon$), hệ thống xác nhận phương tiện đã bám đúng tâm làn và xuất ra tín hiệu lái bằng 0 để giữ vô lăng ổn định. Ngưỡng ε cần được thiết lập cân bằng: đủ nhỏ để không làm suy giảm độ nhạy của hệ thống bám làn, nhưng phải đủ lớn để triệt tiêu hoàn toàn các nhiễu rung động ngẫu nhiên.



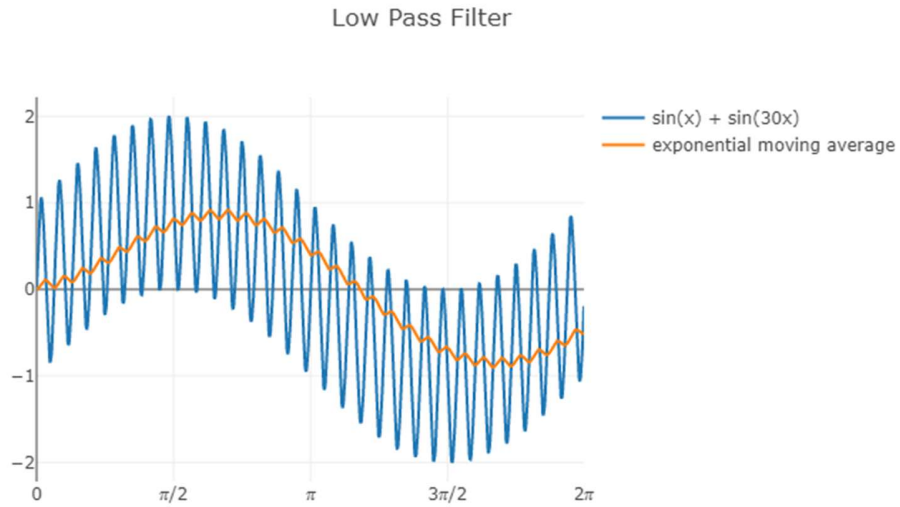
Hình 2.18. Đặc tuyến vào-ra của hàm phi tuyến vùng chết (Khu vực III) kết hợp các giới hạn bão hòa

2.4.9.2. Bộ lọc Trung bình trượt hàm mũ:

Bộ lọc trung bình trượt hàm mũ đóng vai trò như một bộ lọc thông thấp IIR bậc một, có tác dụng làm mượt chuỗi dữ liệu theo thời gian thực. Giá trị đầu ra ở thời điểm hiện tại S_t được tính toán dựa trên sự kết hợp tuyến tính giữa giá trị đo lường mới nhất Y_t và trạng thái quá khứ S_{t-1} :

$$S_t = \alpha \cdot Y_t + (1 - \alpha) \cdot S_{t-1}$$

Trong phương trình trên, hệ số làm mượt $\alpha \in [0,1]$ quyết định đặc tính băng thông của bộ lọc. Khi α mang giá trị lớn tiến gần về 1, bộ lọc đặt trọng số cao cho quan sát hiện tại, đảm bảo thời gian đáp ứng nhanh đối với sự biến thiên của quỹ đạo nhưng bù lại sẽ làm giảm khả năng lọc nhiễu. Sự kết hợp chặt chẽ giữa hàm phi tuyến vùng chết và bộ lọc quán tính EMA tạo ra một tín hiệu điều khiển mềm mại, chống lại các xung lực gắt, qua đó giảm biên độ và tần suất các lệnh điều khiển truyền đến cơ cấu chấp hành phía sau.



Hình 2.19. So sánh tín hiệu chứa dao động nhiễu (xanh) và tín hiệu đầu ra sau khi đi qua bộ lọc quán tính EMA (cam)

2.5. PHÁT HIỆN ĐỐI TƯỢNG BẰNG HỌC SÂU

Song song với chuỗi xử lý ảnh cổ điển đảm nhận việc bám làn và nhận diện vạch đi bộ, hệ thống vận hành một luồng suy luận học sâu độc lập để nhận dạng các đối tượng mang tính ngữ nghĩa: biển báo giao thông, đèn tín hiệu và người đi bộ. Phần này trình bày cơ sở lý thuyết của mạng nơ-ron tích chập và kiến trúc YOLOv8n được triển khai trong hệ thống.

2.5.1. Mạng nơ-ron tích chập

Mạng nơ-ron tích chập là kiến trúc học sâu được thiết kế để khai thác cấu trúc không gian của dữ liệu dạng lưới, điển hình là ảnh hai chiều. Đặc trưng cốt lõi của CNN là chia sẻ trọng số: cùng một bộ lọc được áp dụng tại mọi vị trí trên ảnh, thay vì mỗi vùng ảnh có một tập trọng số riêng như trong mạng kết nối đầy đủ. Cách thiết kế này tạo ra tính bất biến với phép tịnh tiến và giảm mạnh số lượng tham số cần huấn luyện. [33]

2.5.1.1. Lớp tích chập

Lớp tích chập đóng vai trò trích xuất đặc trưng, biến đổi tensor đầu vào $X \in \mathbb{R}^{H \times W \times C_{in}}$ thành tensor đặc trưng $Y \in \mathbb{R}^{H' \times W' \times C_{out}}$ thông qua phép toán tích chập rời rạc:

$$Y[i, j, k] = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} \sum_{c=0}^{C_{in}-1} W[m, n, c, k] \cdot X[i \cdot s + m, j \cdot s + n, c] + b[k] \quad [33]$$

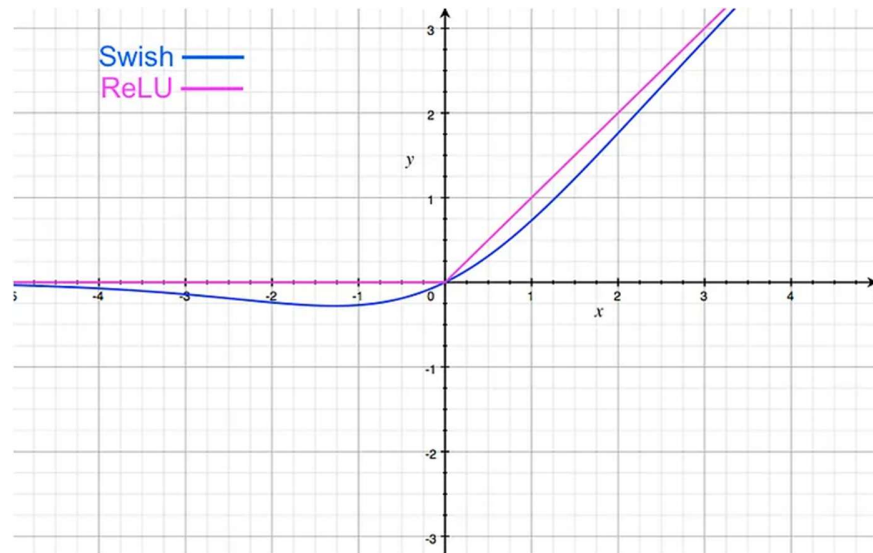
Trong đó, $W \in \mathbb{R}^{k_h \times k_w \times C_{in} \times C_{out}}$ là tensor trọng số có kích thước không gian $k_h \times k_w$, s là bước trượt (stride), và $b[k]$ là độ lệch (bias). Kích thước không gian của đầu ra được xác định bởi công thức: $H' = \left\lfloor \frac{H - k_h + 2p}{s} \right\rfloor + 1$ với p là lượng đệm viền.

2.5.1.2. Hàm kích hoạt phi tuyến

Sau mỗi phép tích chập tuyến tính, một hàm kích hoạt được áp dụng theo từng phần tử để đưa tính phi tuyến vào mô hình, cho phép mạng học được các hàm phức tạp. Mặc dù ReLU ($f(x) = \max(0, x)$) là hàm phổ biến nhất mang tính nền tảng, các kiến trúc hiện đại (bao gồm YOLOv8) ưu tiên sử dụng hàm SiLU:

$$f(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

SiLU khắc phục phần lớn hiện tượng chết nơ-ron của ReLU ở vùng âm, đồng thời tạo ra một bề mặt gradient mượt mà hơn, giúp mô hình hội tụ ổn định trong quá trình huấn luyện..

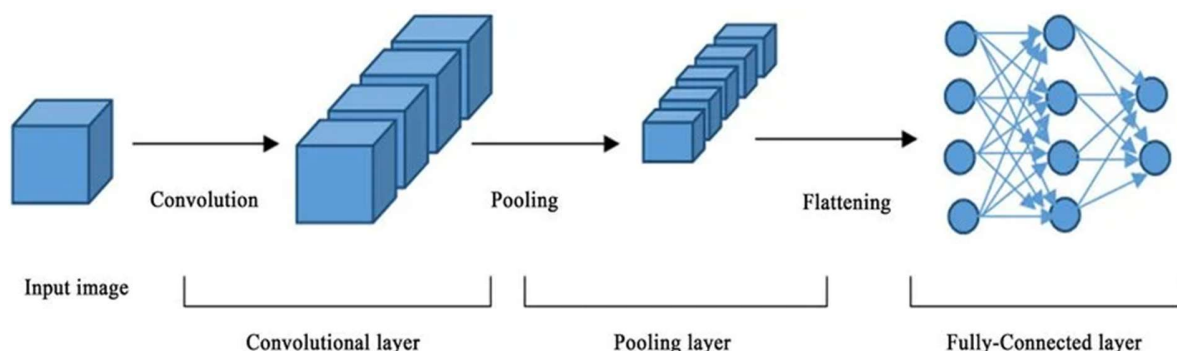


Hình 2.20. Đồ thị so sánh đặc tuyến của hàm kích hoạt ReLU và SiLU

2.5.1.3. Cơ chế giảm mẫu không gian

Để mở rộng trường tiếp nhận của mạng, cho phép các lớp sâu hơn nhìn thấy được cấu trúc vật thể lớn hơn, kích thước không gian (H, W) cần được giảm dần. Trong lý thuyết CNN kinh điển, Lớp gộp cực đại thường được sử dụng để lấy giá trị lớn nhất trong một cửa sổ $k \times k$. Tuy nhiên, trong kiến trúc backbone của YOLOv8n, kỹ thuật tích chập với bước trượt lớn ($stride = 2$) được ưu tiên sử dụng để giảm mẫu thay vì Pooling làm mất đi thông tin không gian một cách cơ học, tích chập với bước trượt lớn cho phép mạng tự học cách

loại bỏ thông tin thừa thông qua các trọng số, bảo toàn tốt hơn các đặc trưng ngữ nghĩa của biên báo giao thông.



Hình 2.21. Kiến trúc mạng CNN điển hình

2.5.2. Bài toán phát hiện đối tượng và cơ chế phân loại - định vị đồng thời

Phát hiện đối tượng yêu cầu mô hình giải đồng thời hai bài toán con. Định vị xác định vị trí và kích thước của hộp giới hạn chứa đối tượng, tham số hoá bởi một vector bốn chiều:

$$b = [x_c, y_c, w, h]^T \quad [35]$$

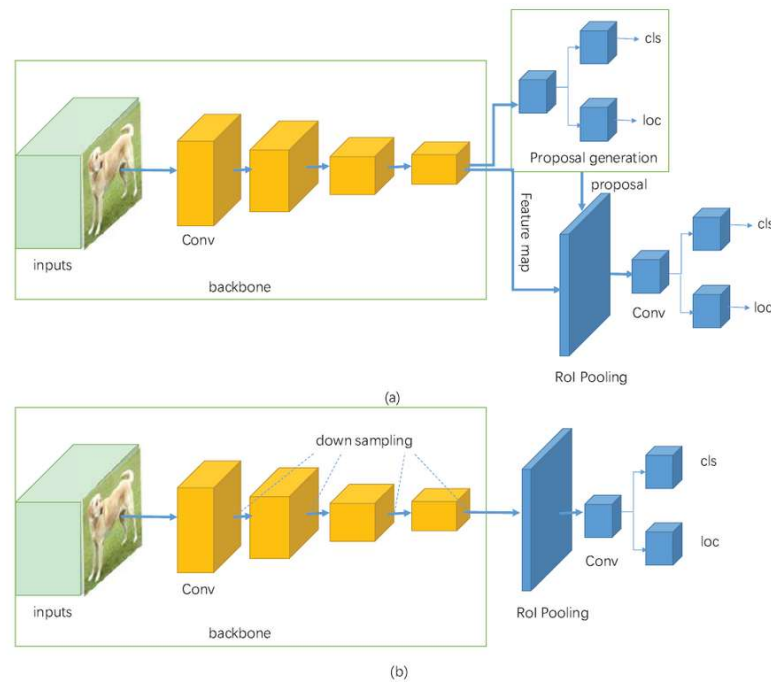
trong đó (x_c, y_c) là toạ độ tâm hộp, w và h là chiều rộng và chiều cao, thường được chuẩn hoá về $[0, 1]$ để không phụ thuộc kích thước ảnh đầu vào. Phân loại xác định danh tính ngữ nghĩa của đối tượng trong hộp đó, biểu diễn bằng một phân phối xác suất trên C lớp:

$$p = [p_1, \dots, p_C]^T, \quad p_i \in [0, 1] \quad [35]$$

đi kèm với một điểm tin cậy thể hiện xác suất vùng không gian đó thực sự chứa một đối tượng thuộc tập đã huấn luyện.

Hai họ kiến trúc giải bài toán này theo hai hướng ngược nhau. Các mô hình hai giai đoạn, điển hình là họ R-CNN, tách quy trình thành hai bước: một mạng đề xuất vùng trích ra các vùng ứng viên, sau đó một mạng phân loại riêng xử lý từng vùng. Cách này cho độ chính xác cao nhưng tốc độ chậm, vì phải xử lý qua nhiều vùng ứng viên liên tiếp. [34]

Các mô hình một giai đoạn, điển hình là họ YOLO, loại bỏ hẳn bước đề xuất vùng trung gian: chỉ cần một lần truyền thuận duy nhất để dự đoán đồng thời toạ độ b và nhãn p trên một lưới dự đoán dày đặc. Vì không phải lặp qua các vùng đề xuất, mô hình một giai đoạn có tốc độ suy luận cao hơn, đánh đổi một phần độ chính xác để phù hợp hơn với các bài toán thời gian thực trên phần cứng nhúng có tài nguyên hạn chế. [35]



Hình 2.22. So sánh kiến trúc tổng quát của mô hình phát hiện đối tượng một giai đoạn (YOLO) và hai giai đoạn (R-CNN)

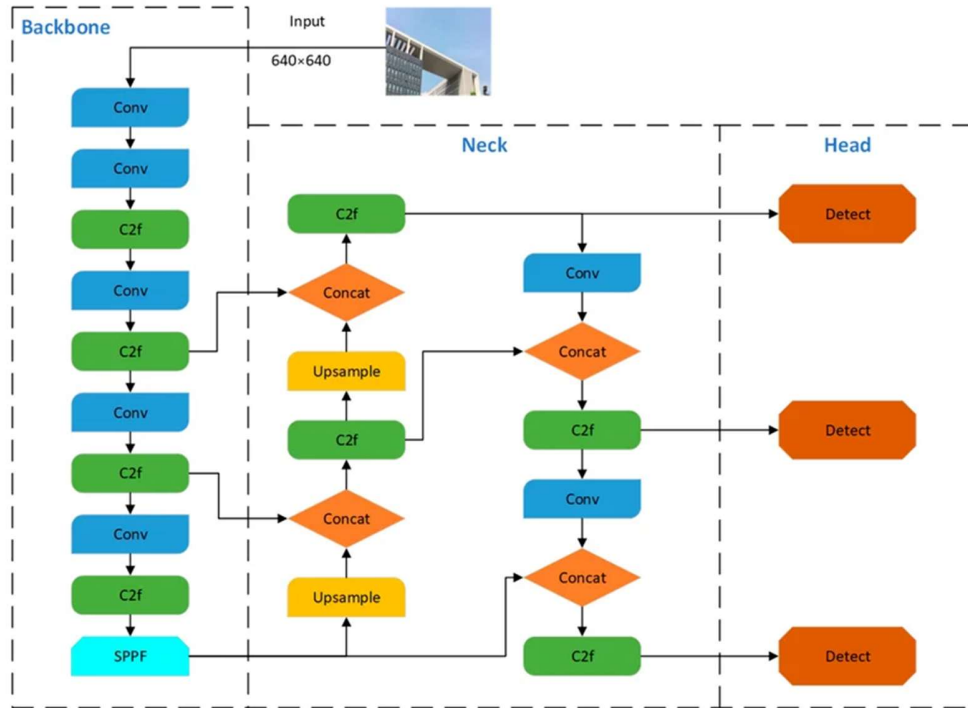
2.5.3. Kiến trúc YOLOv8n

YOLOv8 (Ultralytics, 2023) là thể hệ mô hình học sâu thuộc họ One-stage Detector. So với các phiên bản tiền nhiệm như YOLOv5, YOLOv7, YOLOv8 cải tiến đáng kể về kiến trúc thông qua khối trích xuất C2f, cấu trúc cổ FPN/PAN đa tỷ lệ, và đặc biệt là chuyển sang mô hình không sử dụng hộp neo. Trong họ YOLOv8, biến thể Nano (YOLOv8n) có số tham số nhỏ nhất (~3,2 triệu), phù hợp với các nền tảng vi xử lý ARM không có đơn vị xử lý đồ họa chuyên dụng, nơi cần đánh đổi một phần độ chính xác để đạt tần số xử lý thời gian thực. [36]

2.5.3.1. Backbone C2f và SPPF

Backbone chịu trách nhiệm trích xuất đặc trưng không gian từ ảnh đầu vào ở ba mức: P3 (bước trượt 8), P4 (bước trượt 16) và P5 (bước trượt 32). YOLOv8 dùng khối C2f thay cho khối C3 ở các phiên bản trước: luồng đặc trưng được chia thành một nhánh truyền thẳng và một nhánh đi qua chuỗi bottleneck, sau đó nối lại với nhau. Cách thiết kế này giúp gradient lan truyền ổn định hơn trong quá trình huấn luyện và giảm chi phí tính toán so với C3.

Hệ thống trích xuất đặc trưng ở ba mức không gian: P3 (bước trượt 8), P4 (bước trượt 16) và P5 (bước trượt 32). Ở tầng sâu nhất, khối SPPF hợp nhất thông tin từ nhiều kích thước trường tiếp nhận khác nhau mà không làm giảm tốc độ suy luận.



Hình 2.23. Lưu đồ cấu trúc mạng YOLOv8

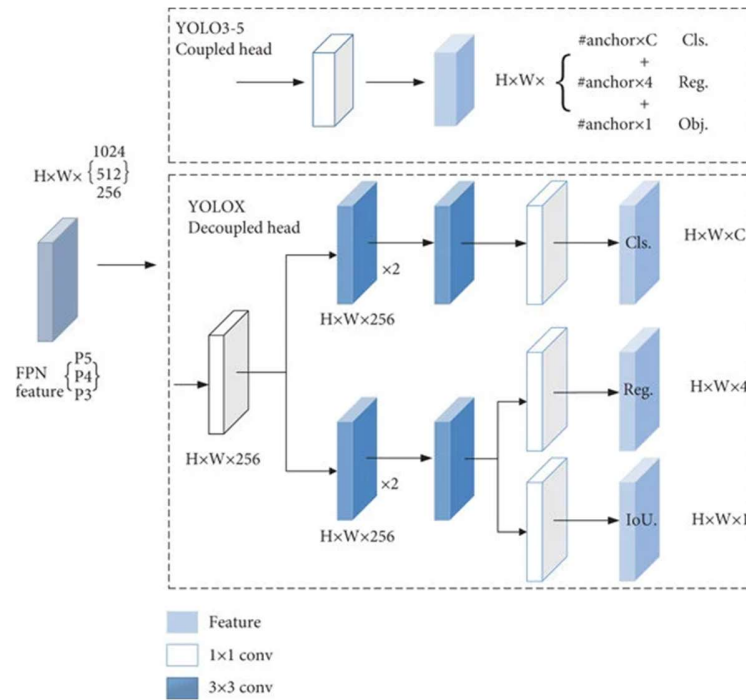
2.5.3.2. Neck FPN + PAN

Neck kết hợp thông tin giữa các tầng đặc trưng có độ phân giải khác nhau. FPN truyền thông tin ngữ nghĩa từ tầng sâu P5 lên các tầng nông P4, P3 qua phép nâng mẫu; PAN bổ sung một đường truyền ngược lại, từ tầng nông xuống tầng sâu, mang theo thông tin định vị chi tiết. Sự kết hợp hai chiều này giúp mô hình nhận diện tốt cả các biển báo nhỏ ở xa nhờ P3 có độ phân giải cao và các đối tượng lớn ở gần nhờ P5.

2.5.3.3. Đầu dự đoán tách biệt và Anchor-free

Khác với các phiên bản YOLO trước dùng một lớp tích chập chung cho cả định vị và phân loại, YOLOv8 sử dụng đầu tách biệt với hai nhánh độc lập: nhánh phân loại dự đoán xác suất cho từng lớp đối tượng bằng hàm mất Binary Cross-Entropy, và nhánh hồi quy không dùng các hộp neo định trước dự đoán trực tiếp tọa độ giới hạn bằng tổ hợp hàm mất CIoU và DFL nhằm mô hình hóa phân phối xác suất của tọa độ biên, để xử lý độ mờ ở ranh giới vật thể. Tách hai nhánh này giúp giải quyết xung đột giữa yêu cầu của tác

vụ định vị cần nhảy với tọa độ chính xác và tác vụ phân loại cần bắt biến với phép tịnh tiến nhỏ.



Hình 2.24. So sánh cấu trúc Coupled Head (bên trái) của các thế hệ trước và Decoupled Head (bên phải) được ứng dụng trên YOLOv8

2.5.4. Huấn luyện mô hình: học chuyển giao và tăng cường dữ liệu

2.5.4.1. Học chuyển giao

Huấn luyện một mạng như YOLOv8n từ đầu trên một tập dữ liệu nhỏ cỡ vài trăm đến vài nghìn ảnh mỗi lớp dễ dẫn đến quá khớp, mô hình ghi nhớ dữ liệu huấn luyện nhưng mất khả năng tổng quát hoá. Học chuyển giao giải quyết vấn đề này bằng cách khởi tạo trọng số từ một mô hình đã huấn luyện trước trên tập dữ liệu lớn và đa dạng như COCO, 80 lớp, 118.000 ảnh thay vì khởi tạo ngẫu nhiên: các lớp tích chập ở phần nông của backbone đã học được các đặc trưng thị giác phổ quát - cạnh, góc, độ tương phản - có giá trị tái sử dụng cho hầu hết bài toán thị giác máy tính khác. [37]

Khi áp dụng học chuyển giao, có hai chiến lược tinh chỉnh phổ biến: đóng băng backbone và chỉ huấn luyện lại phần đầu ra trong giai đoạn đầu - phù hợp khi dữ liệu mục tiêu gần giống phân phối dữ liệu gốc, hoặc giải đông toàn bộ trọng số ngay từ đầu để thuật toán tối ưu cập nhật gradient xuống tận các lớp sâu nhất - phù hợp khi dữ liệu mục tiêu khác biệt đáng kể so với dữ liệu đã huấn luyện trước.

2.5.4.2. Tăng cường dữ liệu

YOLOv8 tích hợp một số kỹ thuật tăng cường dữ liệu tự động trong quá trình huấn luyện. Mosaic ghép bốn ảnh ngẫu nhiên thành một khung hình duy nhất, buộc mô hình nhận diện đối tượng ở tỉ lệ nhỏ hơn và giảm phụ thuộc vào bối cảnh nền, đồng thời tăng số lượng hộp giới hạn xuất hiện trong mỗi batch. Biến dạng màu HSV nhiễu loạn ngẫu nhiên giá trị Hue, Saturation và Value mô phỏng sự thay đổi ánh sáng môi trường. Với các đối tượng mang tính định hướng như biển cấm rẽ hoặc biển STOP sẽ mất ý nghĩa nếu bị lật theo trục đứng, phép lật ngang thường bị tắt hoàn toàn. [38]

2.5.5. Các chỉ số đánh giá mô hình

2.5.5.1. Intersection over Union

IoU là thước đo cốt lõi để đánh giá mức độ chính xác định vị. IoU đo lường tỷ lệ chồng lấp giữa hộp giới hạn dự đoán (B_{pred}) và hộp nhãn thực tế (B_{gt}):

$$IoU(B_{pred}, B_{gt}) = \frac{|B_{pred} \cap B_{gt}|}{|B_{pred} \cup B_{gt}|}$$

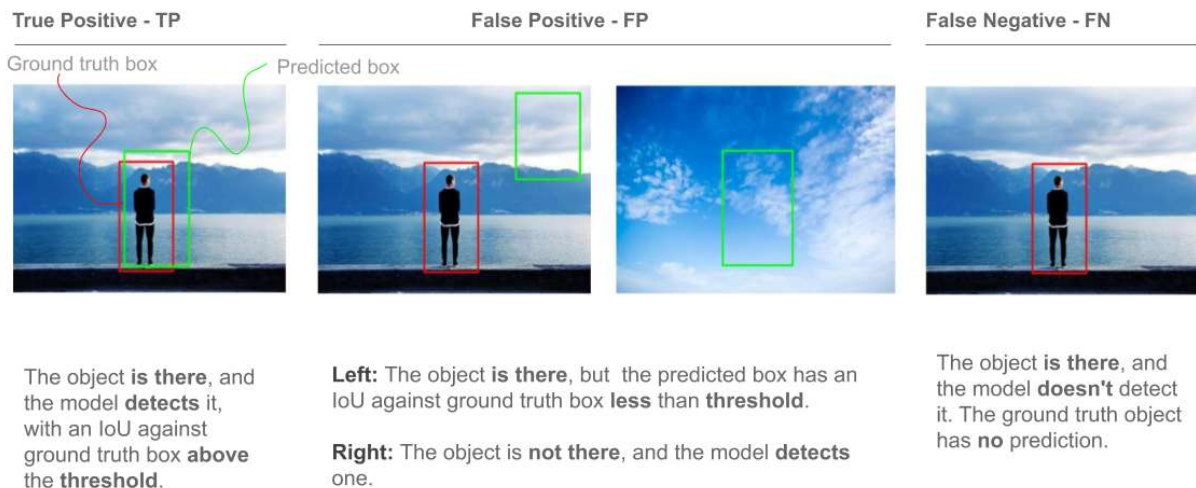
IoU nhận giá trị trong khoảng $[0, 1]$; $IoU = 1$ khi hai hộp trùng khít hoàn toàn. Hàm mất định vị CIOU mà YOLOv8 sử dụng mở rộng IoU thông thường bằng cách cộng thêm thành phần phạt cho sai khác về khoảng cách tâm và tỉ lệ khung giữa hai hộp, giúp mô hình hội tụ nhanh và cho ra hộp dự đoán khít hơn so với chỉ tối thiểu hoá $1 - IoU$.

2.5.5.2. Precision, Recall và F1-Score

Với một ngưỡng IoU cho trước thường là 0,5, mỗi dự đoán được gán nhãn True Positive nếu khớp đủ với một nhãn thật, False Positive nếu không khớp với nhãn thật nào, hoặc nhãn thật bị bỏ sót được tính là False Negative. Từ đó:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$
$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision phản ánh tỉ lệ dự đoán đúng trong số các dự đoán được đưa ra, còn Recall phản ánh tỉ lệ đối tượng thật được phát hiện. Điểm F1, là trung bình điều hoà của hai chỉ số, được dùng khi cần một con số duy nhất cân bằng giữa hai tiêu chí thường đối lập nhau này.



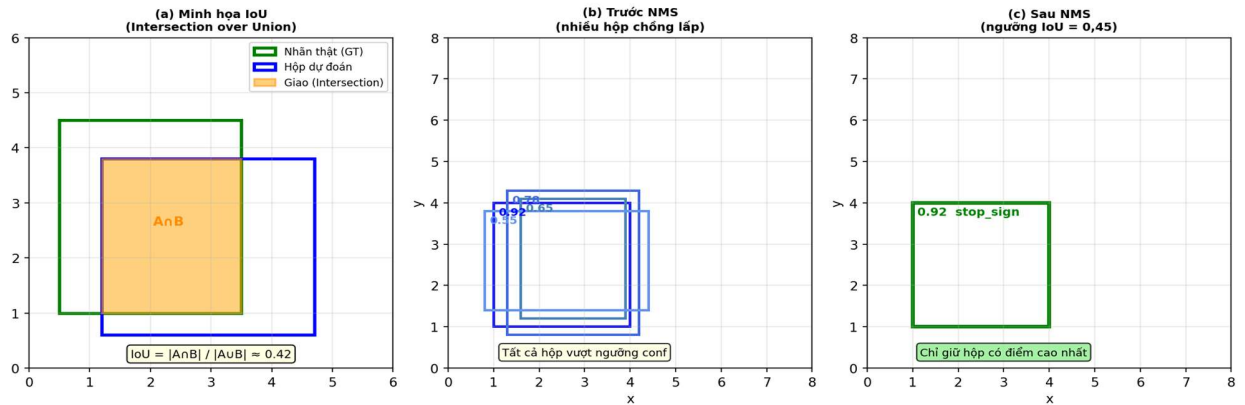
Hình 2.25. Phân loại các kết quả dự đoán (TP, FP, FN) dựa trên ngưỡng IoU trong bài toán phát hiện đối tượng

2.5.5.3. Mean Average Precision

Average Precision của một lớp là diện tích dưới đường cong Precision, Recall khi quét qua mọi ngưỡng tin cậy. mAP là trung bình AP trên tất cả các lớp. Chỉ số mAP@0,5 (IoU ngưỡng 0,5) thường dùng để theo dõi tiến trình huấn luyện; mAP@0,5:0,95: trung bình qua các ngưỡng IoU từ 0,5 đến 0,95 với bước 0,05, đánh giá nghiêm ngặt hơn vì đòi hỏi hộp dự đoán khít với nhãn thật ở nhiều mức độ chính xác khác nhau.

2.5.5.4. Thuật toán triệt phi cực đại - NMS

Một đối tượng thực tế thường nhận nhiều hộp dự đoán chồng lấp từ các vị trí lưới lân cận. NMS loại các hộp dư thừa theo thuật toán tham lam: sắp xếp các hộp giảm dần theo điểm tin cậy, giữ lại hộp có điểm cao nhất, loại bỏ mọi hộp còn lại có IoU với hộp vừa giữ vượt một ngưỡng cho trước, rồi lặp lại quá trình với các hộp còn sót. Ngưỡng này cần đủ thấp để triệt các hộp trùng lặp của cùng một vật thể, đồng thời đủ cao để không xoá nhầm các hộp của hai vật thể riêng biệt đứng gần nhau.



Hình 2.26. (a) Minh họa IoU giữa hộp nhãn thật (xanh lá) và hộp dự đoán (xanh dương); vùng cam là giao nhau. (b) Nhiều hộp chồng lấp trước NMS. (c) Kết quả sau NMS ($IoU_{NMS} = 0,45$): chỉ giữ hộp điểm cao nhất

2.5.6. Tối ưu hóa suy luận trên thiết bị biên

Việc triển khai trực tiếp các mô hình học sâu nguyên bản thường gắn liền với các framework phát triển như PyTorch hay TensorFlow lên thiết bị nhưng gặp rào cản lớn về tài nguyên phần cứng, đặc biệt là bộ nhớ RAM và năng lực tính toán. Để giải quyết vấn đề này, đồ thị tính toán của mô hình thường được chuyển đổi sang một định dạng trung gian độc lập framework, điển hình là ONNX. Định dạng này loại bỏ sự phụ thuộc vào các thư viện huấn luyện công kênh, cho phép suy luận thông qua một engine gọn nhẹ hơn và tận dụng các phép tối ưu hóa đồ thị tính toán như gộp lớp, loại bỏ nhánh không dùng đến thường không có sẵn khi chạy trực tiếp trên framework huấn luyện gốc. [39]

Quy trình suy luận của một mạng nơ-ron phát hiện đối tượng trên thiết bị nhưng thường tuân theo ba bước nối tiếp độc lập:

Tiền xử lý: Co giãn ảnh đầu vào về độ phân giải chuẩn của mạng sử dụng phương pháp bù viền - letterbox để không làm méo tỷ lệ hình học và chuẩn hóa phân phối cường độ điểm ảnh về miền giá trị xác định thường là $[0,1]$.

Lan truyền thuận: Đẩy ma trận ảnh đã chuẩn hóa qua toàn bộ đồ thị tính toán của mạng CNN để trích xuất các tensor dự đoán thô.

Hậu xử lý: Giải mã tensor đầu ra để tái tạo lại tọa độ không gian của các hộp giới hạn, kết hợp với thuật toán triệt tiêu phi cực đại để lọc bỏ các dự đoán trùng lặp, trả về kết quả cuối cùng với độ tin cậy cao nhất.

2.6. Cơ sở lý thuyết điều khiển

2.6.1. Bộ điều khiển PID và dạng rời rạc

PID là bộ điều khiển phản hồi vòng kín được ứng dụng rộng rãi nhất trong lĩnh vực tự động hóa nhờ kiến trúc đơn giản nhưng mang lại hiệu năng xác lập cao. Gọi $r(t)$ là giá trị tham chiếu, $y(t)$ là tín hiệu đo lường thực tế, sai lệch của hệ thống được định nghĩa là $e(t) = r(t) - y(t)$. Dựa trên sai lệch này, luật điều khiển PID dạng liên tục tổng quát được biểu diễn dưới dạng toán học như sau: [31]

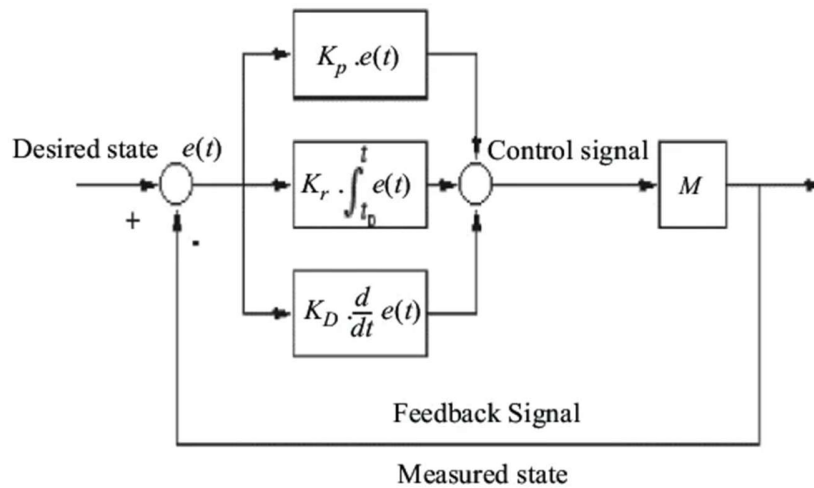
$$u(t) = K_p \cdot e(t) + K_i \cdot \int_0^t e(\tau) d\tau + K_d \cdot \frac{de(t)}{dt}$$

Ba khâu trong phương trình trên đóng vai trò tương hỗ lẫn nhau: khâu tỷ lệ (P) tạo ra phản ứng tức thời tỷ lệ thuận với biên độ sai lệch; khâu tích phân (I) cộng dồn sai lệch theo thời gian nhằm triệt tiêu hoàn toàn sai lệch tĩnh mà riêng khâu P không thể khắc phục, ví dụ: lực cản ma sát khiến phương tiện luôn di chuyển chậm hơn tham chiếu; khâu vi phân (D) tính toán độ dốc của sai lệch nhằm dự đoán xu hướng biến thiên, từ đó tạo ra lực hãm sớm để giảm thiểu độ vọt lố.

Bảng 2.5. Vai trò của ba khâu trong bộ điều khiển PID

Khâu	Tác dụng chính	Tác dụng phụ khi quá lớn
P	Tăng tốc độ đáp ứng tức thời theo sai lệch hiện tại	Gây ra hiện tượng vọt lố và dao động mạnh quanh giá trị tham chiếu
I	Triệt sai lệch tĩnh tích lũy	Gây bão hòa tích phân, làm chậm thời gian đáp ứng và tạo dao động tần số thấp
D	Dự đoán xu hướng, tạo lực hãm sớm để giảm vọt lố	Khuếch đại nhiễu đo lường tần số cao, gây dao động gắt ở tín hiệu đầu ra

Trong các vòng điều khiển tốc độ động cơ phản hồi qua cảm biến tốc độ chạy trên vi điều khiển với chu kỳ cố định, bộ điều khiển thường chỉ cần hai khâu P-I: tín hiệu đếm xung trong một cửa sổ thời gian ngắn vốn đã lượng tử hóa thô và nhiễu, nên đạo hàm của nó (khâu D) chủ yếu khuếch đại nhiễu thành rung PWM vô ích hơn là mang lại lợi ích dự đoán xu hướng. Đây là lý do nhiều ứng dụng điều khiển tốc độ động cơ trong thực tế chỉ dùng PI.



Hình 2.27. Cấu trúc bộ điều khiển PID vòng kín

2.6.2. Feedforward và kỹ thuật chống tràn tích phân

PID thuần phản hồi phải chờ sai lệch xuất hiện rồi mới phản ứng. Với động cơ DC, một lượng điều khiển tối thiểu luôn cần có chỉ để thắng ma sát tĩnh, phần này biết trước, không cần chờ sai lệch xuất hiện. Kỹ thuật feedforward cộng thẳng thành phần biết trước đó vào đầu ra, để PID chỉ phải bù phần sai lệch còn lại quanh điểm làm việc, giúp đáp ứng khởi động nhanh hơn và giảm gánh nặng cho khâu tích phân. [31]

Bão hòa tích phân là hiện tượng bất lợi đặc trưng của khâu I: khi cơ cấu chấp hành bão hòa hoặc động cơ bị giữ cứng, sai lệch tồn tại lâu làm tổng tích phân phình to; đến khi nguyên nhân biến mất, lượng tích phân lớn này tiếp tục đẩy đầu ra sai trong một khoảng thời gian trước khi xả hết. Biện pháp đơn giản và hiệu quả nhất là kẹp giới hạn tổng tích phân trong một khoảng cố định, đủ để xóa sai lệch tĩnh nhưng không đủ lớn để gây mất điều khiển.

2.7. Mô hình hóa và Ước lượng trạng thái quỹ đạo

2.7.1. Thuật toán nội suy điểm nhìn xa

Trong lĩnh vực điều khiển phương tiện tự hành, việc thiết lập quỹ đạo mục tiêu chỉ dựa trên vị trí hiện tại thường dẫn đến độ trễ trong đáp ứng động học, đặc biệt tại các đoạn đường cong. Để khắc phục hạn chế này, phương pháp nội suy điểm nhìn xa được áp dụng nhằm mô hình hóa không gian phía trước, cung cấp khả năng dự báo quỹ đạo cho hệ thống điều khiển. [32]

Về mặt hình học, mỗi đoạn thẳng biểu diễn vạch kẻ đường được nội suy thông qua một phương trình tuyến tính có dạng:

$$y = ax + b$$

Từ phương trình cơ sở, hoành độ x tại một độ sâu y bất kỳ trong không gian ảnh được xác định bằng hệ thức:

$$x = \frac{y - b}{a}$$

Phương pháp nội suy điểm nhìn xa không sử dụng trực tiếp tọa độ vật lý của các đoạn thẳng đứt khúc. Thay vào đó, thuật toán thực hiện phép chiếu các đường thẳng này lên hai mốc độ sâu tham chiếu: điểm gần (y_{base}) biểu thị vị trí liền kề phương tiện, và điểm xa (y_{ext}) biểu thị giới hạn tầm nhìn không gian. Hoành độ tương ứng tại hai mốc này được ký hiệu là x_{base} và x_{ext} .

Tọa độ nội suy cuối cùng (x_{proj}) được xác định thông qua tổ hợp tuyến tính với các hệ số trọng số:

$$x_{proj} = \alpha \cdot x_{ext} + \beta \cdot x_{base}$$

Trong đó, α và β là các hệ số phân bổ trọng số với điều kiện ràng buộc $\alpha + \beta = 1$. Hệ số α đặc trưng cho khả năng dự báo trước góc cua, trong khi β duy trì mức độ bám sát quỹ đạo thực tại. Cơ sở toán học này cho phép hệ thống điều khiển thiết lập một quỹ đạo ảo có tính liên tục, cải thiện thời gian đáp ứng của cơ cấu chấp hành so với việc chỉ dựa vào vị trí hiện tại.

2.7.2. Thuật toán tổng hợp quỹ đạo dựa trên Trung bình có trọng số

Dữ liệu đầu ra từ bước nội suy hình học là một tập hợp các vector đoạn thẳng rời rạc cấu thành giới hạn làn đường. Do ảnh hưởng của nhiễu quang học và khuyết tật trên bề mặt di chuyển, các đoạn thẳng này thường tồn tại sự phân tán lớn về chiều dài và độ tin cậy. Phép tính trung bình cộng thông thường bộc lộ sai số lớn khi đồng nhất mức độ đóng góp của các vạch nhiễu ngắn với các dải vạch liền mạch. [32]

Nhằm nâng cao độ tin cậy của bộ lọc trạng thái, thuật toán tổng hợp quỹ đạo ứng dụng hàm trung bình có trọng số. Tọa độ đại diện của làn đường (X_{lane}) được xác định bằng tổng có trọng số của các tọa độ nội suy (x_i), với biến trọng số là chiều dài không gian (L_i) của từng đoạn thẳng:

$$X_{lane} = \frac{\sum_{i=1}^n (x_i \cdot L_i)}{\sum_{i=1}^n L_i}$$

Trong đó, chiều dài L_i được tính toán theo khoảng cách Euclid giữa hai điểm đầu mút của đoạn thẳng:

$$L_i = \sqrt{(x_{end} - x_{start})^2 + (y_{end} - y_{start})^2}$$

Ý nghĩa thống kê của hàm toán học này là gán trọng số tín nhiệm cao cho các đặc trưng hình học rõ nét. Các đoạn thẳng có độ dài lớn mang thông tin quỹ đạo chủ đạo sẽ chi phối giá trị tọa độ đại diện, đồng thời suy giảm ảnh hưởng của các phân đoạn đứt gãy hoặc nhiễu cục bộ, giúp tăng độ ổn định của đầu ra so với phép tính trung bình cộng thông thường.

2.7.3. Mô hình toán học về Sai số lệch ngang

Trong lý thuyết điều khiển bám quỹ đạo, đại lượng cơ sở để đánh giá và duy trì trạng thái cân bằng động học là sai số lệch ngang. Đại lượng này biểu thị khoảng cách không gian giữa trục dọc tuyến tính của phương tiện và trục tâm lý tưởng của quỹ đạo tham chiếu. [32]

Trong hệ tọa độ của cảm biến thị giác đặt tại trục đối xứng của hệ thống, hoành độ tâm điểm ảnh (X_{car}) phản ánh phương vị thực tế của phương tiện. Tọa độ tâm làn đường lý tưởng (X_{lane}) được xác định từ vị trí của hai dải phân cách giới hạn (làn trái X_L và làn phải X_R):

$$X_{lane} = \frac{X_L + X_R}{2}$$

Trong điều kiện thiếu hụt dữ liệu (hệ thống thị giác chỉ trích xuất được một vạch kẻ biên), tọa độ tâm làn được suy diễn thông qua phép tính tiến dựa trên tham số độ rộng làn đường tiêu chuẩn (W_{lane}): $X_{lane} = X_L + \frac{W_{lane}}{2}$ hoặc $X_{lane} = X_R - \frac{W_{lane}}{2}$.

Nhằm đảm bảo tính độc lập của tín hiệu với độ phân giải cảm biến quang học và tương thích với các thuật toán điều khiển kinh điển, sai số lệch ngang $e(t)$ được chuẩn hóa về miền giá trị $[-1,1]$. Biểu thức chuẩn hóa là tỷ số giữa độ lệch tuyệt đối và bán kính chiều rộng khung hình ($W/2$):

$$e(t) = \frac{X_{lane} - X_{car}}{W/2}$$

Thông qua mô hình toán học này, trạng thái $e(t) = 0$ tương ứng với quỹ đạo hoàn hảo. Các trạng thái $e(t) \neq 0$ định lượng mức độ chệch hướng, cung cấp tín hiệu sai số $e(t)$ đầu vào cho khâu tính toán luật điều khiển bù trừ tại cơ cấu chấp hành.

2.7.4. Đặc tính nhiễu dao động trong hệ thống thị giác máy tính

Các thuật toán xử lý ảnh truyền thống thể hiện độ nhạy cảm cao đối với biến thiên của môi trường vật lý. Dưới tác động của sự thay đổi quang độ đột ngột, hiện tượng lóa sáng hoặc bề mặt di chuyển không đồng nhất, các đặc trưng hình học giả thường bị kích hoạt. [10]

Hậu quả của hiện tượng này là sự thiếu ổn định của tọa độ vạch kẻ đường giữa các chu kỳ lấy mẫu liên tiếp, sinh ra tín hiệu nhiễu tần số cao được thuật ngữ hóa là nhiễu dao động. Nhiễu dao động làm sai số $e(t)$ biến thiên với đạo hàm biên độ lớn. Đối với các bộ điều khiển có thành phần vi phân, tín hiệu nhiễu sẽ bị khuếch đại, gây ra hiện tượng dao động cường bức tại cơ cấu chấp hành và ảnh hưởng đáng kể đến tính ổn định động lực học của phương tiện.

2.7.5. Kỹ thuật Lọc đột biến biên độ

Nhằm cô lập cơ cấu chấp hành khỏi các xung nhiễu dao động tần số cao, một khâu lọc đột biến biên độ được tích hợp trong chuỗi xử lý trạng thái. Nguyên lý thiết kế của bộ lọc này căn cứ trên ràng buộc vật lý về giới hạn động học của phương tiện: trong một khoảng thời gian lấy mẫu vi mô của cảm biến (Δt), biến thiên sai số quỹ đạo không thể vượt qua một ngưỡng gia tốc không gian xác định. [10]

Thuật toán được mô hình hóa thông qua phép tính đạo hàm bậc nhất rời rạc của sai số theo thời gian:

$$\Delta e = |e(t) - e(t - 1)|$$

Hệ thống đối chiếu giá trị Δe với một ngưỡng động lực học cực đại (θ_{max}). Khi vi phạm điều kiện giới hạn ($\Delta e > \theta_{max}$), hệ thống xác định mẫu dữ liệu hiện tại là nhiễu tích cực giả. Khâu lọc sẽ thực thi cơ chế ngắt bỏ tín hiệu quỹ đạo của chu kỳ này, giả định phương tiện tạm thời mất dấu tham chiếu. Phản ứng loại trừ này tận dụng quán tính động lực học của phương tiện, ngăn chặn các lệnh rẽ lái đột biến và duy trì hướng di chuyển an toàn cho đến khi tín hiệu xác thực được khôi phục.

2.7.6. Cơ sở toán học của luật điều khiển bám làn phi tuyến có vùng chết

Trong bài toán điều khiển phương tiện tự hành bám quỹ đạo dựa trên thị giác máy tính, việc áp dụng các luật điều khiển tuyến tính thông thường thường dẫn đến hai hạn chế động học đối nghịch: phương tiện dễ bị dao động liên tục khi di chuyển trên đường thẳng do nhạy cảm với nhiễu vi mô, hoặc phản hồi thiếu cục bộ dẫn đến hiện tượng thiếu lái khi tiến vào các góc cua gắt. Để cải thiện chất lượng đáp ứng động học trong cả hai trường hợp, một luật điều khiển phi tuyến kết hợp vùng chết thường được áp dụng thay cho luật tuyến tính thông thường. [31]

Vùng chết là một khoảng sai số đối xứng quanh trục tham chiếu lý tưởng, trong đó mọi biến động nhỏ của tín hiệu sai số bị triệt tiêu để bảo vệ cơ cấu chấp hành khỏi dao động cường bức tần số cao. Sai số hiệu dụng $e_{eff}(t)$ sau khi đi qua vùng chết được mô hình hóa toán học như sau:

$$e_{eff}(t) = \begin{cases} 0 & \text{nếu } |e(t)| \leq e_{dead} \\ e(t) - \text{sgn}(e(t)) \cdot e_{dead} & \text{nếu } |e(t)| > e_{dead} \end{cases}$$

Trong đó:

$e(t)$: Tín hiệu sai số lệch ngang thô đã được chuẩn hóa tại thời điểm t .

e_{dead} : Ngưỡng giới hạn của không gian vùng chết thực nghiệm.

$\text{sgn}(\cdot)$: Hàm dấu để bảo toàn chiều bẻ lái.

Vượt ra ngoài vùng chết, góc lái chỉ lệnh $\theta_{raw}(t)$ được nội suy theo hàm lũy thừa phi tuyến nhằm điều chỉnh linh hoạt hệ số khuếch đại theo biên độ sai số:

$$\theta_{raw}(t) = K \cdot \text{sgn}(e_{eff}(t)) \cdot |e_{eff}(t)|^p$$

Trong đó:

K : Hệ số khuếch đại tỷ lệ cơ bản của hệ thống.

p : Tham số bậc lũy thừa phi tuyến ($p > 1$).

Khi sai lệch quỹ đạo nhỏ, bậc lũy thừa giữ hệ số góc của hàm truyền ở mức thấp, sinh ra lệnh bẻ lái biên độ nhỏ, giúp xe di chuyển êm. Khi sai lệch lớn ở các cua gắt, thành phần lũy thừa khuếch đại phi tuyến biên độ góc lái, tạo đủ mô-men để xe bám sát quỹ đạo hình học của sa hình.

2.8. Lập trình đa luồng trong hệ thống thị giác thời gian thực

2.8.1. Bài toán: các tác vụ có nhịp thời gian khác nhau

Một hệ thống thị giác máy tính thời gian thực thường phải vận hành đồng thời nhiều tác vụ với nhịp hoạt động khác nhau: thu khung hình từ cảm biến, xử lý đặc trưng hình ảnh, suy luận mạng nơ-ron, và phát lệnh điều khiển.

Nếu toàn bộ các tác vụ này được thực hiện tuần tự trong một vòng lặp duy nhất, tác vụ chậm nhất sẽ quyết định nhịp của cả hệ thống. Một lần suy luận kéo dài có thể làm gián đoạn luồng xử lý ảnh, khiến lệnh điều khiển đến trễ. Nếu tầng chấp hành phía dưới có cơ chế giám sát mất kết nối, độ trễ này còn có thể bị hiểu nhầm thành sự cố và kích hoạt dừng an toàn không cần thiết.

Giải pháp cho vấn đề này là tách mỗi tác vụ vào một luồng thực thi độc lập, cho phép hệ điều hành lập lịch và phân bổ các luồng song song trên nhiều nhân xử lý, thay vì để các tác vụ tranh chấp tài nguyên trong một vòng lặp tuần tự duy nhất.

2.8.2. Mô hình sản xuất và tiêu thụ cùng các cơ chế đồng bộ hóa

Cách tổ chức phổ biến cho các luồng độc lập kiểu trên là mô hình sản xuất và tiêu thụ. Một luồng đóng vai trò sản xuất, liên tục đưa dữ liệu mới vào một vùng đệm dùng chung; một hoặc nhiều luồng khác đóng vai trò tiêu thụ, đọc dữ liệu từ vùng đệm đó để xử lý tiếp. Một luồng tiêu thụ ở tầng này có thể đồng thời là luồng sản xuất cho tầng kế tiếp, tạo thành một chuỗi xử lý nhiều tầng cho đến khi dữ liệu được tổng hợp thành quyết định cuối cùng và phát ra ngoài.

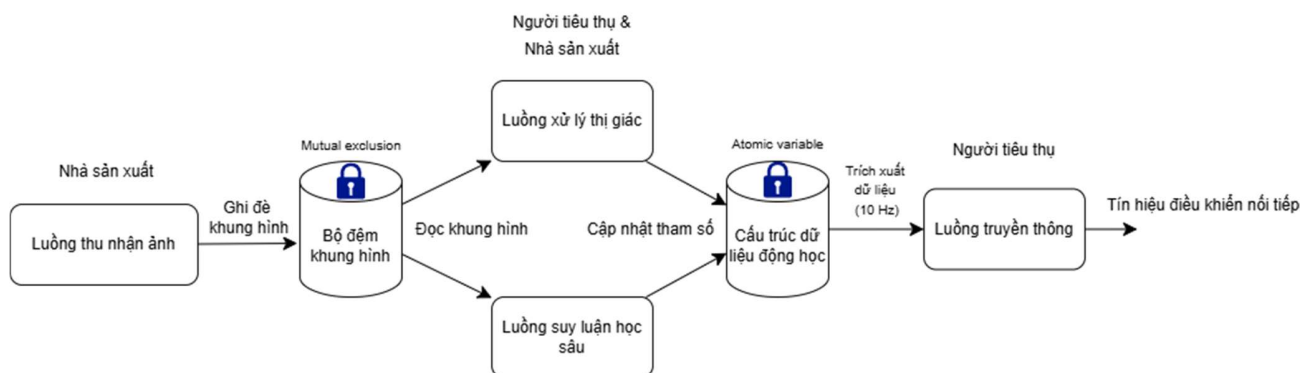
Đặc thù của một hệ thống thời gian thực mềm đòi hỏi vùng đệm chia sẻ phải có độ sâu chỉ gồm một phần tử và áp dụng cơ chế ghi đè: nếu luồng tiêu thụ xử lý không kịp, dữ liệu cũ bị thay bằng dữ liệu mới nhất chứ không xếp hàng chờ. Nguyên lý ở đây là một mẫu dữ liệu phản ánh hiện trạng luôn có giá trị sử dụng cao hơn một mẫu dữ liệu đã cũ, nên xếp hàng dữ liệu lỗi thời chỉ làm tăng độ trễ tích lũy mà không mang lại lợi ích nào.

Để bảo vệ tính toàn vẹn của dữ liệu chia sẻ giữa các luồng và tránh hiện tượng kết quả tính toán sai lệch do nhiều luồng cùng truy xuất một vùng nhớ tại một thời điểm, có ba cơ chế đồng bộ hóa thường được áp dụng:

- Mutual exclusion (loại trừ tương hỗ): đảm bảo tại một thời điểm chỉ một luồng được giữ quyền truy cập vùng nhớ dùng chung. Mọi thao tác đọc và ghi lên vùng đệm

hoặc cấu trúc dữ liệu chia sẻ cần được thực hiện trong vùng được khóa bảo vệ. Nguyên tắc quan trọng cần tuân thủ là giữ khóa trong thời gian ngắn nhất có thể, chỉ đủ để sao chép dữ liệu, không thực hiện các phép tính phức tạp khi đang giữ khóa.

- Condition variable (biến điều kiện): cho phép luồng tiêu thụ chuyển sang trạng thái chờ thay vì liên tục kiểm tra vòng lặp, vốn gây lãng phí tài nguyên xử lý. Luồng sản xuất phát tín hiệu đánh thức ngay khi có dữ liệu mới. Trạng thái chờ thường được giới hạn bằng một ngưỡng thời gian, để luồng vẫn có thể định kỳ kiểm tra các cờ điều khiển và kết thúc an toàn khi cần.
- Atomic variable (biến nguyên tử): dùng cho các giá trị đơn lẻ cần cập nhật độc lập, ví dụ góc lái hay vận tốc tham chiếu. Phép đọc/ghi nguyên tử ở cấp phần cứng bảo đảm giá trị không bị đọc nửa cũ nửa mới khi hai luồng truy xuất đồng thời, mà không cần dùng khóa, phù hợp để truyền các giá trị nhỏ giữa luồng ra quyết định và luồng truyền thông.



Hình 2.28. Mô hình đa luồng producer - consumer của phần mềm điều khiển

Đối với các tác vụ cần giữ nhịp phát tín hiệu ổn định trong thời gian dài, kỹ thuật định thời tuyệt đối thường được ưu tiên hơn so với việc cho luồng ngủ một khoảng thời gian tương đối sau mỗi vòng lặp. Cách ngủ tương đối dễ tích lũy sai số qua nhiều vòng lặp liên tiếp, do thời gian thực thi của các dòng lệnh giữa hai lần ngủ không cố định. Định thời tuyệt đối khắc phục điều này bằng cách neo mỗi lần ngủ vào một mốc thời gian tuyệt đối trong tương lai, giúp nhịp phát tín hiệu giữ ổn định trong suốt quá trình vận hành dài hạn.

2.8.3. Đa nhiệm thời gian thực trên vi điều khiển

Tại Khác với bộ xử lý trung tâm chạy hệ điều hành Linux đầy đủ, vi điều khiển ở tầng chấp hành thường vận hành dưới một hệ điều hành thời gian thực gọn nhẹ. RTOS chia

chương trình thành nhiều tác vụ độc lập, mỗi tác vụ có ngăn xếp và trạng thái thực thi riêng, được bộ lập lịch của nhân hệ điều hành cấp quyền thực thi xen kẽ dựa trên mức độ ưu tiên đã khai báo.

Khi một tác vụ có độ ưu tiên cao hơn trở nên sẵn sàng thực thi, ví dụ đến đúng chu kỳ định kỳ hoặc nhận được tín hiệu chờ, bộ lập lịch lập tức tạm dừng tác vụ đang chạy có độ ưu tiên thấp hơn để nhường quyền xử lý. Cơ chế này gọi là lập lịch ưu tiên có thể chiếm dụng, đảm bảo các tác vụ đòi hỏi tính thời gian thực nghiêm ngặt, chẳng hạn vòng điều khiển tốc độ động cơ, luôn được thực thi đúng chu kỳ dù vi điều khiển đồng thời đang xử lý các tác vụ kém khẩn cấp hơn như tiếp nhận dữ liệu nối tiếp.

Trên các vi điều khiển có nhiều nhân xử lý vật lý, RTOS còn cho phép gán cố định mỗi tác vụ vào một nhân riêng. Cách phân bổ này tách không gian thực thi của các tác vụ có yêu cầu thời gian khác nhau, ví dụ dành một nhân cho vòng lặp điều khiển tần số cao và một nhân khác cho các tác vụ giao tiếp ít khẩn cấp hơn, hạn chế tình trạng một tác vụ chiếm dụng CPU làm trễ tác vụ còn lại.

Song song với cơ chế đa nhiệm ở tầng phần mềm, các sự kiện ngoại vi có tần số rất cao như xung từ cảm biến mã hóa vòng quay vẫn cần xử lý bằng ngắt cứng như đã trình bày ở Mục 2.3.3, vì ngay cả tác vụ ưu tiên cao nhất của RTOS cũng có độ trễ phản hồi cỡ vài đến vài chục micro giây, không đủ nhanh để bắt kịp các xung tồn tại trong khoảng thời gian ngắn hơn mức đó. Mô hình phân tầng quen thuộc trên vi điều khiển nhúng kết hợp cả ba lớp: ngắt cứng xử lý các sự kiện tần số cao nhất, tác vụ RTOS ưu tiên cao đảm nhiệm vòng điều khiển định kỳ, và tác vụ ưu tiên thấp hơn xử lý các công việc còn lại.

Chương 3. THIẾT KẾ HỆ THỐNG XE TỰ HÀNH

3.1. Yêu cầu hệ thống

Trước khi thiết kế chi tiết, các yêu cầu của hệ thống cần được xác định rõ ràng, vì đây là một hệ thống cơ điện tử tích hợp nhiều tầng phần cứng và phần mềm. Các yêu cầu của đề tài được chia thành ba nhóm: yêu cầu chức năng, yêu cầu phi chức năng và yêu cầu an toàn. Ba nhóm này sẽ được dùng làm cơ sở đánh giá mức độ hoàn thiện của hệ thống ở Chương 4.

3.1.1. Yêu cầu chức năng

Yêu cầu chức năng mô tả những gì hệ thống phải thực hiện được. Đối với mô hình xe tự hành trong đề tài, các yêu cầu chức năng cốt lõi được xác định như sau:

- Thu nhận hình ảnh liên tục từ camera phía trước với độ phân giải 320×240 pixel, định dạng màu RGB888, tốc độ tối thiểu 15 FPS trong điều kiện chiếu sáng nhân tạo.
- Phát hiện vạch kẻ làn đường trên bề mặt sa hình (nền tối, vạch trắng) và xác định vị trí tâm làn đường so với trục dọc của xe.
- Tính toán góc lái điều chỉnh dựa trên độ lệch giữa tâm làn đường và trục xe, biểu diễn dưới dạng giá trị chuẩn hóa trong dải $[-100, +100]$.
- Truyền lệnh điều khiển gồm góc lái và tốc độ tới bộ vi điều khiển ESP32 qua giao thức UART với tần suất ổn định 20 Hz.
- Điều khiển tốc độ hai động cơ DC độc lập thông qua mạch cầu H L298N bằng tín hiệu PWM, đảm bảo xe tiến thẳng và vào cua theo lệnh.
- Điều khiển góc quay của servo lái trong phạm vi 8° - 128° tương ứng với cơ cấu lái Ackermann của mô hình, với vị trí trung tâm hiệu chỉnh tại 68° .
- Cung cấp giao diện điều khiển thủ công qua mạng WiFi nội bộ để kiểm tra phần cứng và thu thập dữ liệu huấn luyện, với khả năng theo dõi telemetry thời gian thực.
- Phản ứng với các tình huống đặc biệt: đường cong, mất một vạch, hoặc không nhận dạng được làn đường, bằng chiến lược điều khiển phù hợp cho từng trường hợp.

3.1.2. Yêu cầu phi chức năng

Yêu cầu phi chức năng định nghĩa các ràng buộc về chất lượng và điều kiện hoạt động mà hệ thống phải đáp ứng bên cạnh các chức năng chính. Nhóm yêu cầu này ảnh hưởng trực tiếp đến quyết định lựa chọn kiến trúc hệ thống.

3.1.2.1. Hiệu năng thời gian thực:

Toàn bộ pipeline từ lúc camera chụp được frame đến khi lệnh điều khiển được phát ra qua UART phải hoàn thành trong vòng 100 ms tương đương 10 FPS. Ràng buộc này đòi hỏi hệ thống phân bổ tài nguyên hợp lý trên máy tính nhúng: tác vụ băm lán đường dùng các thuật toán xử lý ảnh truyền thống được tinh chỉnh, trong khi tác vụ nhận dạng ngữ nghĩa dùng mạng nơ-ron học sâu với khối lượng tham số được giảm tối thiểu.

3.1.2.2. Độ ổn định giao tiếp:

Kênh UART giữa Raspberry Pi và ESP32 phải truyền lệnh không gián đoạn, với tỷ lệ mất gói dưới 1% trong suốt phiên thử nghiệm 30 phút. Luồng UART chạy độc lập với luồng xử lý ảnh để tần suất gửi lệnh 10 Hz không bị ảnh hưởng bởi thời gian xử lý frame biến động.

3.1.2.3. Khả năng mở rộng và bảo trì:

Kiến trúc phần mềm được thiết kế theo hướng mô-đun hóa, phân tách rõ luồng thuật toán cổ điển và luồng suy luận học sâu. Sự cô lập về không gian bộ nhớ và luồng thực thi cho phép mỗi tác vụ được kiểm thử, gỡ lỗi độc lập, đồng thời tạo điều kiện nâng cấp mô hình AI hoặc tích hợp thêm cảm biến phụ trợ về sau mà không phá vỡ logic cốt lõi.

3.1.2.4. Tương thích phần cứng:

Toàn bộ hệ thống điện tử hoạt động ở mức logic 3,3V, tương thích với ESP32 và Raspberry Pi. Nguồn 5V và 12V được phân tách rõ ràng, không có điểm chung trừ đất chung.

3.1.3. Yêu cầu an toàn và độ tin cậy

Một hệ thống điều khiển di chuyển vật lý, dù ở quy mô nhỏ, cũng cần được thiết kế với an toàn là tiêu chí hàng đầu. Các yêu cầu an toàn được xác định ở cả cấp phần cứng và phần mềm:

- Cơ chế dừng khẩn cấp bằng phần mềm: vi điều khiển chấp hành liên tục giám sát chu kỳ nhận lệnh. Nếu vượt quá thời gian quy định mà không nhận được gói tin mới

từ máy tính trung tâm, bộ điều khiển tự động đưa tín hiệu PWM về 0 để dừng động cơ dẫn động.

- Giới hạn hành trình cơ cấu chuyển hướng: góc quay của động cơ servo được ràng buộc trong dải an toàn từ 8° đến 128° trước khi xuất tín hiệu điều khiển, ngăn động cơ quay vượt hành trình cơ học cho phép, nguyên nhân chính gây kẹt trục và hư hỏng thước lái.
- Giới hạn vận tốc cực đại: vận tốc mục tiêu của phương tiện được khóa ở ngưỡng tối đa 30 xung encoder trên mỗi chu kỳ lấy mẫu 50 mili giây tương đương vận tốc tuyến tính khoảng 0,3 đến 0,5 m/s. Mức vận tốc này đảm bảo hệ thống thị giác có đủ thời gian xử lý và phản ứng an toàn trong không gian sa hình khép kín.
- Cách ly nguồn điện động lực: nguồn 12V cho động cơ truyền lực được thiết kế không tiếp xúc trực tiếp với các vi mạch logic. Mạch cầu H trong hệ thống có sẵn diode xả dòng ngược để ngăn xung nhiễu điện từ dội ngược từ cuộn dây động cơ về đường nguồn 5V của vi xử lý trung tâm.
- Ràng buộc khâu tích phân của bộ điều khiển: biến tích lũy sai số trong bộ điều khiển tốc độ PID được giới hạn trong dải $[-200, +200]$, nhằm tránh bão hòa khâu tích phân khi bánh xe bị chặn cơ học, giúp hệ thống không mất kiểm soát khi trở lại trạng thái hoạt động bình thường.

3.2. Kiến trúc tổng thể hệ thống

3.2.1. Phân tích và lựa chọn kiến trúc

Một trong những quyết định thiết kế quan trọng của đề tài là lựa chọn kiến trúc hệ thống xử lý. Hai phương án được cân nhắc:

Phương án 1, kiến trúc tập trung: sử dụng duy nhất Raspberry Pi 4 để thực hiện toàn bộ tác vụ, từ xử lý ảnh, tính toán lệnh, đến phát PWM điều khiển động cơ qua chân GPIO trực tiếp. Phương án này đơn giản về phần cứng nhưng có nhược điểm nghiêm trọng: hệ điều hành Linux trên Raspberry Pi không phải RTOS, không đảm bảo thời gian phát xung PWM chính xác khi hệ thống bận xử lý ảnh. Hơn nữa, GPIO của Raspberry Pi không có đủ kênh PWM phần cứng để điều khiển hai kênh motor và một servo đồng thời ở độ phân giải 8-bit, tần số 5 kHz.

Phương án 2, kiến trúc phân tán Master - Slave (được chọn): phân tách chức năng giữa hai vi xử lý chuyên biệt. Raspberry Pi 4 đóng vai trò master, xử lý toàn bộ tác vụ tính toán nặng (xử lý ảnh, AI). ESP32 đóng vai trò slave, nhận lệnh qua UART và thực thi điều khiển thời gian thực. Sự phân tách này giải quyết bài toán xung đột tài nguyên: luồng suy luận AI mất 50-100 ms/khung hình, biến động không còn ảnh hưởng đến vòng điều khiển PID động cơ chạy cứng nhịp 50 ms trên ESP32.

Bảng 3.1 so sánh hai phương án theo các tiêu chí kỹ thuật chính, làm rõ hơn lý do chọn kiến trúc phân tán.

Bảng 3.1. So sánh hai phương án kiến trúc hệ thống điều khiển

Tiêu chí	Kiến trúc tập trung (RPi)	Kiến trúc phân tán (RPi + ESP32)
Điều khiển PWM thời gian thực	Không đảm bảo, Linux không phải RTOS	Đảm bảo, ESP32 bare-metal, LEDC timer cứng
Xử lý AI và ảnh	Đủ năng lực, ARM A72, 4 GB RAM	Đủ năng lực, phân cho RPi
Số kênh PWM phần cứng	2 kênh, hạn chế, cần pigpio	16 kênh LEDC + 2 MCPWM, dư dả
Chi phí linh kiện	Thấp hơn (1 chip)	Cao hơn nhưng không đáng kể (~150k VNĐ)
Độ phức tạp tích hợp	Thấp	Trung bình, cần thiết kế UART protocol
An toàn khi RPi bị treo	Không có fallback, xe mất điều khiển	ESP32 timeout → tự dừng an toàn

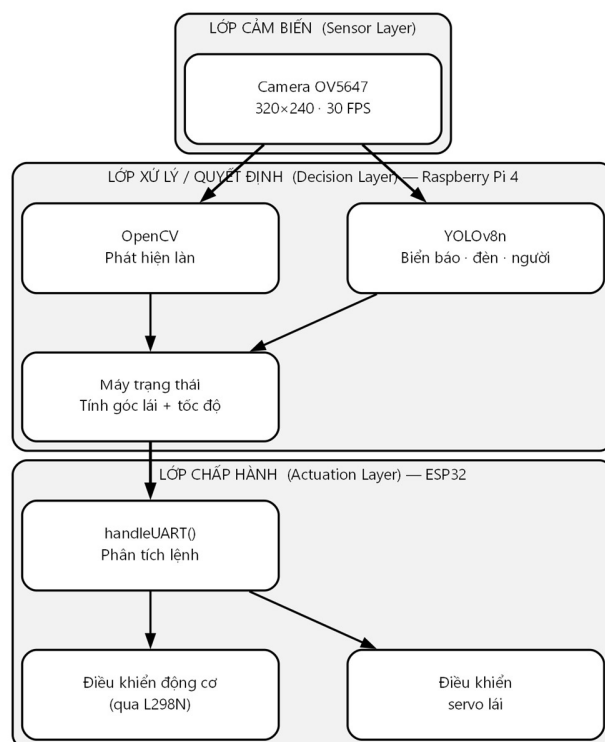
3.2.2. Sơ đồ khối tổng thể

Hình 3.1 thể hiện kiến trúc tổng quát của hệ thống, chia thành ba lớp: lớp cảm biến, lớp xử lý / quyết định, và lớp chấp hành.

Luồng dữ liệu giữa ba lớp này theo mô hình một chiều từ trên xuống. Lớp cảm biến thu nhận và cung cấp dữ liệu hình ảnh thô cho lớp xử lý. Tại đây, các thuật toán phân tích trích xuất đặc trưng môi trường, đưa ra quyết định tại máy trạng thái, rồi truyền lệnh điều

kiểm soát lớp chấp hành. Lớp chấp hành nhận các lệnh này và phân phối tín hiệu điều khiển trực tiếp đến động cơ và servo.

Hệ thống hoạt động ở chế độ vòng hở đối với quỹ đạo tổng thể. Vòng phản hồi kín duy nhất nằm hoàn toàn trong lớp chấp hành, là bộ điều khiển tốc độ PID chạy trên ESP32, dùng tín hiệu từ encoder ở bánh sau để bù sai số, giữ vận tốc thực tế của xe bám sát giá trị tham chiếu từ lớp xử lý đưa xuống.



Hình 3.1. Sơ đồ khối tổng thể kiến trúc hệ thống xe tự hành

3.2.3. Luồng dữ liệu và giao tiếp giữa các khối

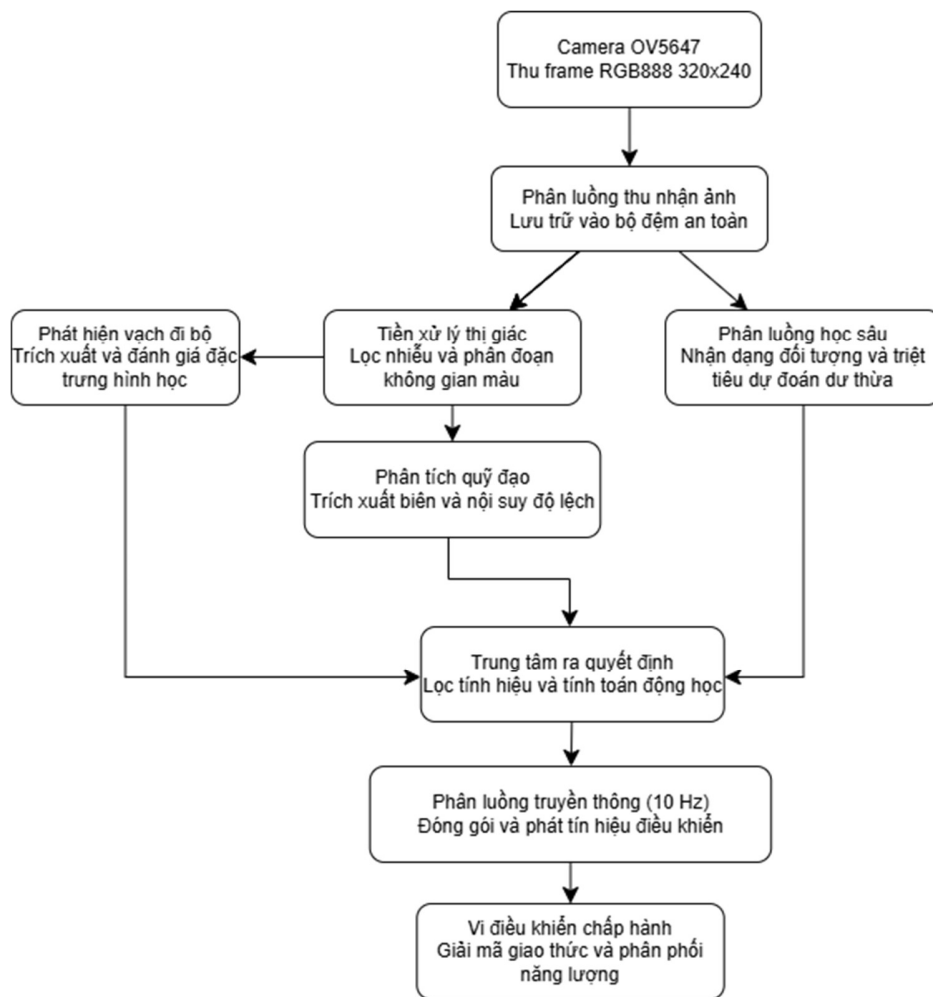
Luồng dữ liệu cốt lõi của hệ thống vận hành theo một chu trình khép kín và liên tục giữa các khối xử lý. Bắt đầu từ lớp cảm biến, khung hình ảnh màu được thu nhận ở độ phân giải 640×480, cao hơn độ phân giải xử lý thực tế nhằm tránh hiện tượng cắt xén trường nhìn của trình điều khiển camera, sau đó được hạ mẫu về 320×240 trước khi đưa vào các bước xử lý tiếp theo. Quá trình thu nhận này được tách thành một luồng riêng, có nhiệm vụ đẩy khung hình vào một bộ đệm dùng chung được bảo vệ bằng khóa loại trừ tương hỗ, nhằm tránh xung đột khi các luồng khác truy xuất đồng thời.

Từ bộ đệm dữ liệu, luồng thông tin được rẽ thành hai nhánh xử lý song song. Nhánh thứ nhất là luồng thị giác máy tính, thực hiện tuần tự các bước lọc nhiễu, phân đoạn không

gian màu và trích xuất mặt nạ nhị phân để nhận diện làn đường và vạch đi bộ. Nhánh thứ hai là luồng suy luận học sâu, hoạt động với chu kỳ lấy mẫu thưa hơn, chỉ xử lý một phần năm số khung hình từ luồng thị giác nhằm giảm tải tính toán. Luồng này thực thi mạng nơ-ron để nhận dạng đối tượng, từ đó cung cấp tín hiệu định hướng cho các kịch bản giao thông.

Vòng lặp điều khiển trung tâm tổng hợp kết quả từ cả hai nhánh xử lý trên. Tại đây, hệ thống tính toán độ lệch quỹ đạo, áp dụng bộ lọc làm mượt tín hiệu, và xác định góc lái cùng vận tốc mục tiêu. Các giá trị điều khiển này được cập nhật liên tục thông qua biến trạng thái dùng chung, đảm bảo tính nhất quán dữ liệu giữa các luồng.

Ở khâu truyền thông, một luồng giao tiếp hoạt động độc lập ở tần số 20 Hz, liên tục đọc các giá trị điều khiển, đóng gói thành chuỗi lệnh và truyền xuống vi điều khiển qua giao tiếp nối tiếp ở tốc độ 115200 bit/giây. Tại tầng chấp hành, vi điều khiển phân tích gói lệnh nhận được, phân phối tín hiệu điều khiển đến động cơ và servo lái, đồng thời cập nhật giá trị tốc độ tham chiếu cho vòng điều khiển tốc độ động cơ



Hình 3.2. Sơ đồ luồng dữ liệu và giao tiếp giữa các thành phần

3.3. Thiết kế phần cứng

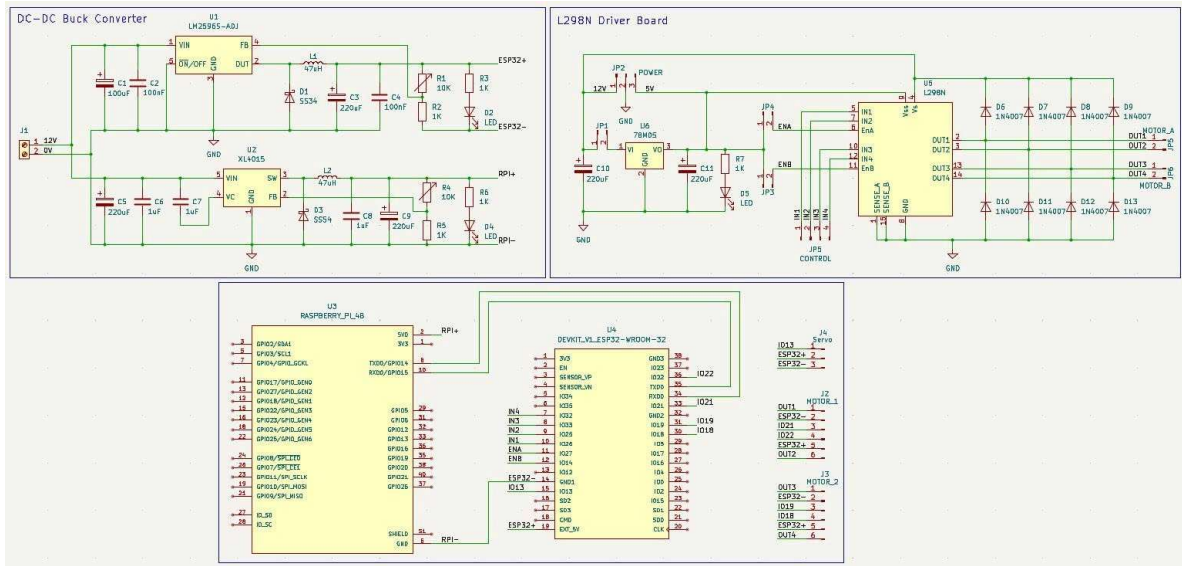
Phần này trình bày chi tiết thiết kế phần cứng của hệ thống, từ sơ đồ kết nối tổng thể đến đặc tả từng khối chức năng. Mọi giá trị điện trở, điện áp và chân kết nối đều được lấy trực tiếp từ firmware và sơ đồ thực tế của mô hình đã xây dựng.

3.3.1. Sơ đồ kết nối tổng thể và phân bổ chân

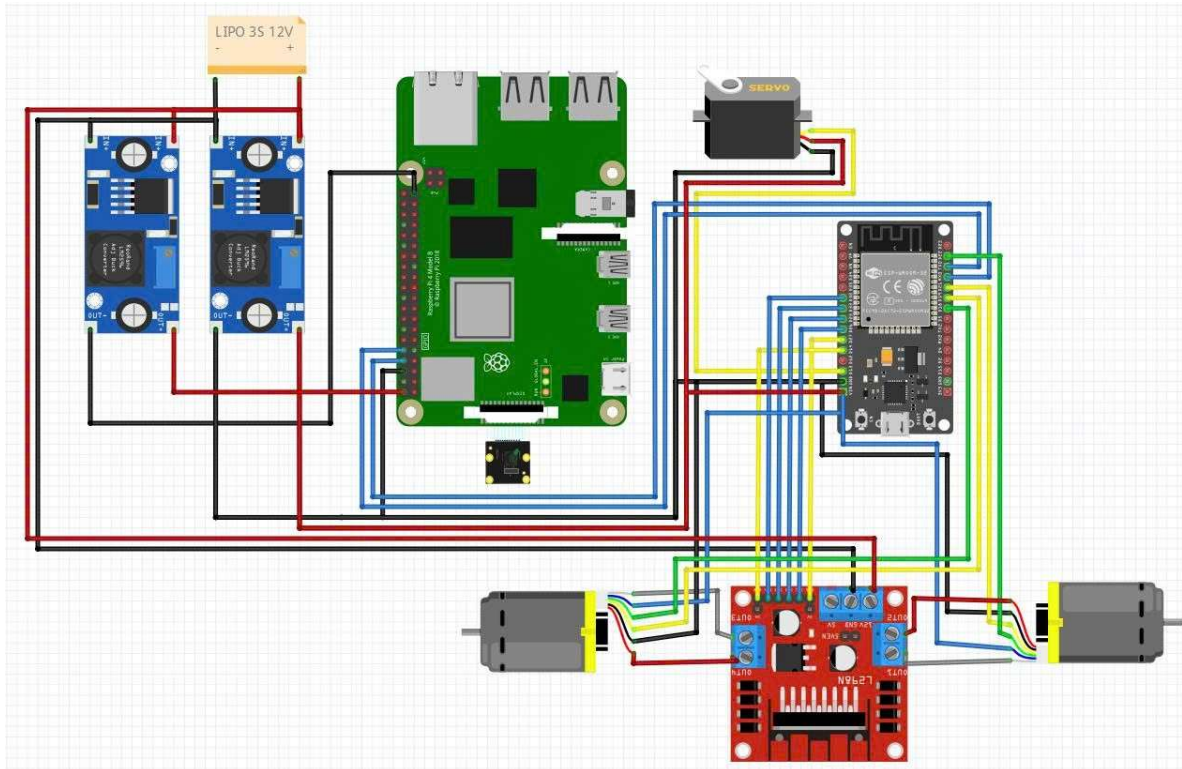
Bảng 3.2 tổng hợp toàn bộ kết nối chân của hệ thống. Nguyên tắc thiết kế là phân tách bus: UART2 (GPIO16/17 của ESP32) dành riêng cho lệnh AI từ Raspberry Pi. Tất cả độc lập nhau, không chia sẻ đường tín hiệu, loại bỏ rủi ro xung đột.

Bảng 3.2. Bảng phân bố chân kết nối toàn hệ thống

Thành phần	Chân nguồn	Chân tín hiệu / Bus	Ghi chú
Camera Module OV5647	3,3V (từ CSI)	CSI 15 chân (MIPI CSI-2)	Cắm thẳng vào cổng CSI của RPi 4
ESP32 ← RPi (UART)	ESP32 tự cấp từ nguồn 5V riêng	RPi GPIO14(TX) → ESP32 GPIO16(RX2) (baud 115200, 8N1)	Chỉ dùng TX một chiều (RPi→ESP32)
L298N ENA (động cơ trái)	12V và 5V logic	IN1=GPIO26, IN2=GPIO25 ENA=GPIO27 (PWM LEDC)	LEDC Timer 0, 5 kHz, 8-bit
L298N ENB (động cơ phải)	12V và 5V logic	IN3=GPIO33, IN4=GPIO32 ENB=GPIO14 (PWM LEDC)	LEDC Timer 1, 5 kHz, 8-bit
Servo MG995	5V, GND	Tín hiệu = GPIO13 (PWM 50 Hz)	ESP32Servo dùng Timer 3 riêng biệt
Encoder bánh trái	3,3V, GND	Kênh A = GPIO19, Kênh B = GPIO21	Ngắt CHANGE trên cả 2 kênh (4× decoding)
Encoder bánh phải	3,3V, GND	Kênh A = GPIO18, Kênh B = GPIO22	Ngắt CHANGE trên cả 2 kênh (4× decoding)



Hình 3.3. Sơ đồ nguyên lý đầy đủ hệ thống phản ứng xe tự hành



Hình 3.4. Sơ đồ khối tổng thể

3.3.2. Khối nguồn điện

Thiết kế nguồn là nền tảng đảm bảo toàn bộ hệ thống hoạt động ổn định. Hệ thống sử dụng ba đường nguồn tách biệt hoàn toàn: 12V cho động cơ DC, 5V-5A cho Raspberry Pi và camera, và 5V-1A cho ESP32 và mạch logic. Hai module buck DC-DC độc lập tạo ra

hai đường 5V với dòng tải khác nhau nhằm tránh sụt áp do động cơ gây ra ảnh hưởng đến mạch xử lý:

- Đường nguồn 12V DC: Cấp cho hai động cơ DC qua mạch cầu H L298N. Dòng tải cực đại khoảng 2-3A khi cả hai động cơ hoạt động đồng thời ở PWM đỉnh. Nguồn này cần có tụ lọc 100 μ F (điện phân) và 100 nF (gốm) đặt sát chân nguồn L298N để triệt tiêu xung điện áp khi động cơ chuyển trạng thái, đặc biệt khi phanh nhanh. Diode Schottky chống đảo cực khuyến nghị đặt trên đường dương nguồn.
- Đường nguồn 5V ổn định: Cấp cho Raspberry Pi 4 yêu cầu tối thiểu 3A, cấp từ buck 5V-5A, ESP32 cần 250 mA tối đa, Camera Module OV5647 cần 250 mA và servo lái cần 700 mA ở tải định mức. Tổng dòng tải 5V-5A ước tính ~3,25A (RPi+camera), 5V-1A ~950 mA (ESP32+servo) , cần bộ chuyển đổi DC-DC step-down hiệu suất cao (trên 85%) thay cho ổn áp tuyến tính 7805 để tránh tổn thất nhiệt lớn.

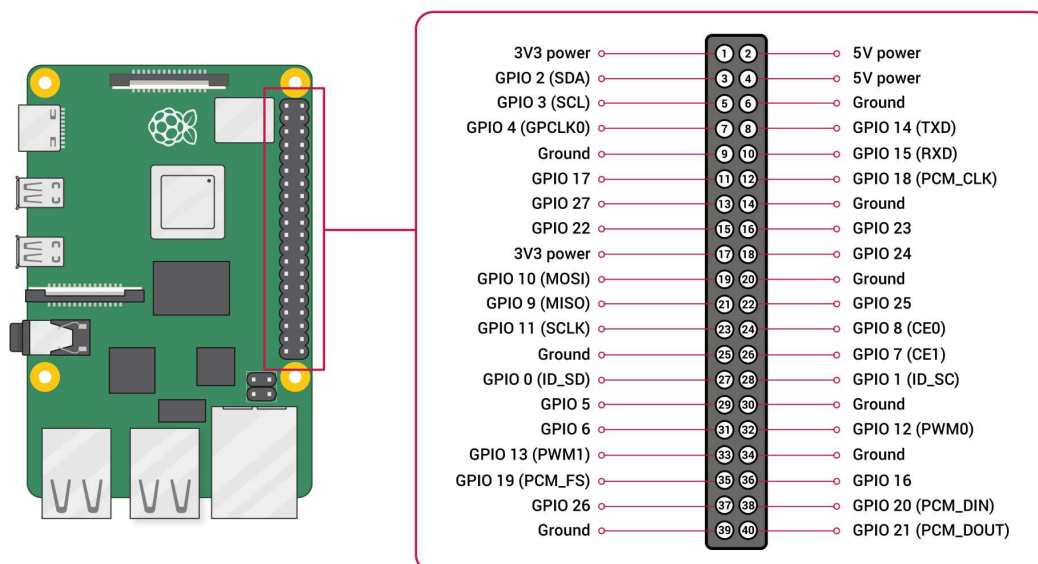
3.3.3. Khối xử lý trung tâm - Raspberry Pi 4 Model B 4GB

Raspberry Pi 4 Model B phiên bản 4 GB RAM được chọn làm bộ não xử lý trung tâm sau khi cân nhắc với các phương án thay thế như Jetson Nano và Orange Pi. Lý do chính là cộng đồng hỗ trợ lớn, thư viện Picamera2 chính thức hỗ trợ Camera Module OV5647 qua CSI với API Python cấp cao, và bộ nhớ 4 GB đủ để chạy đồng thời pipeline OpenCV, YOLOv8n, UART mà không bị tràn bộ nhớ.

Trong hệ thống, Raspberry Pi 4 đảm nhận các tác vụ: (1) Khởi tạo và quản lý cảm biến thị giác, thiết lập các tham số cấu hình tĩnh (khóa thời gian phơi sáng ở mức 20.000 μ s, hệ số khuếch đại tương tự ở mức 4,0 và khóa cân bằng trắng) nhằm đảm bảo độ chói khung hình luôn ổn định dưới điều kiện chiếu sáng nhân tạo của sa hình; (2) Thực thi chuỗi thuật toán xử lý ảnh tĩnh để phân đoạn mặt nạ và nhận diện quỹ đạo di chuyển; (3) Vận hành mạng nơ-ron để nhận diện hệ thống biển báo giao thông; (4) Quản lý giao thức truyền thông nối tiếp để đóng gói và xuất lệnh điều khiển xuống vi mạch tó ở tốc độ 115200 bit/giây; và (5) Điều phối vòng lặp trung tâm để đồng bộ hóa dữ liệu giữa các phân luồng xử lý song song.

Raspberry Pi OS (phiên bản Bookworm 64-bit) được cài đặt trên thẻ microSD Class 10 32 GB. Giao diện UART phải được kích hoạt trong raspi-config trước khi hệ thống hoạt

động. Nhiệt độ SoC cần được giám sát khi chạy liên tục , một hệ thống tản nhiệt nhôm được gắn chip BCM2711.



Hình 3.5. Raspberry Pi 4 Model B , bộ xử lý trung tâm của hệ thống

3.3.4. Khối điều khiển chấp hành - ESP32

Vi điều khiển trung tâm (ESP32) đóng vai trò là nền tảng chấp hành thời gian thực, chịu trách nhiệm tiếp nhận gói lệnh từ máy tính nhúng và trực tiếp phân phối năng lượng điều khiển đến các cơ cấu truyền động. Việc vận hành firmware trên nền tảng hệ điều hành thời gian thực với cơ chế quản lý ngắt ưu tiên nghiêm ngặt giúp vi điều khiển duy trì tuyệt đối chu kỳ vòng lặp của bộ điều khiển tỷ lệ - tích phân - vi phân ở mức 50 mili-giây. Đặc tính này giúp hệ thống khắc phục hoàn toàn hiện tượng trễ nhịp do cơ chế phân thời sinh ra trên các hệ điều hành đa nhiệm thông thường.

Quy trình phân bổ các bộ định thời phần cứng bên trong vi điều khiển được thiết kế và quy hoạch chặt chẽ nhằm triệt tiêu rủi ro xung đột tài nguyên. Nền tảng vi mạch này được trang bị bốn bộ đếm thời gian độc lập, được phân chia chuyên biệt cho tác vụ băm xung PWM truyền lực và điều khiển góc lái, chi tiết được minh họa trong Bảng 3.3.

Bảng 3.3. Phân bổ timer phần cứng ESP32 trong hệ thống

Timer ESP32	Chức năng phân bổ trong hệ thống	Tần số	Độ phân giải
Timer 0	LEDC , ENA (PWM động cơ trái)	5.000 Hz	8-bit (0-255)
Timer 1	LEDC , ENB (PWM động cơ phải)	5.000 Hz	8-bit (0-255)
Timer 2	Dự phòng / server HTTP	Không áp dụng	Không áp dụng
Timer 3	ESP32Servo , Servo lái GPIO13	50 Hz	~10-bit hiệu dụng

Để bảo vệ tính toàn vẹn của hệ thống điều khiển cơ khí, bộ định thời số 3 được cấu hình ưu tiên cấp phát độc lập và khóa bảo vệ ngay từ giai đoạn khởi tạo thiết bị phần cứng ban đầu. Thao tác kỹ thuật này ngăn chặn triệt để hiện tượng tranh chấp bộ đếm từ các thư viện điều khiển xung ngoại vi khác, đảm bảo động cơ chuyển hướng luôn vận hành chính xác, trơn tru và không sinh ra các xung nhiễu tự kích làm lệch góc lái ngoài ý muốn.

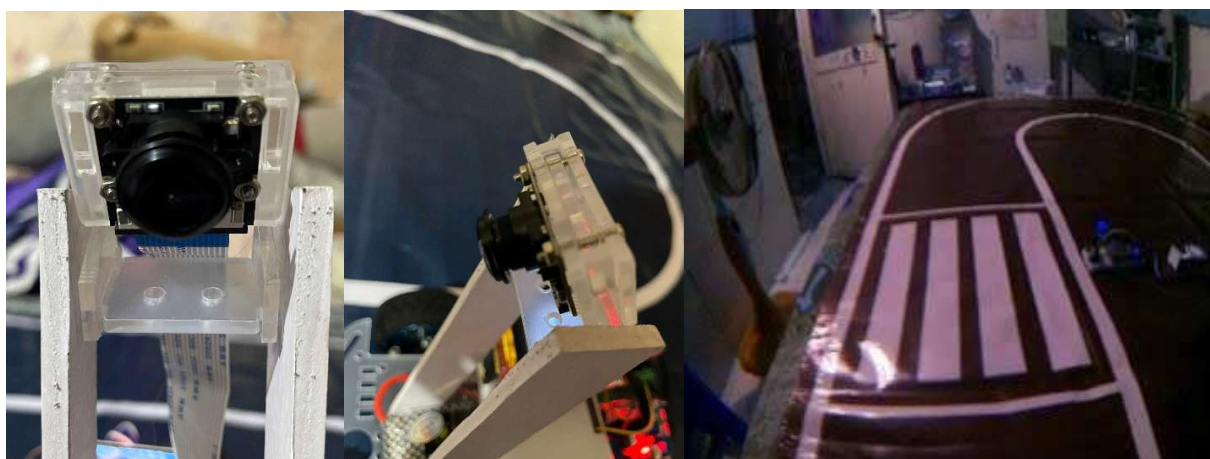
3.3.5. Khối cảm biến hình ảnh - Camera Module OV5647

Camera Module sử dụng cảm biến OV5647 5MP góc rộng 160° kết nối qua cổng CSI 15 chân của Raspberry Pi. Trong hệ thống, camera được cấu hình hoạt động ở chế độ không phải độ phân giải tối đa để đảm bảo tốc độ xử lý:

- Độ phân giải capture: 320×240 pixel , đủ để phát hiện vạch làn đường rộng 2-3 cm trên sa hình từ khoảng cách 20-40 cm.
- Định dạng pixel: RGB888 (3 byte/pixel) , trực tiếp đưa vào pipeline OpenCV mà không cần chuyển đổi định dạng trung gian.
- Frame rate mục tiêu: 30 FPS, main loop xử lý khoảng 10-15 FPS thực tế tùy độ phức tạp scene.
- Khóa thời gian phơi sáng: Thời gian mở trập được neo cố định ở mức 20.000 μ s, đồng thời vô hiệu hóa hoàn toàn cơ chế phơi sáng tự động. Thiết lập này nhằm triệt tiêu hiện tượng nhấp nháy quang học hoặc lóa sáng khi cường độ phản xạ ánh sáng từ vạch kẻ nền thay đổi đột ngột khi xe di chuyển.

Về mặt cơ khí học bố trí, nhờ sở hữu trường nhìn siêu rộng, cảm biến thị giác không yêu cầu góc chú ý quá lớn. Góc nghiêng của ống kính được neo trong khoảng từ 15° đến 25° so với phương ngang, với cao độ lắp đặt xấp xỉ từ 10 đến 12 cm tính từ mặt đường, đảm bảo bao quát toàn bộ hành lang di chuyển.

Đặc tính góc rộng này cũng tác động trực tiếp đến không gian xử lý thuật toán: Vùng quan tâm được thiết lập loại bỏ 45% không gian ảnh tính từ cạnh trên (tương đương với việc cắt bỏ 108 điểm ảnh theo trục tung). Mức độ cắt xén này sâu hơn đáng kể so với các ống kính tiêu chuẩn nhằm mục đích loại bỏ hoàn toàn nhiễu hậu cảnh và vùng không gian chết ở hai biên, tập trung tối đa tài nguyên xử lý vào bề mặt sa hình ngay phía trước hướng tuyến của phương tiện.



Hình 3.6. Vị trí gắn Camera Module OV5647 trên mô hình xe và ví dụ frame thu được

3.3.6. Khối truyền động - L298N, Động cơ DC và Encoder

Hai động cơ điện một chiều dẫn động cầu sau được tích hợp sẵn cảm biến mã hóa vòng quay vuông góc và được cung cấp năng lượng thông qua mạch cầu H L298N.

Nguyên lý điều khiển động lực học từng bánh xe được thực thi như sau: Chiều quay của động cơ được định tuyến bởi tổ hợp logic của hai chân điều hướng; IN1/IN2 cho cụm bên trái, IN3/IN4 cho cụm bên phải theo nguyên lý đóng ngắt chéo của mạch cầu H (chi tiết tại Bảng 3.4). Vận tốc quay thực tế được điều áp thông qua tín hiệu băm xung cấp trực tiếp vào chân Enable với độ phân giải phân cứng 8-bit; tham số băm xung chạy từ 0 đến 255, tương đương việc điều chế độ rộng xung từ 0% đến 100%.

Bảng 3.4. Bảng chân trị điều khiển chiều quay động cơ qua L298N

IN1 (IN3)	IN2 (IN4)	Trạng thái động cơ
HIGH	LOW	Quay thuận chiều (tiến)
LOW	HIGH	Quay ngược chiều (lùi)
LOW	LOW	Phanh nhanh (brake)
HIGH	HIGH	Phanh nhanh (brake)

Hệ thống cảm biến mã hóa vuông góc bao gồm hai kênh xung A và B lệch pha nhau 90° , cung cấp tín hiệu phản hồi vận tốc cho bộ điều khiển hồi tiếp. Nhằm tối đa hóa độ phân giải đo lường, toàn bộ bốn đường tín hiệu từ hai cụm động cơ đều được cấu hình đăng ký ngắt cứng theo chế độ thay đổi trạng thái kép; bắt tín hiệu ở cả sườn lên và sườn xuống. Chế độ giải mã 4X này kết hợp với thuật toán kiểm tra sự đồng pha/lệch pha giữa kênh A và kênh B trong chương trình phục vụ ngắt để xác định chính xác chiều quay thực tế của bánh xe.

Để đáp ứng yêu cầu khắt khe về thời gian thực, các chương trình phục vụ ngắt được chỉ định phân bổ trực tiếp vào vùng nhớ truy cập nhanh bên trong vi mạch. Giải pháp cấp phát bộ nhớ này đảm bảo thời gian phản hồi ngắt được duy trì ổn định ở mức 1 đến 3 μs , kể cả khi bộ nhớ Flash đang bị chiếm dụng bởi các tác vụ đa nhiệm khác.

3.3.7. Khối cơ cấu lái - Servo MG955

Servo MG955 là servo kim loại torque cao (13 kg·cm) điều khiển cơ cấu lái Ackermann thu nhỏ của mô hình xe. Đặc điểm kỹ thuật quan trọng nhất đối với hệ thống là dải góc làm việc được hiệu chỉnh theo thực nghiệm cơ khí:



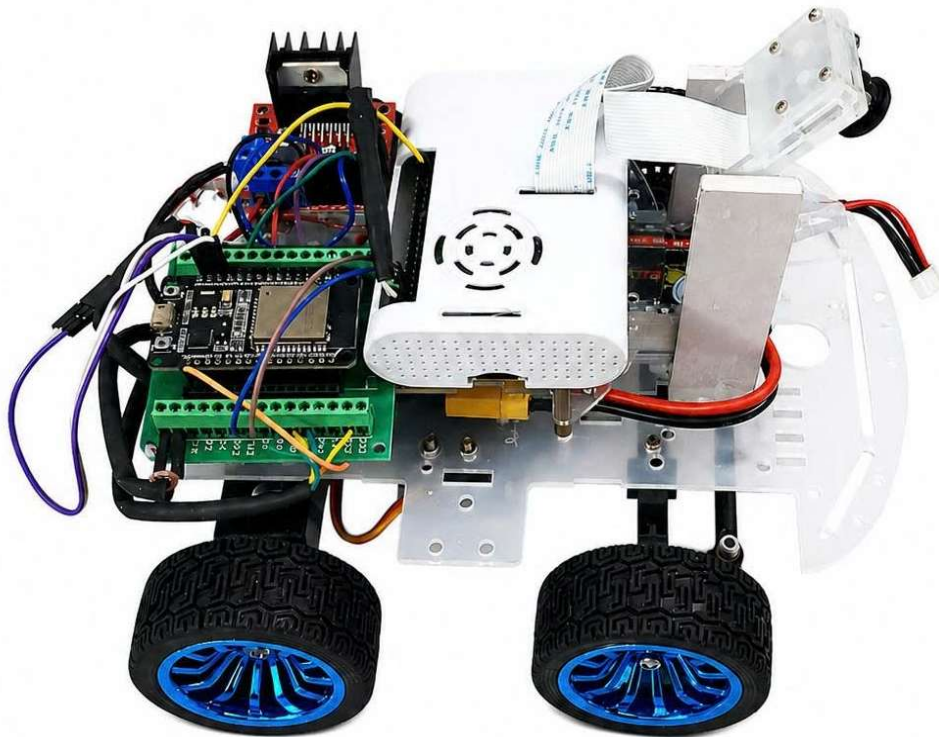
Hình 3.7. Servo MG955 được sử dụng để điều khiển cơ cấu lái

- SERVO_LEFT = 40° , vị trí lái hết trái, giới hạn bởi thanh kéo cơ học
- SERVO_CENTER = 68° , vị trí thẳng, hiệu chỉnh thực nghiệm, không phải 90° lý thuyết do sai lệch lắp ráp của cơ cấu Ackermann
- SERVO_RIGHT = 120° , vị trí lái hết phải, giới hạn bởi thanh kéo cơ học

Do cấu trúc hành trình lái bị lệch tâm không đối xứng, biên độ xoay trái là 28°, trong khi biên độ xoay phải là 52°, một thuật toán nội suy tuyến tính bất đối xứng đã được phát triển để ánh xạ giá trị tín hiệu lái chuẩn hóa ($S \in [-100, +100]$) thu được từ luồng xử lý ảnh sang góc quay thực tế của trục servo (θ):

$$\theta = \begin{cases} 68 + \frac{S \cdot (68 - 40)}{100} = 68 + 0.28 \cdot S & \text{nếu } S \leq 0 \\ 68 + \frac{S \cdot (120 - 68)}{100} = 68 + 0.52 \cdot S & \text{nếu } S > 0 \end{cases}$$

Bên cạnh đó, để giảm thiểu tiêu hao điện năng tĩnh và triệt tiêu tiếng ồn rung lắc không cần thiết sinh ra do sai số rời rạc của tín hiệu điều chế, thuật toán cập nhật servo chỉ kích hoạt việc xuất tín hiệu xung phần cứng khi và chỉ khi giá trị góc lái mục tiêu có sự biến thiên thực sự so với giá trị hiện tại đã được chốt giữ.



Hình 3.8. Hình ảnh thực tế mô hình xe tự hành hoàn chỉnh

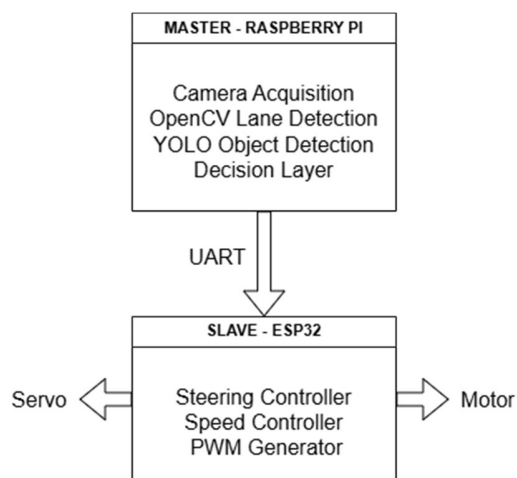
3.4. THIẾT KẾ PHẦN MỀM

Phần này trình bày phương pháp hiện thực hóa cấu trúc phân tách chức năng của hệ thống xe tự hành thành các cấu phần phần mềm cụ thể. Nội dung bao gồm việc thiết kế giao thức truyền thông giữa hai bộ xử lý, cấu trúc tổ chức các luồng thực thi song song bên trong bộ xử lý trung tâm và cấu trúc chương trình điều khiển đáp ứng thời gian thực trên bộ vi điều khiển chấp hành.

3.4.1. Tổng quan Kiến trúc Phần mềm và Cơ chế Đa luồng

3.4.1.1. Kiến trúc phân tán Master - Slave

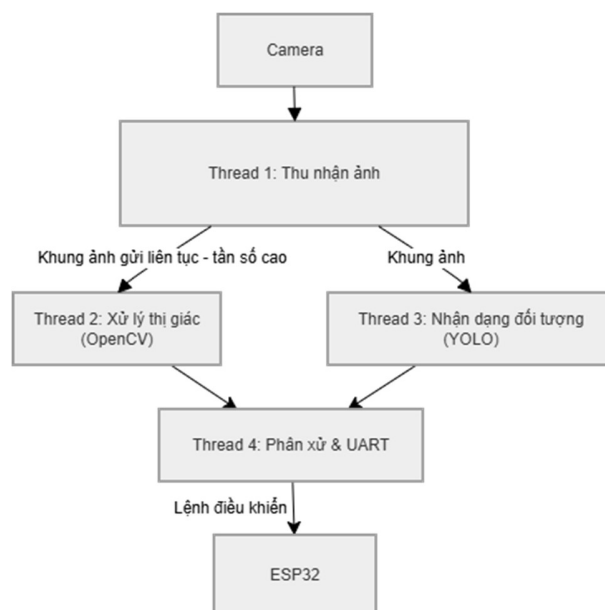
Hệ thống điều khiển của mô hình được vận hành theo kiến trúc phân tán Master - Slave: bộ xử lý trung tâm đóng vai trò Master, đảm nhận toàn bộ chuỗi xử lý nhận thức và ra quyết định; bộ vi điều khiển đóng vai trò Slave, nhận lệnh điều khiển góc lái, tốc độ mục tiêu và thực thi vòng lặp điều khiển động cơ cục bộ. Kênh giao tiếp giữa hai khối được thiết kế một chiều: Master ban hành lệnh theo chu kỳ cố định, Slave không gửi phản hồi trạng thái ngược lại, giúp đơn giản hóa giao thức truyền thông.



Hình 3.9. Kiến trúc phân tán giữa bộ xử lý trung tâm và bộ điều khiển thực thi

3.4.1.2. Thiết kế đa luồng và quản lý tài nguyên

Bốn nhiệm vụ xử lý của hệ thống được hiện thực hóa thành bốn luồng cụ thể trên bộ xử lý trung tâm: Thread 1 thu nhận ảnh đóng vai trò nhà sản xuất gốc; Thread 2 xử lý thị giác và Thread 3 nhận dạng đối tượng vừa tiêu thụ khung hình từ Thread 1 vừa sản xuất kết quả tương ứng; Thread 4 phân xử & UART tiêu thụ đồng thời kết quả của cả hai để tổng hợp lệnh điều khiển cuối cùng, cấu trúc minh họa được thể hiện ở Hình 3.11.



Hình 3.10. Sơ đồ cơ chế đồng bộ hóa đa luồng giữa các khối xử lý

Trên vi điều khiển, hai tác vụ tương ứng gồm vòng lặp điều khiển động học và tiếp nhận lệnh UART vận hành theo cấu trúc không chặn đã trình bày ở Mục 2.7. Bảng 3.5 tổng hợp phân bổ tài nguyên xử lý, tần số vận hành và cơ chế đồng bộ hóa dữ liệu của toàn bộ sáu tác vụ cùng bốn luồng trên bộ xử lý trung tâm và hai tác vụ trên vi điều khiển.

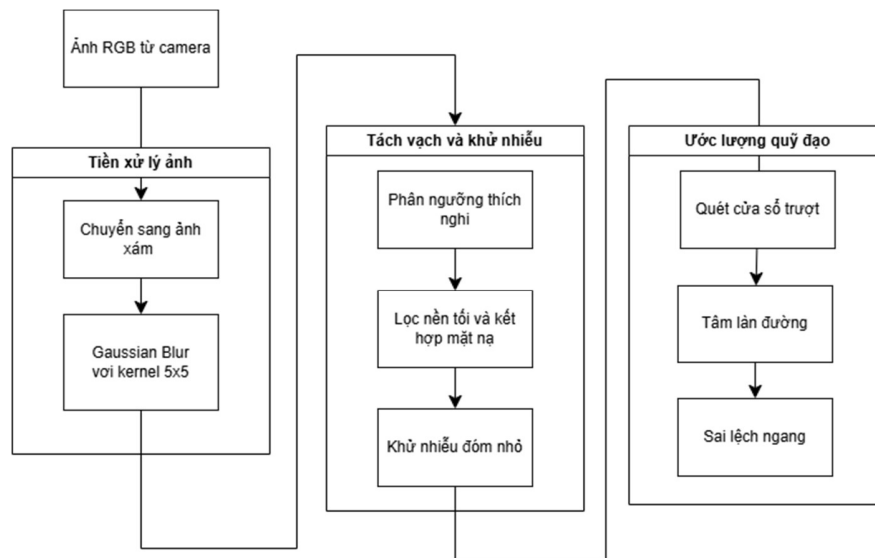
Bảng 3.5. Phân bổ tài nguyên CPU và thời gian thực thi kỳ vọng cho từng luồng

Tác vụ xử lý	Bộ xử lý	Tần số vận hành	Thời gian thực thi	Cơ chế đồng bộ
Thu nhận ảnh	Bộ xử lý trung tâm	~30 khung hình/giây	~5 ms/khung hình	Vùng nhớ dùng chung, cập nhật ghi đè
Xử lý thị giác (lần đường)	Bộ xử lý trung tâm	~25 khung hình/giây	~15 ms/khung hình	Vùng nhớ dùng chung có khóa đồng bộ
Nhận dạng đối tượng (YOLO)	Bộ xử lý trung tâm	~5 khung hình/giây	200-350 ms/lần	Hàng đợi một phần tử, xử lý bất đồng bộ
Hợp nhất & truyền lệnh điều khiển	Bộ xử lý trung tâm	20 Hz	~1 ms/chu kỳ	Biến trạng thái dùng chung có khóa đồng bộ

Điều khiển động học (PID)	Bộ vi điều khiển	50 Hz cố định	< 1 ms/chu kỳ	Ngắt cứng + vùng nhớ bảo vệ
Tiếp nhận lệnh điều khiển	Bộ vi điều khiển	Theo sự kiện UART	< 0.5 ms/gói tin	Ngắt cứng + vùng nhớ bảo vệ

3.4.2. Khối Nhận thức Không gian: Pipeline Phát hiện Làn đường

Khối phát hiện làn đường thực hiện một chuỗi xử lý ảnh có thứ tự nhằm trích xuất vị trí của vạch kẻ đường và ước lượng sai lệch quỹ đạo ngang của xe so với tâm làn đường. Hình 3.12 trình bày các giai đoạn chính của pipeline phát hiện làn đường, bao gồm tiền xử lý ảnh, trích xuất đặc trưng hình học và ước lượng vị trí tương đối của xe trong làn đường, không yêu cầu giai đoạn huấn luyện và có chi phí tính toán thấp, phù hợp với yêu cầu xử lý theo thời gian thực trên thiết bị có cấu hình hạn chế.



Hình 3.11. Pipeline phát hiện làn đường sử dụng xử lý ảnh truyền thống

3.4.2.1. Tiền xử lý: ảnh xám và lọc Gaussian

Khung hình màu 320×240 được chuyển sang ảnh xám vì thông tin cần để tách vạch làn nằm ở độ sáng chứ không ở màu sắc; xử lý trên một kênh xám cũng giảm tải tính toán cho máy tính nhúng. Ảnh xám tiếp đó được làm mịn bằng bộ lọc Gauss 5×5 nhằm khử nhiễu hạt, tránh các điểm sáng/tối lẻ gây nhiễu cho bước phân ngưỡng. Toàn bộ pipeline làm việc trên ảnh xám.

3.4.2.2. Tối ưu hóa không gian: Hình học thấu thị và Vùng quan tâm

Khung hình 320×240 pixel sau khi được chuyển sang ảnh xám và qua bước tiền xử lý tạo mặt nạ vạch nhị phân, được giới hạn xử lý trong một vùng quan tâm hình chữ nhật để loại bỏ hậu cảnh phía trên trước khi đưa vào các bước trích xuất đặc trưng làn đường.

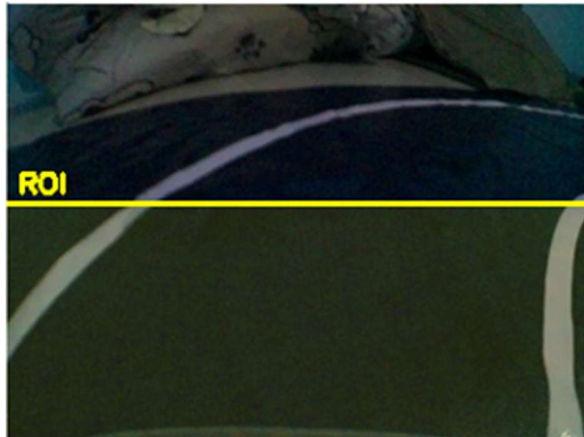
Do camera OV5647 sử dụng ống kính góc rộng 160° , cả hai vạch kẻ đường đều nằm trong toàn bộ chiều rộng khung hình ở phần mặt đường hữu ích. Vì vậy, ROI được thiết lập theo một đường cắt ngang đơn giản tại y_{top} , giữ lại toàn bộ chiều rộng ảnh từ $x = 0$ đến $x = W$:

- Ngưỡng cắt dọc y_{top} : loại bỏ phần ảnh phía trên, chỉ giữ lại vùng mặt đường phía dưới.
- Chiều rộng giữ nguyên $[0, W]$: không thu hẹp hai bên vì vạch làn nằm trong toàn bộ chiều rộng ở vùng cần cảnh.

Giá trị y_{top} được hiệu chỉnh thực nghiệm dựa trên chiều cao lắp đặt của camera và góc nghiêng so với mặt đường, được liệt kê trong Bảng 3.6.

Bảng 3.6. Tham số cấu hình giới hạn Vùng quan tâm hình thang

Tham số cấu hình	Ký hiệu	Giá trị thực nghiệm	Mục đích thiết lập
Giới hạn đỉnh ROI	y_{top}	$42\% \times H$	Cắt bỏ vùng hậu cảnh, chỉ giữ mặt đường
Cạnh trái ROI	x_L	0	Giữ nguyên toàn bộ chiều rộng
Cạnh phải ROI	x_R	W	Giữ nguyên toàn bộ chiều rộng



Hình 3.12. Vùng quan tâm được triển khai

3.4.2.3. Tách vạch: phân ngưỡng thích nghi và lọc nền

Ảnh xám trong vùng quan tâm được đưa qua phân ngưỡng thích nghi cục bộ; ô 41×41 , $C = -15$ để làm nổi vạch làn. Tiếp đó, mặt nạ vạch được lọc qua hai bước: lấy giao với mặt nạ vùng đường để loại các vùng sáng nằm ngoài mặt đường, và phân tích vùng liên thông để xóa các đốm nhỏ hơn ngưỡng diện tích. Kết quả là mặt nạ làn sạch, chỉ còn lại vạch làm đầu vào ổn định cho bước tìm tâm làn.

3.4.2.4. Quét cửa sổ trượt và ước lượng tâm làn

Trên mặt nạ làn, hệ thống chia vùng quan tâm thành 8 dải ngang và quét cửa sổ trượt từ đáy lên để bám vạch trái/phải, suy ra tâm làn của từng dải rồi tổng hợp thành tâm làn chung bằng trung bình có trọng số. Khi chỉ thấy một vạch, hệ thống dựa vào độ nghiêng của vạch để xác định hướng cua và cho xe ôm sát mép trong. Từ tâm làn, độ lệch được tính và chuyển thành lệnh lái theo luật phi tuyến có vùng chết và bộ lọc trung bình trượt.

Khi tạm thời không phát hiện đủ số dải làn hợp lệ hay gặp ở cua gắt, hệ thống giữ nguyên góc lái tốt gần nhất và cho xe bò chậm tiếp trong một số khung hình; chỉ khi mất làn kéo dài quá ngưỡng thì xe mới dừng. Cơ chế này giúp xe vượt các đoạn mất làn ngắn mà vẫn giữ quỹ đạo mượt.

3.4.3. Khối Ước lượng Trạng thái và Lọc tín hiệu Thời gian

3.4.3.1. Ước lượng tâm làn đường và Độ lệch quỹ đạo

Tọa độ tâm làn đường được hệ thống nội suy trực tiếp từ trung điểm của hai dải vạch kẻ biên thu được từ khối mô hình hóa. Trong tình huống cảm biến thị giác bị khuất

tầm nhìn một bên vạch, thuật toán sẽ tự động tính tiền tọa độ từ dải vạch còn lại dựa trên tham số độ rộng làn đường tiêu chuẩn đã được đo đạc thực nghiệm trên sa hình.

Sau khi xác định được tâm làn đường, hệ thống tiến hành tính toán sai lệch quỹ đạo ngang và chuẩn hóa giá trị về dải $[-1; 1]$. Tương ứng với khung hình camera độ phân giải 320x240 pixel, hoành độ tâm tham chiếu của mũi xe được thiết lập cố định tại giá trị 160. Tín hiệu sai số chuẩn hóa này đóng vai trò là biến trạng thái cốt lõi để đưa vào khối tính toán luật điều khiển bù trừ.

Bảng 3.7. Tham số cấu hình hệ thống không gian và chuẩn hóa sai số

Tham số cấu hình	Ký hiệu	Giá trị thực thi	Ý nghĩa trên hệ thống
Chiều rộng khung hình	W	320	Cơ sở không gian để tính toán tỷ lệ chuẩn hóa biên độ sai số
Tâm tham chiếu mũi xe	X_{car}	160	Định vị phương vị thực tế của trục đối xứng phương tiện trên ảnh
Dải sai số chuẩn hóa	$e(t)$	$[-1; 1]$	Đồng bộ miền giá trị tín hiệu đầu vào cho luật điều khiển bám làn phi tuyến

3.4.3.2. Xử lý nhiễu rung lắc và hiện tượng đứt gãy tín hiệu

Hệ thống xử lý hiện tượng mất dấu vạch tạm thời do cua gắt, lóa sáng hoặc vạch bị khuất bằng cơ chế giữ trạng thái, thay cho một khâu lọc đột biến phức tạp. Ở mỗi chu kỳ, nếu khối thị giác trích xuất được làn hợp lệ, góc lái mới được tính và được lưu lại làm "góc lái tốt gần nhất".

Khi tạm thời không phát hiện đủ số dải làn hợp lệ, hệ thống không bẻ lái theo tín hiệu nhiễu mà giữ nguyên góc lái tốt gần nhất và cho xe tiếp tục bò chậm tiến tới. Cơ chế này tận dụng quán tính của phương tiện để vượt qua các đoạn mất làn ngắn mà vẫn giữ quỹ đạo mượt. Chỉ khi mất dấu vạch kéo dài liên tục quá 30 khung hình, hệ thống mới ép vận tốc về 0 để tránh xe trượt khỏi sa hình. Tín hiệu lái sau đó tiếp tục được làm mượt bằng bộ lọc trung bình trượt hàm mũ trình bày ở mục kế tiếp.

Bảng 3.8. Tham số cấu hình khâu lọc đột biến động lực học

Tham số cấu hình	Ký hiệu	Giá trị thực thi	Ý nghĩa trên hệ thống
Ngưỡng biến thiên cực đại	θ_{max}	0,65	Giới hạn phân loại nhiễu đột biến để ngắt khung hình lỗi
Giá trị an toàn ngoại lệ	e_{null}	0	Tín hiệu cấp bù để ép cơ cấu lái duy trì hướng thẳng khi mất dấu quỹ đạo

3.4.3.3. Lọc tín hiệu bằng Trung bình trượt hàm mũ

Tín hiệu sai số quỹ đạo sau khi đi qua khâu cắt đột biến vẫn tồn tại các dao động vi mô tần số cao. Để đảm bảo cơ cấu chấp hành bề lái mượt mà, tín hiệu tiếp tục được đưa qua bộ lọc thông thấp Trung bình trượt hàm mũ.

Hệ số làm mượt α của bộ lọc được cấu hình tĩnh sau quá trình tinh chỉnh thực nghiệm. Mục tiêu thiết lập là đạt được điểm cân bằng động lực học tối ưu: hệ số đủ nhỏ để triệt tiêu hiện tượng rung lắc vô lăng trên đoạn đường thẳng, nhưng vẫn phải đảm bảo độ lớn cần thiết để không gây trễ pha tín hiệu, giúp phương tiện đáp ứng nhạy bén khi tiến vào các góc cua gắt. Tín hiệu sai số sau khi được làm mượt sẽ được đóng gói và truyền xuống vi điều khiển để thực thi điều khiển bám làn phi tuyến.

Bảng 3.9. Tham số cấu hình khâu lọc thông thấp thời gian thực

Tham số cấu hình	Ký hiệu	Giá trị thực thi	Ý nghĩa trên hệ thống
Hệ số làm mượt	α	Tinh chỉnh thực nghiệm	Cân bằng giữa khả năng triệt tiêu dao động cơ cấu lái và độ trễ pha đáp ứng.
Quán tính quỹ đạo	$e_{filtered}(t - 1)$	Lưu trữ động	Kế thừa trạng thái động học từ chu kỳ lấy mẫu liên trước.

3.4.4. Khối Nhận thức Ngữ nghĩa: Mạng nơ-ron Tích chập YOLO

3.4.4.1. Xây dựng bộ dữ liệu hận dạng đối tượng giao thông

Bộ dữ liệu được thu thập theo hai nguồn. Nguồn thứ nhất gồm ảnh chụp trực tiếp bằng camera OV5647 gắn trên mô hình xe trong điều kiện chiếu sáng nhân tạo thực tế của sa hình, đây là nguồn ảnh quan trọng nhất vì phản ánh đúng phân phối đầu vào mà mô hình

sẽ gặp khi triển khai. Nguồn thứ hai gồm ảnh tổng hợp được tạo bằng cách ghép đối tượng vào các nền sa hình khác nhau, giúp tăng độ đa dạng mà không cần thu thập thêm ảnh thực.

Mỗi lớp đối tượng mục tiêu được thu thập khoảng 1.000 ảnh, phân chia theo tỉ lệ 80/20 giữa tập huấn luyện và tập kiểm định. Ngoài ra, 500 ảnh thực từ camera xe khi di chuyển trên sa hình chứa nhiều lớp đồng thời được bổ sung vào cả hai tập để tăng tính đa dạng ngữ cảnh. Bảng 3.13 tổng hợp thống kê bộ dữ liệu.

Bảng 3.10. Phân bố bộ dữ liệu nhận dạng đối tượng giao thông

Lớp đối tượng	Mô tả vật thể thực tế	Số ảnh train	Số ảnh val	Tổng
stop_sign	Biển STOP bát giác đỏ thu nhỏ	800	200	1.000
no_entry	Biển cấm (tròn đỏ, vạch ngang trắng)	800	200	1.000
red_light	LED đỏ trên cột đèn tín hiệu	800	200	1.000
yellow_light	LED vàng trên cột đèn tín hiệu	800	200	1.000
green_light	LED xanh trên cột đèn tín hiệu	800	200	1.000
person	Biển/hình người đi bộ tại vạch zebra	800	200	1.000
Sa hình (đa lớp)	Ảnh thực từ camera xe, nhiều đối tượng	400	100	500
Tổng cộng	—	5.200	1.300	6.500

3.4.4.2. Cấu hình và quy trình huấn luyện

Mô hình được huấn luyện trên nền tảng Kaggle với GPU T4, sử dụng framework Ultralytics phiên bản 8.2. Điểm khởi đầu là trọng số pretrained YOLOv8n.pt huấn luyện trên COCO theo kỹ thuật Transfer Learning đã trình bày tại Mục 2.5.4.1: các lớp Backbone giữ nguyên trọng số, chỉ các lớp Detection Head được điều chỉnh để phân loại đúng 6 lớp mục tiêu.

Quyết định thiết kế quan trọng nhất là chiến lược tăng cường dữ liệu theo đặc thù môi trường sa hình: nền tối, đèn LED nhỏ, ánh sáng nhân tạo không đồng đều, camera cố định không bao giờ lật ngược ảnh. Bảng 3.11 trình bày toàn bộ siêu tham số huấn luyện và lý do lựa chọn từng giá trị.

Bảng 3.11. Siêu tham số huấn luyện YOLOv8n và lý do lựa chọn

Tham số	Giá trị	Lý do lựa chọn
Kiến trúc	YOLOv8n	Kích thước nhỏ nhất trong họ YOLOv8, phù hợp tài nguyên Raspberry Pi 4
IMG_SIZE	320×320	Khớp độ phân giải thực tế camera 320×240 + letterbox padding
EPOCHS	100	Kết hợp Early Stopping (patience=25) tránh overfit
BATCH	32	Tối ưu cho GPU T4 trên Kaggle
Optimizer	AdamW, lr=0.001, lrf=0.005	AdamW phù hợp dataset nhỏ; lr cuối thấp để fine-tune đối tượng nhỏ
Warmup	5 epoch	Ổn định gradient giai đoạn đầu khi dataset nhỏ
hsv_v	0.6	Jitter độ sáng ±60% - ảnh sa hình tối, cần giả lập nhiều mức sáng
hsv_h	0.01	Jitter màu sắc nhỏ - màu LED và biển báo nhạy với hue
scale	0.4	Mở rộng phạm vi kích thước đối tượng - biển báo nhiều cự ly

mosaic	1.0	Kết hợp 4 ảnh - hiệu quả nhất cho đối tượng nhỏ trên nền đơn giản
mixup	0.05	Mức nhẹ - tránh làm nhòe đèn LED nhỏ
copy_paste	0.0	Tắt - nền sa hình đồng nhất
flipud	0.0	Tắt - camera cố định, không có ảnh lật ngược
box	10.0	Tăng trọng số định vị - bbox chính xác cho đối tượng nhỏ
cls	0.3	Giảm trọng số phân loại - 6 lớp, ít nhầm lẫn ngữ nghĩa
dfl	1.5	Giữ mặc định cho Distribution Focal Loss
Định dạng	ONNX, opset=12	Tương thích OpenCV DNN 4.x trên Raspberry Pi
dynamic	False (batch=1)	Cố định batch size 1 cho suy luận thời gian thực
half	False (FP32)	Raspberry Pi không hỗ trợ FP16 hardware
simplify	True	Tối ưu đồ thị ONNX, loại bỏ node dư thừa

Cơ chế Early Stopping với patience = 25 epoch được kích hoạt: nếu mAP@0.5 trên tập kiểm định không cải thiện sau 25 epoch liên tiếp, quá trình dừng sớm và trọng số tốt nhất được lưu. Cơ chế này giúp tránh overfitting trên bộ dữ liệu ~6.500 ảnh so với các tập công khai quy mô lớn như COCO (~120.000 ảnh).

3.4.4.3. Bài toán phát hiện đối tượng trong môi trường giao thông

Khối nhận thức ngữ nghĩa thu thập thông tin phi hình học từ môi trường giao thông: biển báo, trạng thái đèn tín hiệu, người đi bộ nhằm cung cấp dữ liệu đầu vào cho tầng quyết định. Hệ thống định nghĩa sáu lớp đối tượng mục tiêu để bao phủ các kịch bản điều hướng trên sa hình thử nghiệm (Bảng 3.12).

Để giảm hiện tượng nhận diện sai hoặc nhiễu tích cực giả ở khoảng cách quá xa ngoài tầm kiểm soát của phương tiện, hệ thống áp dụng đồng thời hai ràng buộc: ngưỡng

tin cậy 0,25 và ngưỡng diện tích khung bao tối thiểu 200 px², áp dụng đồng nhất cho cả sáu lớp

Bảng 3.12. Danh sách lớp đối tượng mục tiêu và ngưỡng vận hành

Lớp đối tượng	Ngưỡng tin cậy	Hành vi vận hành tương ứng	Diện tích tối thiểu
Biển báo dừng	0.25	Dừng khi còn thấy biển; tự đi lại khi biển khuất khỏi khung hình	200
Biển cấm đi ngược chiều	0.25	Dừng khi còn thấy biển; tự đi lại khi biển khuất khỏi khung hình	200
Đèn đỏ	0.25	Chốt dừng đến khi nhận diện đèn xanh, hoặc tự nhả sau 8 giây an toàn	200
Đèn vàng	0.25	Không đổi tốc độ; tiếp tục chạy bình thường	200
Đèn xanh	0.25	Chạy bình thường; giải phóng chốt dừng của đèn đỏ (nếu có)	200
Người đi bộ	0.25	Dừng khi còn thấy người; tự đi lại khi người rời khỏi khung hình	200

3.4.4.4. Kỹ thuật Tối ưu hóa đường ống Suy luận hệ thống thời gian thực

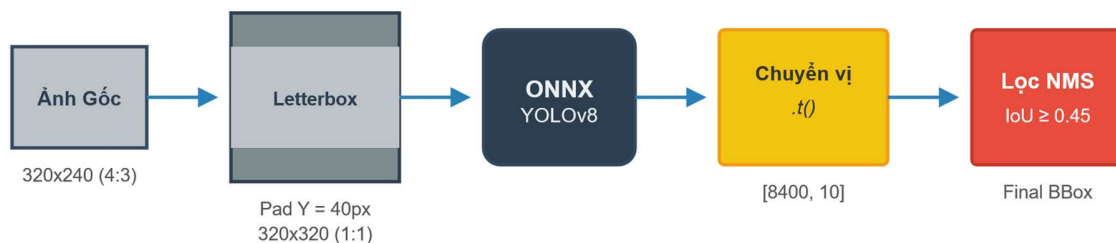
Mô hình sau huấn luyện được chuyển đổi từ cấu trúc động sang đồ thị tính toán tĩnh định dạng ONNX, giữ độ chính xác số thực dấu phẩy động 32-bit (FP32) để tương thích với Raspberry Pi không có GPU. Định dạng này cho phép module dnn của OpenCV nạp và chạy suy luận trực tiếp bằng C++, không cần đến runtime của framework huấn luyện.

Đường ống suy luận thời gian thực gồm hai giai đoạn xử lý hình học chính, minh họa ở Hình 3.16:

- Tiền xử lý bằng đệm ảnh đối xứng: khung hình thô tỷ lệ 4:3 (320×240) được chèn hai dải viền xám trung tính có giá trị màu cố định 114, 114, 114, mỗi dải cao 40 pixel, ở cạnh trên và cạnh dưới, đưa ảnh về kích thước vuông 320×320 theo đúng yêu cầu đầu vào của mạng, mà không làm biến dạng tỷ lệ hình học của biển báo.

- Hậu xử lý bằng chuyển vị tensor và lọc trùng: tensor đầu ra thô có dạng $[1, 10, 8400]$, gồm 10 thuộc tính 4 tọa độ khung bao và 6 điểm tin cậy lớp trên 8400 vị trí dự đoán. Tensor được chuyển vị thành dạng $[8400, 10]$, đưa các thuộc tính của từng khung bao về cùng một vùng nhớ liên tục, giúp việc quét và lọc theo từng dự đoán hiệu quả hơn cho CPU. Các khung bao trùng lặp sau đó được loại bỏ bằng thuật toán triết tiêu cực đại với ngưỡng IoU 0,45.

Việc bù các biến động về độ tương phản và ánh sáng yếu của sa hình được xử lý ngay trong pha huấn luyện, bằng kỹ thuật biến đổi dải màu kết hợp trộn ảnh đa tỷ lệ, thay vì xử lý lại tại thời điểm suy luận. Nhờ vậy, đường ống thời gian thực không cần thêm bước cân bằng lược đồ quang độ trước khi đưa ảnh vào mạng, giúp tiết kiệm chu kỳ xử lý cho vi xử lý trung tâm.



Hình 3.13. Pipeline Tiền xử lý Letterbox và Hậu xử lý Tensor

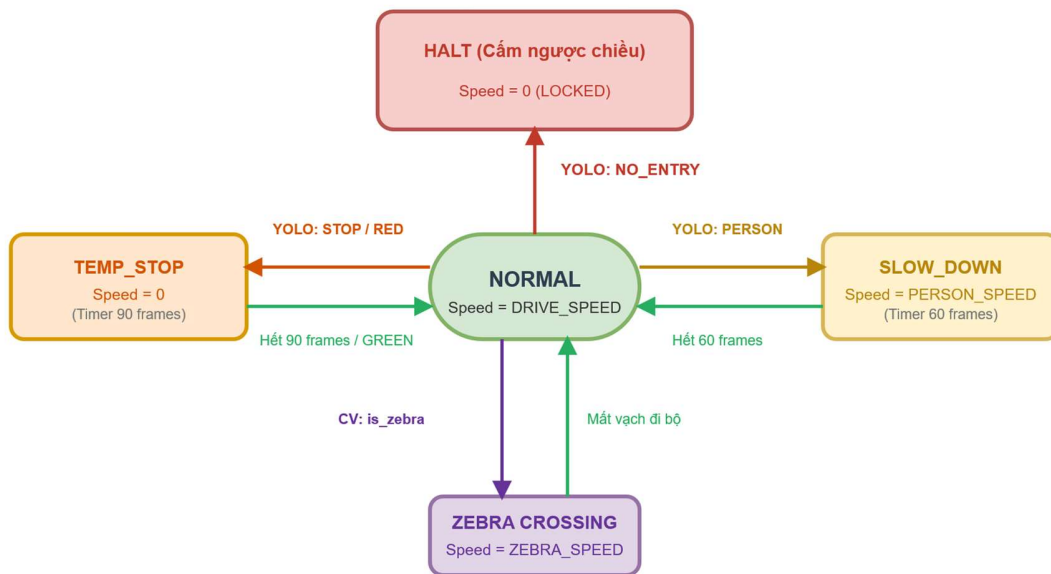
3.4.5. Khối Phân xử Hành vi và Lập kế hoạch

3.4.5.1. Mô hình Máy trạng thái hữu hạn

Hệ thống phân xử hành vi của phương tiện được thiết kế theo kiến trúc máy trạng thái hữu hạn. Ở điều kiện tiêu chuẩn, phương tiện vận hành ở trạng thái bình thường với vận tốc cố định ở mức 8. Các trạng thái can thiệp được kích hoạt dựa trên tín hiệu phân loại từ mạng nơ-ron, với hai nhóm cơ chế xử lý khác nhau tùy theo loại đối tượng dừng theo sự hiện diện của đối tượng, hoặc chột-giữ trạng thái dừng:

- Biển STOP: khi biển còn xuất hiện trong khung hình, vận tốc bị ép về 0; ngay khi biển khuất khỏi tầm nhìn của camera, xe tự động trở lại trạng thái bình thường. Cơ chế dừng dựa trên sự hiện diện của biển trong khung hình, không sử dụng bộ đếm thời gian cố định.

- Đèn đỏ: sau khi đèn đỏ được xác nhận qua cơ chế bỏ phiếu hai khung hình liên tiếp, hệ thống chốt trạng thái dừng và ép vận tốc về 0, duy trì kể cả khi đèn đã trôi ra khỏi khung hình. Trạng thái chốt được giải phóng khi nhận diện được đèn xanh, hoặc tự nhả sau 8 giây như một cơ chế an toàn phòng khi không quan sát được đèn xanh.
- Đèn vàng: trong cấu hình hiện tại, hệ thống nhận diện được đèn vàng nhưng không thay đổi vận tốc; xe tiếp tục vận hành bình thường và bám làn như thường lệ.
- Người đi bộ: khi còn phát hiện người trong khung hình, vận tốc bị ép về 0; xe tự động đi lại khi người đã rời khỏi tầm nhìn của camera. Cơ chế này giúp xe dừng hẳn trước người đi bộ cho đến khi lối đi thông thoáng.
- Biển cấm đi ngược chiều: khi biển còn xuất hiện trong khung hình, vận tốc bị ép về 0; xe tự động đi lại khi biển khuất khỏi khung hình, tương tự cơ chế dừng-theo-hiện-diện của biển STOP và người đi bộ.



Hình 3.14. Sơ đồ chuyển đổi trạng thái của xe

3.4.5.2. Cơ chế hợp nhất thông tin

Dữ liệu từ luồng thị giác máy tính và luồng nhận thức ngữ nghĩa được hợp nhất theo nguyên tắc phân quyền độc lập:

- Luồng CV: kiểm soát cơ cấu bề lái trong mọi trạng thái của FSM thông qua giá trị sai lệch quỹ đạo. Luồng này cũng được phép giảm vận tốc về mức 5 một cách độc

lập khi phát hiện khu vực vạch đi bộ, áp dụng song song với bất kỳ trạng thái nào khác của FSM.

- Luồng YOLO: quyết định trạng thái hiện tại của FSM và áp đặt lệnh vận tốc mục tiêu tương ứng lên phương tiện.

Để tín hiệu từ mạng YOLO đủ điều kiện tham gia vào bộ phân xử, hệ thống áp dụng hai lớp lọc: ngưỡng tin cậy tối thiểu 0,25 (lọc tại bước hậu xử lý NMS) và ngưỡng diện tích khung bao tối thiểu 200 pixel vuông (lọc tại bước phân loại kích bản). Hai lớp lọc này giúp giảm đáng kể các lệnh kích hoạt giả từ các biển báo nằm ngoài tầm ảnh hưởng vật lý của xe.

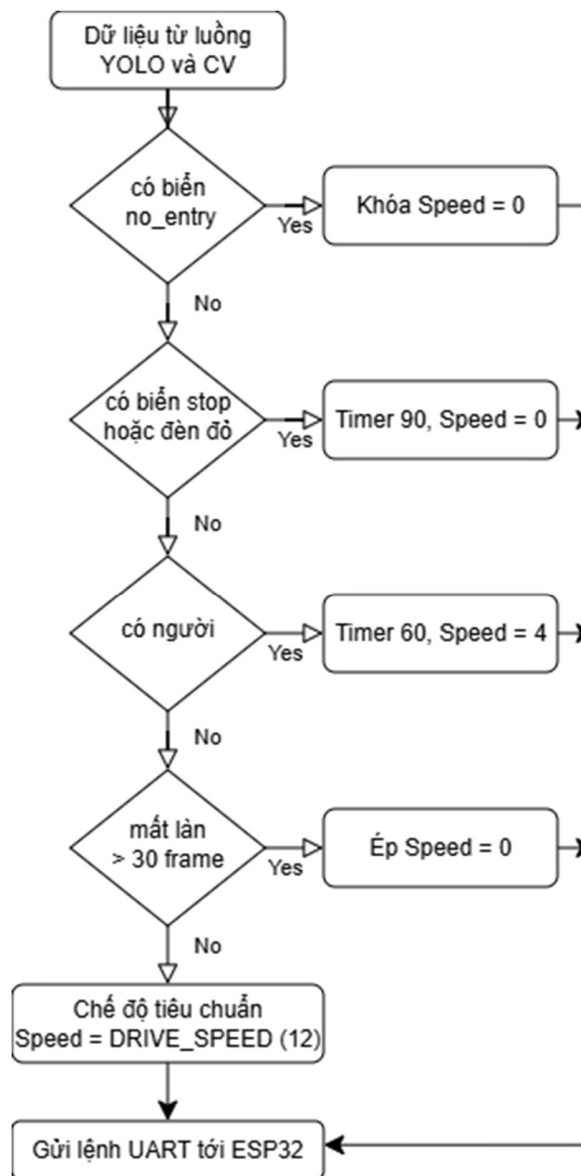
3.4.5.3. Logic phân xử ưu tiên

Khi nhiều sự kiện xuất hiện cùng lúc trên một khung hình, các nhánh quyết định từ YOLO được xét theo thứ tự ưu tiên cố định: biển cấm đi ngược chiều, biển STOP, đèn đỏ, người đi bộ, đèn vàng, rồi đèn xanh. Bốn nhóm đầu gồm biển cấm, biển STOP, đèn đỏ, người đi bộ đều dẫn đến lệnh dừng bằng cách ép vận tốc về 0; trong đó riêng đèn đỏ dùng cơ chế chốt-giữ đến khi thấy đèn xanh hoặc hết 8 giây, còn biển cấm, biển STOP và người đi bộ dừng theo sự hiện diện của đối tượng trong khung hình. Đèn vàng và đèn xanh tương ứng với chế độ vận hành bình thường.

Song song với nhánh quyết định trên, hệ thống áp dụng hai lớp giới hạn vận tốc độc lập, có hiệu lực sau khi FSM đã tính ra vận tốc, bất kể nhánh nào vừa được chọn:

- Nếu phát hiện khu vực vạch đi bộ, hệ thống giữ lái thẳng và giới hạn vận tốc ở mức 5 để bò chậm qua vạch. Lưu ý rằng chức năng nhận diện vạch đi bộ hiện đang được tắt trong cấu hình chạy thử nghiệm vì dễ nhầm với các vạch làn đường, và sẽ được bật lại sau khi tinh chỉnh chuẩn xác.
- Nếu phần mềm mất dấu vạch kẻ đường liên tục quá 30 khung hình, vận tốc bị ép thẳng về 0, kể cả khi nhánh quyết định trước đó vừa đặt một giá trị. Cơ chế này hoạt động độc lập với FSM, nhằm tránh phương tiện mất kiểm soát động lực học khi trượt ra khỏi sa hình.

Quyết định vận tốc và góc lái cuối cùng được đóng gói thành một khung dữ liệu văn bản và truyền qua UART xuống vi điều khiển chấp hành.



Hình 3.15. Lưu đồ thuật toán ra quyết định

3.4.6. Khối Điều khiển Động học cấp thấp và Giao tiếp

3.4.6.1. Điều khiển tốc độ vòng hở và định hướng vòng kín

Trong mô hình, tốc độ xe được điều khiển bằng độ rộng xung cấp cho mạch cầu H, theo chế độ vòng hở: lệnh tốc độ từ máy tính nhúng được giới hạn trong một dải an toàn và đưa thẳng tới động cơ, không có phản hồi tốc độ. Lý do là tín hiệu encoder trong điều kiện thử nghiệm chưa ổn định, nên để đảm bảo xe chạy đều và ổn định khi demo, đề tài chọn chạy vòng hở ở vận tốc thấp.

Hệ thống vẫn được thiết kế sẵn bộ điều khiển vòng kín PID kèm thành phần bù tiến và cơ chế chống tràn tích phân, đặt tại vi điều khiển. Khi tín hiệu encoder được hoàn thiện, bộ

điều khiển này sẽ giữ tốc độ ổn định trên dốc và khi tải thay đổi. Dạng rời rạc của luật PID như sau:

$$u[k] = K_p \cdot e[k] + K_i \cdot \sum e[j] \cdot T_s + K_d \cdot (e[k] - e[k-1]) / T_s$$

với $e[k]$ là sai số tốc độ tại bước k , T_s là chu kỳ lấy mẫu; K_p , K_i , K_d là các hệ số. Việc hoàn thiện điều khiển vòng kín bằng encoder được xếp vào hướng phát triển của đề tài.

3.4.6.2. Giao thức truyền thông UART và Kỹ thuật tối ưu hóa băng thông

Tín hiệu điều khiển được truyền từ bộ xử lý trung tâm tới vi điều khiển chấp hành qua UART, dưới dạng một khung văn bản ASCII phân tách bằng dấu phẩy và kết thúc bằng ký tự xuống dòng. Độ dài khung thay đổi theo giá trị cụ thể của góc lái và tốc độ, nhưng luôn bị chặn trên bởi số byte tối đa của từng trường, giúp giữ băng thông và độ trễ truyền tin ở mức dự đoán được.

Bảng 3.13. Cấu trúc khung truyền dữ liệu qua UART

Trường	Kiểu dữ liệu	Phạm vi giá trị	Ý nghĩa
Chế độ chuyển động	Ký tự ASCII (1 byte)	{F, B, S}	Chỉ định hướng chuyển động (Tiến, Lùi, Dừng)
Góc lái	Số nguyên ASCII (tối đa 4 byte)	[-100, 100]	Góc bẻ lái chuẩn hóa cho Servo cơ cầu lái
Tốc độ	Số nguyên ASCII (tối đa 2 byte)	[0, 30]	Vận tốc mục tiêu quy đổi từ Encoder bánh sau
Ký tự kết thúc khung	Ký tự (1 byte)	\n	Đánh dấu kết thúc một khung truyền

3.4.6.3. Thiết kế cơ chế điều khiển chấp hành trên ESP32

Vi điều khiển ESP32 vận hành vòng lặp điều khiển thời gian thực với chu kỳ $T_s = 20\text{ms}$, tần số 50 Hz dựa trên nền tảng FreeRTOS, đảm bảo tính ổn định động lực học cho phương tiện.

- Phân hệ Cơ cấu lái: Góc lái chỉ lệnh được ánh xạ trực tiếp vào không gian hoạt động vật lý của Servo quanh góc trung tâm 68° . Hệ thống tích hợp khâu giới hạn tốc độ góc để triệt tiêu hiện tượng sụt áp và giật lắc cơ khí do lệnh lái thay đổi đột ngột.

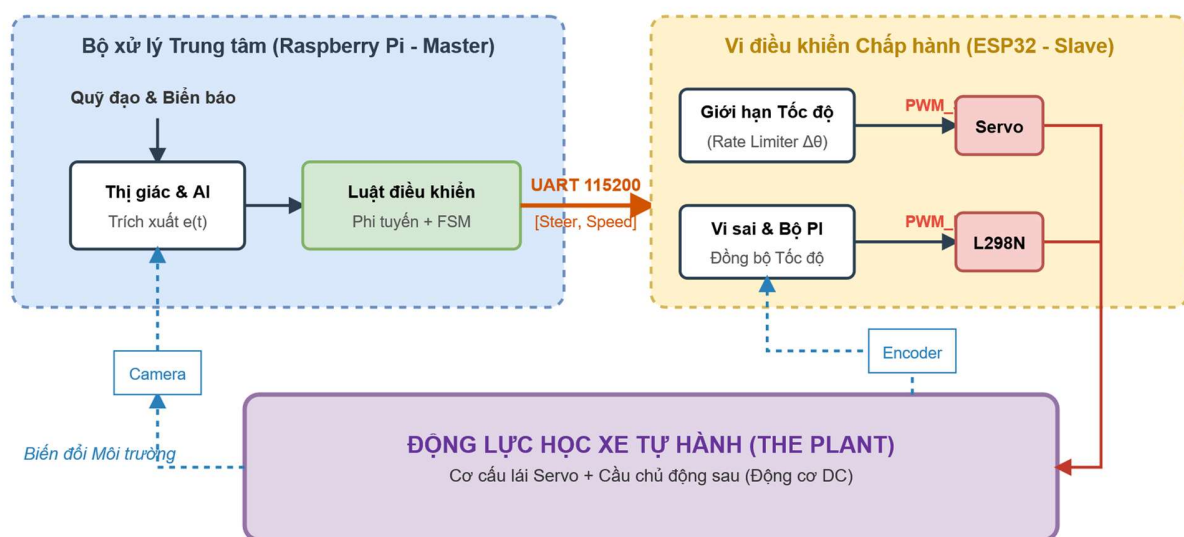
Góc Servo được ép buộc không thay đổi quá 3° sau mỗi chu kỳ lấy mẫu, tương đương tốc độ quay tối đa $150^\circ/\text{giây}$.

- Phân hệ Vi sai và Điều khiển tốc độ: Tốc độ hai bánh sau được đồng bộ bằng bộ điều khiển PI độc lập. Tín hiệu đầu ra động cơ tổng hợp từ khâu truyền thẳng bù ma sát tĩnh và khâu PI bù sai số. Thành phần tích phân được kẹp giới hạn trong dải ± 200 để chống bão hòa khi bánh xe chịu tải cao.
- Vi sai điện tử: Khi thực thi góc lái, tốc độ bánh trong được tự động suy giảm theo hệ số vi sai 0.55 để đảm bảo bán kính quay vòng chính xác, hạn chế trượt bánh.
- Cơ chế giám sát an toàn: Hệ thống duy trì một bộ định thời giám sát kết nối. Nếu vi điều khiển không nhận được gói tin UART hợp lệ trong vòng 1000ms, cờ an toàn kích hoạt, lập tức ép giá trị PWM về 0 nhằm dừng xe khẩn cấp.

Toàn bộ quá trình từ ước lượng quỹ đạo, qua xử lý tín hiệu, truyền thông UART, đến phản hồi tốc độ qua Encoder tạo thành một vòng lặp kín, đảm bảo phương tiện bám sát quỹ đạo thiết kế với độ trễ tối thiểu và tính ổn định cơ khí cao.

Bảng 3.14. Tham số cấu hình bộ điều khiển và giới hạn chấp hành

Tham số cấu hình	Ký hiệu	Giá trị	Vai trò vận hành
Hệ số tỷ lệ	K_p	1.2	Tác động bù trừ tức thời theo sai lệch phản hồi
Hệ số tích phân	K_i	0.08	Bù sai lệch tĩnh do ma sát
Bù truyền thẳng	FF_0	100/255	Vượt quán tính tĩnh động cơ
Chu kỳ điều khiển	T_s	20ms	Tần số vòng lặp cập nhật trạng thái
Góc trung tâm	θ_c	68°	Vị trí hướng thẳng của Servo
Biên độ góc lái	θ_L/θ_R	$8^\circ/128^\circ$	Giới hạn vật lý của cơ cấu lái
Tốc độ thay đổi góc	$\Delta\theta$	$3^\circ/20\text{ms}$	Chống rung lắc cho Servo
Hệ số vi sai	$diff$	0.55	Tỷ lệ ép tốc độ bánh trong khi vào cua
Thời gian Watchdog	T_{WD}	1000ms	Ngắt an toàn khi mất kết nối



Hình 3.16. Sơ đồ vòng lặp điều khiển phản hồi kín

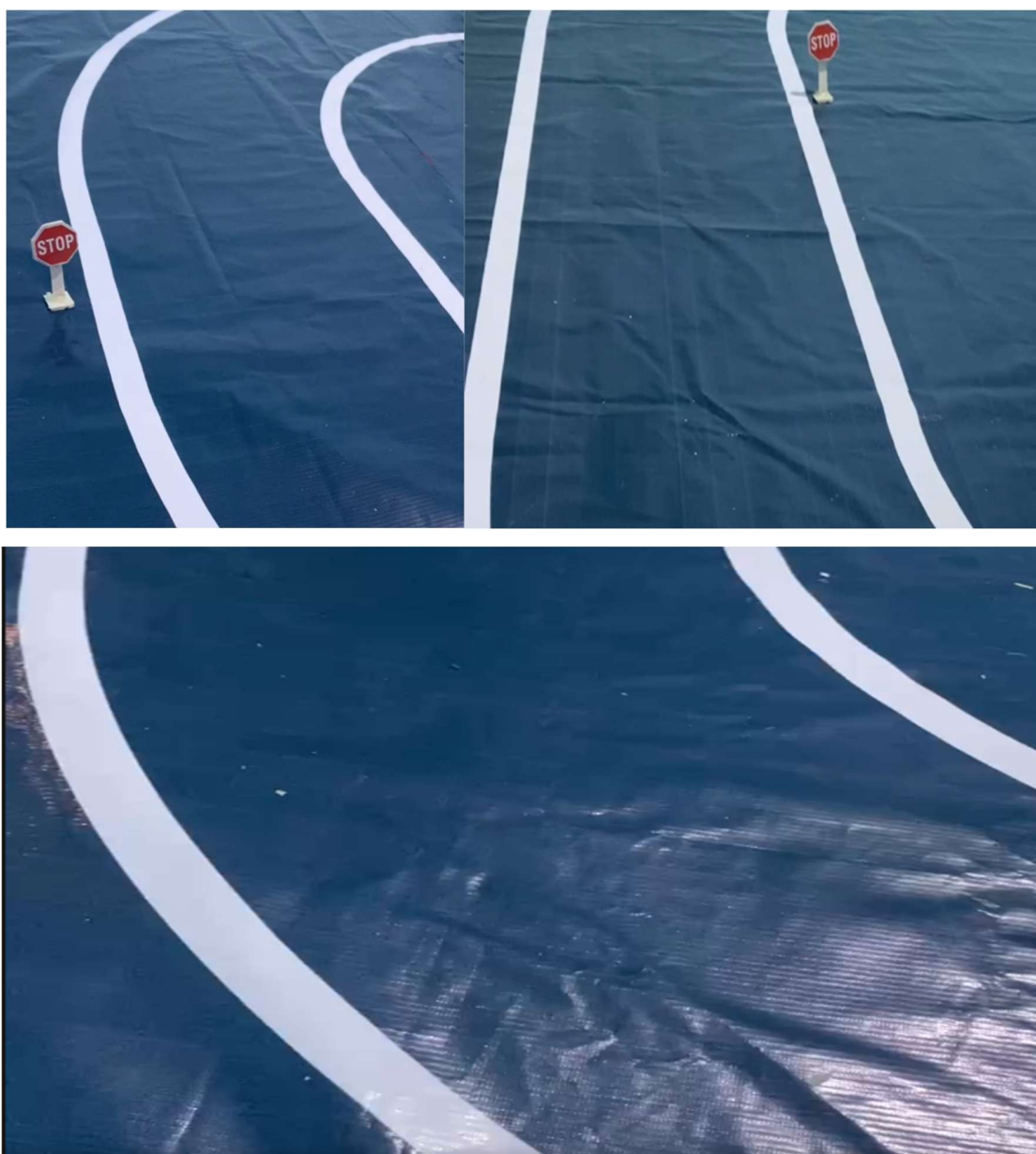
Chương 4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương 4 trình bày toàn bộ kết quả đạt được sau quá trình thiết kế, lắp ráp và tinh chỉnh mô hình xe tự hành. Nội dung được tổ chức theo đúng trình tự ba tầng chức năng đã đặt ra ở Chương 3: trước hết mô tả môi trường và điều kiện thử nghiệm cùng kết quả lắp ráp phần cứng hoàn chỉnh; tiếp theo là kết quả của tầng nhận thức gồm khối phát hiện làn đường bằng thị giác máy tính và khối phát hiện biển báo, đèn tín hiệu bằng học sâu; sau đó là kết quả của tầng chấp hành điều khiển lái và tốc độ qua vi điều khiển ESP32; cuối cùng là kết quả khi tích hợp toàn hệ thống chạy thực tế trên sa hình, các vấn đề kỹ thuật đã gặp cùng giải pháp khắc phục, và phần đánh giá chung kèm những hạn chế còn tồn tại. Mỗi kết quả đều được minh họa bằng hình ảnh thực nghiệm và đối chiếu với mục tiêu đã đề ra.

4.1. Môi trường và điều kiện thử nghiệm

4.1.1. Sa hình thử nghiệm

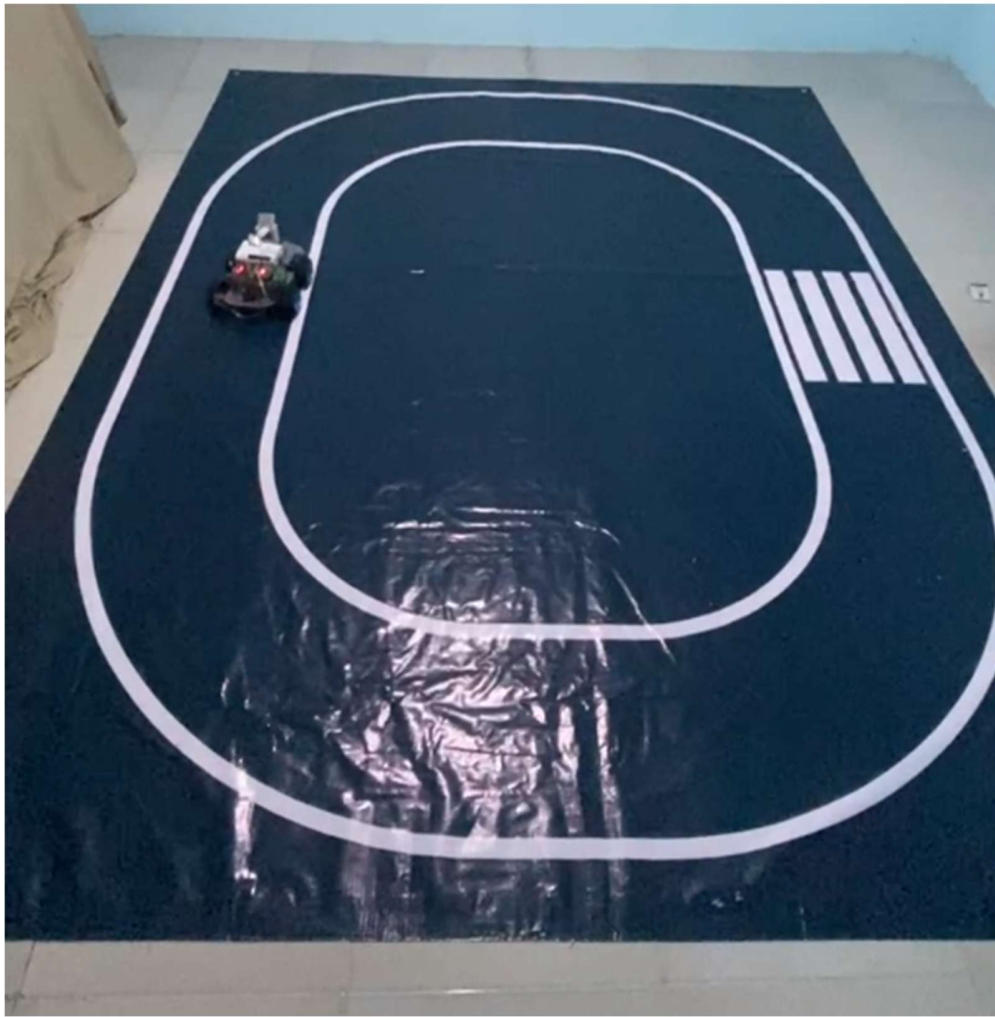
Toàn bộ thử nghiệm được thực hiện trên một sa hình - mô hình đường thu nhỏ cố định đặt trong phòng, mô phỏng một tuyến đường khép kín gồm các đoạn thẳng nối với nhiều khúc cua trái và cua phải có bán kính khác nhau. Sa hình được trải phẳng và giữ cố định trong suốt quá trình thử nghiệm để bảo đảm các lần đo có cùng điều kiện. Tuyến đường khép kín cho phép xe chạy nhiều vòng liên tục, thuận tiện cho việc đánh giá độ ổn định của khả năng bám làn qua nhiều lượt.



Hình 4.1. Sơ hình thử nghiệm với các đoạn thẳng và khúc cua

4.1.2. Điều kiện ánh sáng

Mô hình được thử nghiệm trong nhà với ánh sáng tương đối ổn định và đồng đều. Điều kiện ánh sáng được giữ nhất quán trong suốt quá trình thử nghiệm để bộ tham số xử lý ảnh hoạt động ổn định. Đây là phạm vi vận hành đã xác định của mô hình.



Hình 4.2. Môi trường thử nghiệm trong phòng

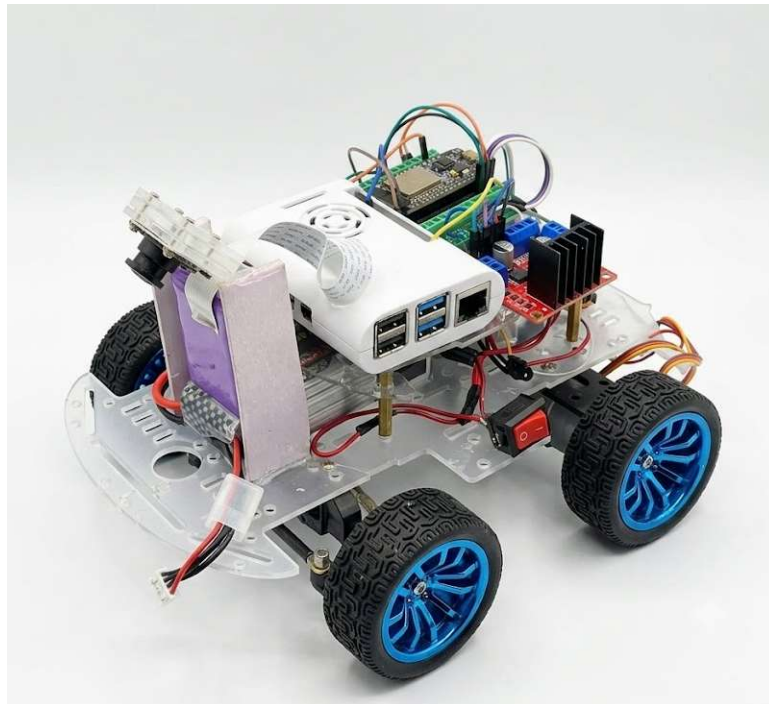
4.1.3. Cấu hình phần cứng và phần mềm khi thử nghiệm

Bảng 4.1. Cấu hình phần cứng và phần mềm trong quá trình thử nghiệm

Thành phần	Cấu hình sử dụng khi đo
Máy tính nhúng	Raspberry Pi 4 Model B (RAM 4 GB), hệ điều hành Raspberry Pi OS 64-bit
Vi điều khiển	ESP32 DevKit, giao tiếp với Pi qua UART tốc độ 115200 baud
Camera	OV5647, thu khung hình và đưa về kích thước xử lý 320×240 điểm ảnh
Thư viện thị giác	OpenCV (C++), mô-đun DNN nạp mô hình ONNX
Mô hình học sâu	YOLOv8n, 6 lớp, định dạng ONNX, đầu vào 320×320
Nguồn Raspberry Pi	Nguồn 5V/3A riêng, tách khỏi nguồn động cơ
Nguồn động cơ	Pin LiPo 3S qua mạch hạ áp cấp cho cầu H L298N và servo
Công cụ quan sát	Luồng video qua giao thức truyền ảnh trên cổng 8080 (xem bảng trình duyệt)

4.2. Kết quả lắp ráp phần cứng hoàn chỉnh

Mô hình xe được lắp ráp hoàn chỉnh theo kiến trúc phân tầng đã thiết kế ở Chương 3. Khung gầm xe sử dụng cơ cấu lái Ackermann (bánh trước dẫn hướng bằng một servo, hai bánh sau dẫn động qua động cơ một chiều và cầu H). Toàn bộ khối tính toán và chấp hành được bố trí thành các tầng gọn gàng trên khung: tầng dưới đặt pin và mạch nguồn, tầng giữa đặt mạch cầu H và vi điều khiển ESP32, tầng trên đặt Raspberry Pi 4 và camera hướng về phía trước với góc cúi xuống mặt đường để quan sát làn.



Hình 4.3. Mô hình xe tự hành sau khi lắp ráp hoàn chỉnh

Bảng 4.2 liệt kê các thành phần phần cứng chính trên mô hình hoàn chỉnh cùng vai trò của từng thành phần trong hệ thống.

Bảng 4.2. Các thành phần phần cứng chính trên mô hình hoàn chỉnh

Thành phần	Số lượng	Vai trò trong hệ thống
Raspberry Pi 4	1	Tầng nhận thức và quyết định: xử lý ảnh, phát hiện làn, suy luận mô hình
ESP32 DevKit	1	Tầng chấp hành: nhận lệnh UART, điều khiển servo lái và động cơ
Camera OV5647	1	Cảm biến hình ảnh, thu khung hình mặt đường phía trước
Cầu H L298N	1	Mạch công suất đảo chiều và điều tốc hai động cơ sau
Động cơ DC + hộp số	2	Dẫn động hai bánh sau
Servo lái	1	Dẫn hướng bánh trước theo cơ cấu Ackermann
Pin LiPo 3S	1	Cấp nguồn cho động cơ và servo qua mạch hạ áp
Nguồn 5V/3A riêng	1	Cấp nguồn độc lập cho Raspberry Pi

4.3. Kết quả khối phát hiện làn đường

Khối phát hiện làn đường là thành phần được đầu tư tinh chỉnh nhiều nhất, vì nó quyết định trực tiếp khả năng tự bám đường của xe. Khối này được viết hoàn toàn bằng ngôn ngữ C++ với thư viện OpenCV, chạy trực tiếp trên Raspberry Pi. Toàn bộ quy trình xử lý từ khung hình thô tới góc lái được chia thành các bước rõ ràng, trình bày lần lượt dưới đây.

4.3.1. Quy trình xử lý ảnh phát hiện làn

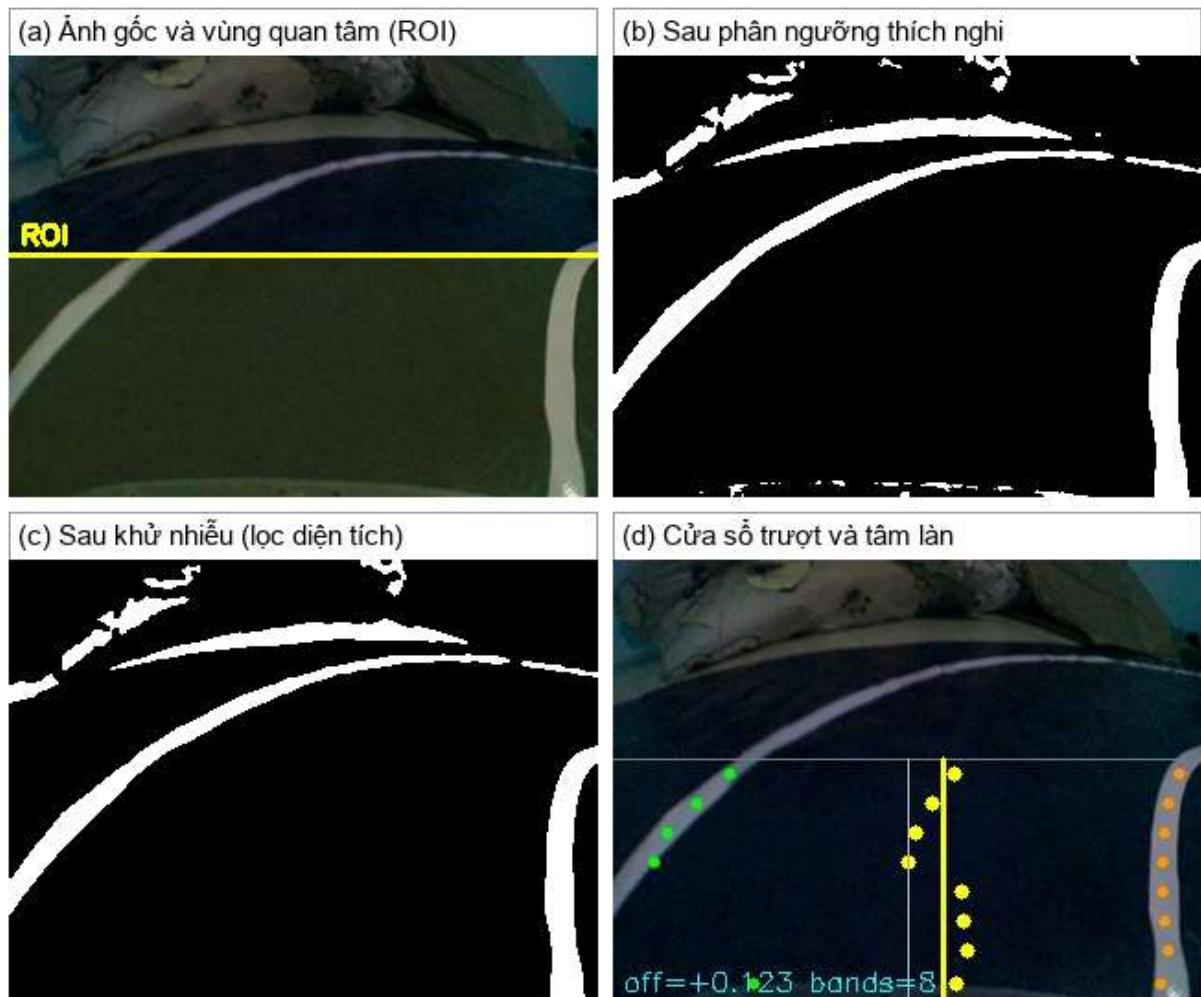
Mỗi khung hình màu thu từ camera với kích thước 320×240 điểm ảnh được đưa qua chuỗi xử lý gồm bốn bước chính:

- Cắt vùng quan tâm: chỉ giữ lại phần phía dưới khung hình - nơi chứa mặt đường ngay trước xe. Hệ thống giữ lại phần ảnh từ khoảng 45% chiều cao trở xuống, vừa loại bỏ phần nền phía trên vừa bảo đảm tầm nhìn đủ xa để xe phản ứng kịp với khúc cua.

- Phân ngưỡng thích nghi để tách vùng sáng: ảnh xám được làm mờ rồi đưa qua phép phân ngưỡng thích nghi cục bộ. Khác với phân ngưỡng cố định, phương pháp này so sánh mỗi điểm ảnh với độ sáng trung bình của vùng lân cận, nhờ đó tách được vạch làn ngay cả khi độ sáng toàn khung thay đổi nhẹ. Kết quả là một ảnh nhị phân làm nổi bật các vùng sáng tương ứng với vạch làn.

- Khử nhiễu: dùng phép hình thái học và lọc theo diện tích để loại các vùng sáng nhỏ, rời rạc không thuộc vạch làn - sẽ trình bày ở mục 4.3.2.

- Quét dải ngang theo cửa sổ trượt và tính độ lệch: vùng ROI được chia thành 8 dải ngang; trên mỗi dải, thuật toán xác định tâm của vạch trái và vạch phải, rồi từ đó suy ra tâm làn và độ lệch của xe so với tâm làn.



Hình 4.4. Kết quả từng bước trong quy trình phát hiện làn đường

4.3.2. Khử nhiễu trong ảnh nhị phân

Sau bước phân ngưỡng, ngoài vạch làn trong ảnh nhị phân còn lẫn một số vùng sáng nhỏ, rời rạc do nhiễu. Nếu giữ nguyên, các vùng này có thể bị nhầm là vạch và làm sai lệch việc xác định tâm làn. Hệ thống khử nhiễu qua hai bước:

- Phép đóng hình thái học: nối liền các đoạn vạch bị đứt do nhiễu, giúp vạch làn liền mạch và rõ nét hơn.

- Lọc theo diện tích vùng liên thông: phân tích các vùng trắng liên thông và loại bỏ những vùng có diện tích nhỏ hơn ngưỡng - vốn là các đốm nhiễu; chỉ giữ lại các vùng đủ lớn tương ứng với vạch làn.

Sau khi khử nhiễu, ảnh nhị phân sạch hơn, chủ yếu còn lại vạch làn, tạo đầu vào ổn định cho bước quét cửa sổ trượt. Hình 4.5 so sánh ảnh trước và sau khi khử nhiễu.



Hình 4.5. Hiệu quả bước khử nhiễu trong ảnh nhị phân

4.3.3. Xác định độ lệch và bám làn qua đoạn thẳng, khúc cua

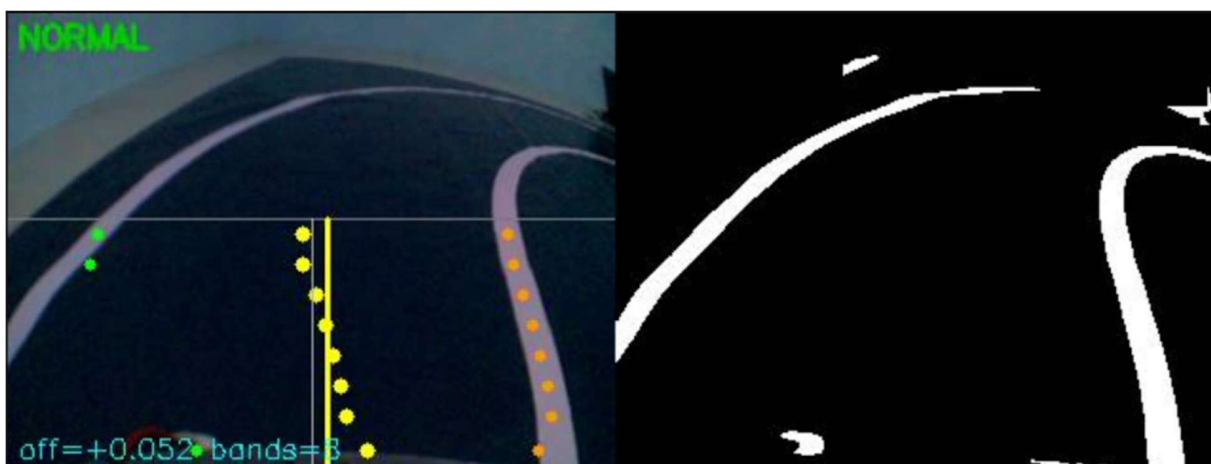
Sau khi có mặt nạ làn sạch, thuật toán quét cửa sổ trượt trên 8 dải ngang để bám theo vạch làn. Ở mỗi dải, tâm của cụm điểm trắng được xem là vị trí vạch; cửa sổ tìm kiếm của dải trên được đặt quanh vị trí vạch đã tìm thấy ở dải dưới, nhờ đó bám liên tục được vạch cong qua khúc cua. Khi chỉ phát hiện được một vạch (thường gặp ở cua gắt, lúc một vạch ra khỏi khung), thuật toán suy ra tâm làn bằng cách dịch vào trong một khoảng bằng nửa bề rộng làn ước lượng.

Độ lệch ngang của xe so với tâm làn, ký hiệu là e , được chuẩn hóa theo nửa bề rộng khung hình nên có giá trị nằm trong khoảng $[-1, +1]$: giá trị âm khi xe lệch sang trái, dương khi lệch sang phải. Độ lệch này được tính theo công thức (4.1):

$$e = (x_{\text{tâm_làn}} - x_{\text{giữa_khung}}) / (W / 2) \quad (4.1)$$

trong đó $x_{\text{tâm_làn}}$ là hoành độ tâm làn ước lượng, $x_{\text{giữa_khung}}$ là hoành độ tâm khung hình và W là chiều rộng khung hình (điểm ảnh). Độ lệch e sau đó được chuyển thành lệnh góc lái, trình bày ở mục 4.5.1.

Trên đoạn thẳng, xe bám tâm làn ổn định, độ lệch dao động nhỏ quanh không. Trên khúc cua, nhờ cửa sổ tìm kiếm được nói rộng và cơ chế suy luận một vạch, xe vẫn giữ được hướng đi men theo vạch. Hình 4.6 minh họa hành vi bám làn ở hai tình huống tiêu biểu.



Hình 4.6. Kết quả bám làn trên đoạn thẳng và trong khúc cua

4.3.4. Cơ chế giữ hướng khi tạm mất làn

Trong thực tế, có những khung hình thuật toán không phát hiện đủ số dải làn hợp lệ (ví dụ khi vào cua gắt cả hai vạch tạm thời mờ hoặc lệch khỏi khung). Nếu để xe dừng hoặc quay xe ngẫu nhiên trong những khung này, xe sẽ giạt cục và mất ổn định. Hệ thống xử lý bằng cách giữ nguyên góc lái tốt gần nhất và cho xe tiếp tục bò chậm trong một số khung hình nhất định; chỉ khi mất làn liên tục vượt quá ngưỡng cho phép thì xe mới dừng hẳn để đảm bảo an toàn. Nhờ cơ chế này, xe vượt qua được các đoạn mất làn ngắn mà vẫn giữ được quỹ đạo mượt.

4.3.5. Bộ tham số cấu hình của khối phát hiện làn

Toàn bộ hành vi của khối phát hiện làn được điều khiển bằng một bộ tham số cấu hình. Bảng 4.3 liệt kê các tham số quan trọng nhất cùng giá trị đang sử dụng và ý nghĩa của chúng. Bộ giá trị này là kết quả của quá trình tinh chỉnh thực nghiệm theo nguyên tắc mỗi lần chỉ thay đổi một tham số rồi quan sát kết quả, nhằm tránh nhiều thay đổi chồng lên nhau gây khó đánh giá.

Bảng 4.3. Bộ tham số chính của khối phát hiện làn đường

Tham số	Giá trị	Ý nghĩa
Vùng quan tâm (ROI)	45% dưới	Giữ phần dưới khung hình chứa mặt đường trước xe
Kích thước ô phân ngưỡng	41	Cửa sổ cục bộ cho phân ngưỡng thích nghi (lớn hơn bề rộng vạch)
Hằng số phân ngưỡng C	-15	Âm: chỉ giữ điểm sáng hơn nền cục bộ (tách vạch)
Diện tích đốm tối thiểu	80	Vùng trắng nhỏ hơn bị xóa như nhiễu

Số dải quét	8	Chia ROI thành 8 dải ngang để bám mượt
Nửa bề rộng làn	0,34	Tỉ lệ theo bề rộng khung, dừng khi chỉ thấy 1 vạch
Độ rộng cửa sổ tìm kiếm	0,18	Nới rộng giúp bám cua gắt
Vùng chết góc lái	0,04	Độ lệch nhỏ hơn coi như đi thẳng (chống rung)
Hệ số khuếch đại lái	120	Độ mạnh phản ứng lái theo độ lệch
Hệ số làm mượt lái	0,4	Lọc trung bình trượt làm mượt góc lái
Tốc độ đoạn thẳng / cua	8 / 5	Giảm tốc khi vào cua để ổn định hơn
Ngưỡng dừng khi mất làn	30 khung	Bò chậm rồi mới dừng nếu mất làn kéo dài

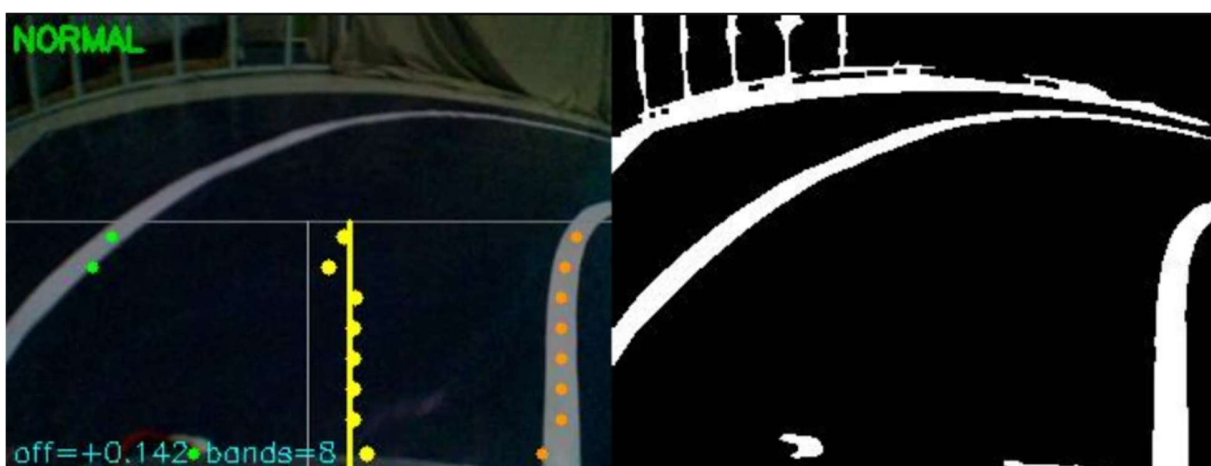
4.3.6. Đánh giá định lượng khối phát hiện làn

Khối phát hiện làn chạy ổn định ở tốc độ khung hình đủ để điều khiển xe ở vận tốc thấp đã thiết lập. Để đánh giá khả năng bám làn, mô hình được cho chạy nhiều lượt liên tục trên sa hình và ghi nhận số vòng xe đi trọn vẹn trước khi lệch ra ngoài. Kết quả cho thấy với bộ tham số ổn định, xe đi được khoảng 2-3 vòng liên tục bám đúng làn; các lỗi lệch chủ yếu xảy ra ở khúc cua gắt khi vạch bên ngoài ra khỏi khung hình. Bảng 4.4 tóm tắt kết quả định tính theo từng tình huống.

Bảng 4.4. Đánh giá định tính khả năng bám làn theo từng tình huống

Tình huống	Kết quả	Ghi chú
Đoạn thẳng	Tốt	Bám tâm làn ổn định, độ lệch dao động nhỏ
Cua rộng	Tốt	Cửa sổ trượt bám vạch cong liên tục
Cua gắt	Trung bình	Đôi lúc lệch khi vạch ngoài ra khỏi khung hình
Mất làn ngắn	Tốt	Giữ hướng và bò chậm vượt qua

Để thuận tiện cho việc quan sát và chỉnh chỉnh khi không có màn hình gắn trực tiếp, hệ thống phát trực tiếp khung hình xử lý; ảnh gốc kèm mặt nạ làn qua mạng để xem trên trình duyệt. Công cụ này cho phép chẩn đoán trực quan các trường hợp lệch làn, là cơ sở để điều chỉnh bộ tham số ở Bảng 4.3.



Hình 4.7. Giao diện quan sát trực tuyến khung hình xử lý phục vụ gỡ lỗi

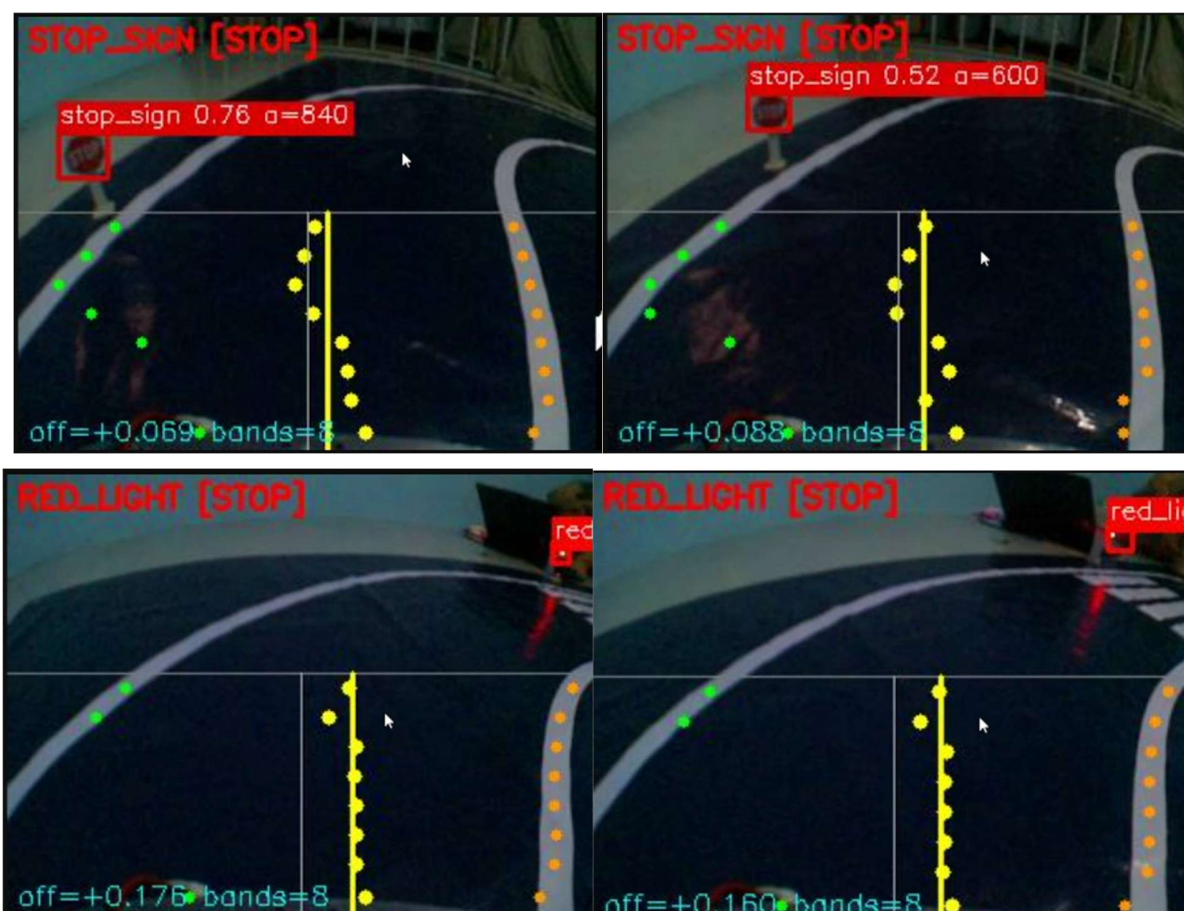
4.4. Kết quả khôi phục phát hiện biển báo và đèn tín hiệu

Bên cạnh bám làn, xe còn được trang bị khối nhận diện biển báo và đèn tín hiệu giao thông bằng mô hình học sâu YOLOv8n, chạy suy luận trực tiếp trên Raspberry Pi qua mô-đun mạng nơ-ron của OpenCV với mô hình ở định dạng ONNX. Mô hình được huấn luyện để nhận 6 lớp đối tượng phù hợp với sa hình, liệt kê ở Bảng 4.5.

Bảng 4.5. Sáu lớp đối tượng mà mô hình YOLOv8n nhận diện

STT	Lớp đối tượng	Ý nghĩa
1	Biển báo dừng (stop)	Yêu cầu xe dừng lại
2	Biển cấm đi (no entry)	Cấm xe đi vào
3	Đèn đỏ	Tín hiệu dừng
4	Đèn vàng	Tín hiệu chuẩn bị dừng / giảm tốc
5	Đèn xanh	Tín hiệu được phép đi
6	Người đi bộ	Phát hiện người để nhường đường

Khung hình đưa vào mô hình được đệm về kích thước vuông 320×320 điểm ảnh nhằm giữ đúng tỉ lệ và tránh méo đối tượng. Kết quả suy luận được lọc theo ngưỡng độ tin cậy 0,25 và khử trùng lặp khung bao bằng phép loại bỏ phi cực đại với ngưỡng 0,45. Mỗi đối tượng phát hiện được vẽ khung bao kèm tên lớp và độ tin cậy để tiện kiểm chứng.

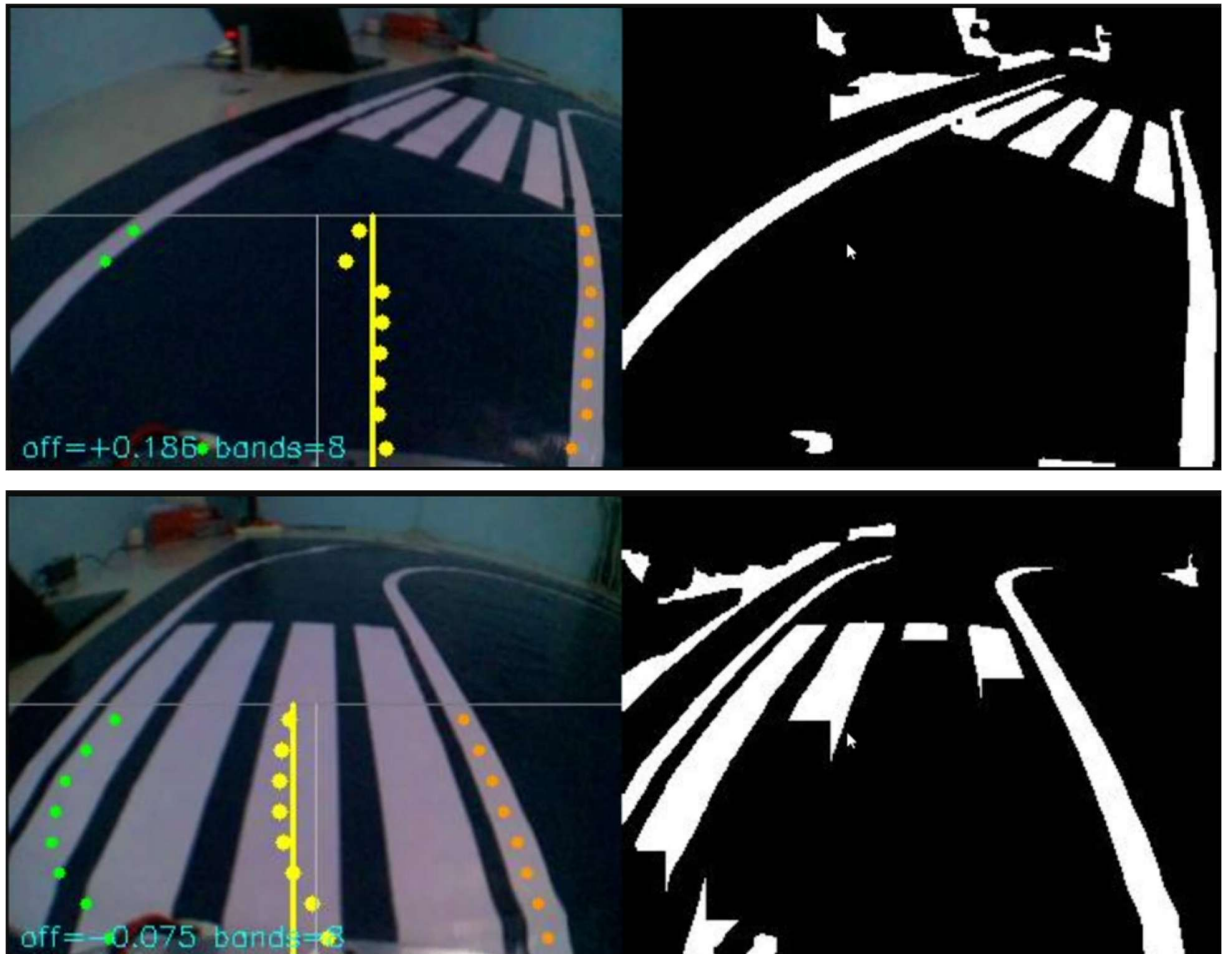


Hình 4.8. Kết quả phát hiện biển báo và đèn tín hiệu bằng YOLOv8n

Trên thực tế, mô hình nhận diện tốt khi đối tượng ở khoảng cách gần, đủ lớn trong khung hình và trong điều kiện ánh sáng ổn định như đã nêu. Do chạy trên máy tính nhúng có tài nguyên hạn chế, khối suy luận không xử lý mọi khung hình mà chỉ xử lý gián đoạn một phần số khung để không làm chậm vòng điều khiển bám làn vốn cần ưu tiên thời gian thực. Khi đối tượng ở xa hoặc nhỏ, độ tin cậy giảm và có thể bỏ sót - đây là hạn chế cố hữu của mô hình kích thước nhỏ trên phần cứng nhúng.

4.4.1. Phát hiện vạch qua đường (vạch ngựa vằn)

Ngoài mô hình học sâu, hệ thống còn dùng thuật toán thị giác truyền thống để phát hiện vạch qua đường: từ mặt nạ vùng sáng, thuật toán tìm các đường bao và đếm số sọc ngang song song; khi số sọc đạt ngưỡng tối thiểu trong một số khung liên tiếp thì xác nhận có vạch qua đường và xe giảm tốc/dừng theo kịch bản. Việc xác nhận qua nhiều khung giúp tránh báo nhầm do nhiễu tức thời.



Hình 4.9. Kết quả phát hiện vạch qua đường

4.5. Kết quả khối chấp hành

Tăng chấp hành do vi điều khiển ESP32 đảm nhiệm. ESP32 nhận lệnh điều khiển từ Raspberry Pi qua đường truyền UART và chuyển thành tín hiệu điều khiển servo lái cùng động cơ. Việc tách phần chấp hành sang một vi điều khiển riêng giúp các tác vụ thời gian thực (xuất xung điều khiển) không bị ảnh hưởng bởi tải xử lý ảnh nặng trên Pi.

4.5.1. Điều khiển góc lái servo theo cơ cấu Ackermann

Độ lệch e (công thức 4.1) được Raspberry Pi chuyển thành giá trị lệnh lái s nằm trong khoảng $[-100, +100]$ theo công thức (4.2), trong đó có một vùng chết để bỏ qua các dao động rất nhỏ quanh tâm làn và một hệ số khuếch đại quyết định độ mạnh phản ứng lái:

$$s = K \cdot |e| \cdot \text{dấu}(e), \text{ khi } |e| > \text{vùng_chết} (4.2)$$

với K là hệ số khuếch đại lái (giá trị 120 ở Bảng 4.3). Giá trị s sau đó được làm mượt bằng bộ lọc trung bình trượt rồi gửi xuống ESP32. Tại ESP32, lệnh lái s được ánh xạ thành

góc quay của servo quanh vị trí trung tâm. Do trục lái cơ khí của mô hình không nằm chính giữa, ánh xạ này được thiết kế bất đối xứng: từ vị trí trung tâm, góc bẻ sang phải và sang trái dùng hai hệ số khác nhau để bù sai lệch cơ khí, theo công thức (4.3):

$$góc_servo = góc_trung_tâm + s \cdot k_trái \quad (khi \ s \leq 0); \quad góc_trung_tâm + s \cdot k_phải \quad (khi \ s > 0) \quad (4.3)$$

Việc hiệu chỉnh điểm trung tâm và hai hệ số bẻ lái là cần thiết vì cơ cấu lái thực tế có sai lệch nhỏ: khi đặt lệnh đi thẳng, xe vẫn hơi lệch về một bên. Quá trình thử nghiệm cho thấy phải tinh chỉnh điểm trung tâm servo theo từng mô hình cụ thể để xe đi thẳng đúng hướng.

4.5.2. Điều khiển tốc độ động cơ

Tốc độ xe được điều khiển bằng độ rộng xung cấp cho cầu H L298N. Lệnh tốc độ gửi từ Pi được giới hạn trong dải an toàn để xe chạy chậm, dễ quan sát và đủ thời gian phản ứng với khúc cua cũng như biển báo. Một lỗi từng gặp là khi nhận lệnh tốc độ bằng 0, phép giới hạn lại ép tốc độ về mức tối thiểu khác 0 khiến xe vẫn bò khi đáng lẽ phải dừng; lỗi này đã được khắc phục bằng cách dừng hẳn động cơ khi lệnh tốc độ bằng hoặc nhỏ hơn 0. Hệ thống cũng có sẵn chế độ điều khiển tốc độ vòng kín bằng encoder, nhưng do tín hiệu encoder không ổn định trong điều kiện thử nghiệm nên đã chuyển sang điều khiển vòng hở để bảo đảm xe chạy ổn định.

4.5.3. Giao tiếp UART giữa Raspberry Pi và ESP32

Hai mạch trao đổi dữ liệu qua UART ở tốc độ 115200 baud bằng một giao thức bản tin dạng văn bản ngắn gọn: bản tin “F,<lái>,<tốc_độ>” yêu cầu xe tiến với góc lái và tốc độ cho trước, bản tin “S,0,0” yêu cầu dừng. Định dạng văn bản giúp dễ kiểm thử (có thể gửi lệnh trực tiếp từ dòng lệnh để kiểm tra servo và động cơ) và dễ gỡ lỗi. Đường truyền hoạt động ổn định, độ trễ giữa lệnh từ Pi và đáp ứng cơ cấu chấp hành đủ nhỏ để không ảnh hưởng tới chất lượng bám làn ở vận tốc thử nghiệm.

4.6. Kết quả tích hợp và chạy thực tế

Khi tích hợp toàn bộ các khối, xe vận hành theo vòng lặp: thu khung hình → phát hiện làn và đối tượng → quyết định góc lái, tốc độ và kích bản ứng xử → gửi lệnh xuống ESP32 → cơ cấu chấp hành thực thi. Trên sa hình trong điều kiện ánh sáng kiểm soát, xe thực hiện được chuỗi hành vi tự hành cơ bản: tự bám làn qua đoạn thẳng và khúc cua, giảm tốc khi

vào cua, phản ứng với biển báo và đèn tín hiệu khi phát hiện được, và dừng trước vạch qua đường. Bảng 4.6 tóm tắt kết quả các kịch bản chạy thực tế.

Bảng 4.6. Kết quả các kịch bản vận hành tích hợp trên sa hình

Kịch bản	Hành vi mong đợi	Kết quả
Bám làn cơ bản	Đi trọn vòng sa hình theo làn	Đạt 2-3 vòng liên tục ổn định
Vào cua	Giảm tốc, ôm vạch qua cua	Đạt với cua rộng; cua gắt còn lệch
Gặp biển dừng / đèn đỏ	Dừng xe	Đạt khi đối tượng đủ gần, rõ
Gặp đèn xanh	Tiếp tục đi	Đạt
Gặp vạch qua đường	Giảm tốc / dừng	Đạt sau xác nhận nhiều khung
Tạm mất làn	Giữ hướng, bò chậm	Đạt với đoạn mất làn ngắn





Hình 4.10. Một số khoảnh khắc xe vận hành tích hợp trên sa hình

Quá trình hoàn thiện mô hình gặp nhiều vấn đề kỹ thuật thực tế. Việc ghi nhận và xử lý các vấn đề này là một phần quan trọng của kết quả đề tài. Bảng 4.7 tổng hợp các vấn đề tiêu biểu cùng nguyên nhân và giải pháp.

Bảng 4.7. Tổng hợp các vấn đề kỹ thuật và giải pháp khắc phục

Vấn đề	Nguyên nhân	Giải pháp
Raspberry Pi tự khởi động lại	Sụt áp khi dùng chung nguồn với động cơ; mạch hạ áp không đủ dòng liên tục cho Pi (cần ~3A)	Cấp nguồn 5V/3A riêng cho Pi, tách hoàn toàn khỏi nguồn động cơ
Một động cơ không quay	Lỗi phần cứng (dây / kênh cầu H)	Kiểm tra bằng bài tự kiểm tra lúc khởi động, xử lý đấu nối
Tín hiệu encoder nhiễu	Chưa cấp nguồn / đấu nối encoder chưa ổn định	Chuyển sang điều khiển tốc độ vòng hồ để xe chạy ổn định
Xe vẫn bò khi lệnh dừng	Phép giới hạn ép tốc độ 0 về mức tối thiểu khác 0	Dừng hẳn động cơ khi tốc độ ≤ 0
Không hiển thị được màn hình từ xa	Chuyển tiếp đồ họa qua mạng không hoạt động	Phát khung hình xử lý qua giao thức truyền ảnh, xem trên trình duyệt

Nhiều trong ảnh nhị phân	Vùng sáng nhỏ rời rạc bị nhầm là vạch	Khử nhiễu bằng phép đóng hình thái + lọc theo diện tích
Xe giạt cục / lệch khi vào cua	Tinh chỉnh chồng nhiễu tham số cùng lúc gây mất ổn định	Quay về mốc chạy tốt, mỗi lần chỉ đổi một tham số
Xe lệch nhẹ khi đi thẳng	Trục lái cơ khí không nằm đúng tâm	Hiệu chỉnh điểm trung tâm servo và hệ số bề lái bất đối xứng

4.7. Đánh giá chung

Nhìn chung, mô hình đã hiện thực hóa được đầy đủ chuỗi chức năng nhận thức - quyết định - chấp hành của một xe tự hành quy mô thu nhỏ. Trong phạm vi vận hành thiết kế (sa hình cố định, ánh sáng kiểm soát), xe tự bám làn ổn định nhiều vòng, phản ứng được với biển báo, đèn tín hiệu và vạch qua đường, đồng thời hệ thống chấp hành đáp ứng đủ nhanh và mượt. Các khó khăn lớn nhất - khử nhiễu trong ảnh và ổn định bám làn ở cua gắt - đã được giảm mức đáng kể nhưng chưa triệt để. Đây là cơ sở thực nghiệm để đề ra các hạn chế và hướng phát triển ở Chương 5.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương 5 tổng kết lại những kết quả đã đạt được của đề tài, đối chiếu với mục tiêu ban đầu, chỉ ra các hạn chế còn tồn tại và đề xuất các hướng phát triển nhằm hoàn thiện mô hình trong tương lai.

5.1. Kết luận

5.1.1. Những kết quả đã đạt được

Đề tài đã thiết kế và chế tạo thành công một mô hình xe tự hành thu nhỏ hoàn chỉnh, vận hành được trên sa hình thực tế. Các kết quả chính bao gồm:

- Xây dựng được kiến trúc phân tầng rõ ràng với hai bộ xử lý: Raspberry Pi 4 đảm nhiệm nhận thức và quyết định, ESP32 đảm nhiệm chấp hành thời gian thực, giao tiếp với nhau qua UART.
- Hiện thực hoàn toàn bằng C++ khối phát hiện làn đường dựa trên thị giác máy tính, với quy trình phân ngưỡng thích nghi, khử nhiễu và quét cửa sổ trượt; xe bám làn ổn định 2-3 vòng liên tục trên sa hình.
- Khử nhiễu hiệu quả trong ảnh nhị phân bằng phép hình thái học và lọc theo diện tích, tạo đầu vào bám làn ổn định.
- Tích hợp mô hình học sâu YOLOv8n nhận diện 6 lớp đối tượng (biển dừng, biển cấm, đèn đỏ, đèn vàng, đèn xanh, người) chạy suy luận trực tiếp trên máy tính nhúng.
- Xây dựng khối chấp hành điều khiển lái theo cơ cấu Ackermann bất đối xứng (bù sai lệch cơ khí) và điều khiển tốc độ động cơ; hệ thống đáp ứng mượt ở vận tốc thử nghiệm.
- Phát triển công cụ quan sát khung hình xử lý qua mạng phục vụ gỡ lỗi khi không có màn hình gắn trực tiếp, hỗ trợ hiệu quả cho quá trình tinh chỉnh tham số.
- Chẩn đoán và khắc phục một loạt vấn đề kỹ thuật thực tế: tụt áp gây khởi động lại Pi, lỗi dừng động cơ, nhiễu encoder, hiển thị từ xa và ổn định bám làn.

5.1.2. Đối chiếu với mục tiêu đề ra

So với mục tiêu ban đầu là chế tạo một mô hình xe có khả năng tự bám làn và nhận diện được các đối tượng giao thông cơ bản, đề tài đã đạt được các mục tiêu cốt lõi trong phạm vi vận hành đã xác định. Bảng 5.1 đối chiếu mức độ hoàn thành từng mục tiêu.

Bảng 5.1. Đối chiếu mức độ hoàn thành so với mục tiêu đề ra

Mục tiêu	Mức độ đạt
Tự bám làn đường	Đạt (ổn định trong điều kiện ánh sáng kiểm soát)
Vượt khúc cua	Đạt với cua rộng; cua gắt còn hạn chế
Nhận diện biển báo, đèn tín hiệu	Đạt khi đối tượng gần và rõ
Phát hiện vạch qua đường	Đạt
Điều khiển lái và tốc độ	Đạt, vận hành mượt ở tốc độ thấp
Hoạt động đa điều kiện ánh sáng	Chưa đạt (giới hạn trong nhà, ánh sáng ổn định)

5.2. Hạn chế của đề tài

Bên cạnh các kết quả đạt được, mô hình vẫn còn một số hạn chế cần nhìn nhận khách quan:

- Phụ thuộc điều kiện ánh sáng: mô hình hoạt động ổn định trong nhà với ánh sáng đồng đều, chưa hướng tới vận hành ngoài trời.
- Ổn định ở cua gắt còn hạn chế: khi vạch ngoài ra khỏi khung hình, xe đôi lúc lệch khỏi làn.
- Bám làn dùng đặc trưng thủ công: bộ tham số phải tinh chỉnh thủ công và gắn với sa hình cụ thể, khả năng tổng quát sang môi trường khác còn thấp.
- Mô hình học sâu nhỏ trên phần cứng nhúng: tốc độ suy luận hạn chế nên chỉ xử lý gián đoạn một phần khung hình, dễ bỏ sót đối tượng ở xa hoặc nhỏ.
- Chưa có điều khiển tốc độ vòng kín hoạt động ổn định do tín hiệu encoder chưa tin cậy; xe đang chạy điều khiển vòng hở.
- Sai lệch cơ khí của cơ cấu lái phải bù bằng phần mềm, cần hiệu chỉnh lại khi thay đổi phần cứng.

5.3. Hướng phát triển

Trên cơ sở các hạn chế đã nêu, đề tài có thể được mở rộng và hoàn thiện theo các hướng sau:

- Tăng khả năng thích nghi với ánh sáng: bổ sung tự động cân bằng độ sáng, cân bằng trắng, hoặc dùng phương pháp phát hiện làn bằng học sâu (phân đoạn làn) để giảm phụ thuộc vào ngưỡng sáng cố định, tiến tới vận hành ngoài trời.
- Nâng cao độ ổn định ở cua gắt: cải tiến thuật toán bám vạch (mô hình hóa đường cong, dự đoán quỹ đạo), bố trí camera tầm nhìn rộng hơn hoặc thêm cảm biến hỗ trợ.

- Bổ sung cảm biến: thêm cảm biến siêu âm hoặc đo xa để phát hiện vật cản và giữ khoảng cách, tiến tới các chức năng an toàn ở cấp độ cao hơn.
- Khôi phục điều khiển tốc độ vòng kín: hoàn thiện đầu nối và lọc tín hiệu encoder để dùng bộ điều khiển PID giữ tốc độ ổn định trên dốc và tải thay đổi.
- Tối ưu mô hình học sâu: lượng tử hóa mô hình, dùng bộ tăng tốc suy luận hoặc mô hình gọn hơn để tăng tốc độ và độ chính xác nhận diện trên phần cứng nhúng.
- Mở rộng tập biển báo và kịch bản giao thông: bổ sung thêm lớp đối tượng, xử lý các tình huống phức tạp hơn (giao lộ, rẽ theo biển chỉ hướng).
- Tự động hiệu chỉnh cơ cấu lái: xây dựng quy trình hiệu chỉnh điểm trung tâm servo bán tự động để giảm công sức tinh chỉnh khi thay đổi phần cứng.

5.4. Lời kết

Đề tài đã hoàn thành việc xây dựng một mô hình xe tự hành thu nhỏ vận hành được trên thực tế, với đầy đủ chuỗi chức năng nhận thức - quyết định - chấp hành. Quá trình thực hiện không chỉ mang lại một sản phẩm chạy được mà còn giúp người thực hiện tích lũy kinh nghiệm thực tiễn quý báu về thị giác máy tính, học sâu, hệ thống nhúng, điều khiển và đặc biệt là kỹ năng chẩn đoán - xử lý sự cố trên hệ thống thật. Những hạn chế còn lại cùng các hướng phát triển đã đề xuất là tiền đề để tiếp tục hoàn thiện mô hình thành một hệ thống ổn định và linh hoạt hơn trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] World Health Organization, "Global status report on road safety 2018," World Health Organization, Geneva, Switzerland, 2018.
- [2] S. Karaman and et al., "Project-based, collaborative, algorithmic robotics for high school students: Programming self-driving race cars at MIT," in Proc. IEEE Int. Conf. on Robotics and Automation (ICRA), Singapore, 2017.
- [3] Amazon Web Services, "AWS DeepRacer: Reinforcement learning driven autonomous scale vehicle," 2019. [Online]. Available: <https://docs.aws.amazon.com/deepracer/>.
- [4] Y. S. Dube, R. R. Shinde, and A. I. Chavhan, "Deep Learning-Based Autonomous Driving System with OpenCV Integration," International Journal on Science and Technology (IJSAT), vol. 16, no. 2, pp. 1-9, Apr.-Jun. 2025.
- [5] B. Dhakal, A. Mallik, S. Khanal, S. Tamang, G. Gautam, and S. Nakarmi, "End-to-End Steering Prediction and Object Detection for Self-Driving Car using Machine Learning," KEC Journal of Science and Engineering, vol. 9, no. 1, pp. 86-94, 2025.
- [6] K. Alawaji, R. Hedjar, and M. Zuair, "Traffic Sign Recognition Using Multi-Task Deep Learning for Self-Driving Vehicles," Sensors, vol. 24, no. 11, art. 3282, 2024.
- [7] V. H. Du and T. V. Đạt, "Nghiên cứu và phát triển mô hình xe tự hành ứng dụng AI," Khoa Điện - Điện tử, Trường Đại học Sư phạm Kỹ thuật TP.HCM, TP.HCM, 2024.
- [8] Bộ Giao thông vận tải Việt Nam, "Quy chuẩn kỹ thuật quốc gia về báo hiệu đường bộ, QCVN 41:2019/BGTVT," Bộ Giao thông vận tải Việt Nam, 2019.
- [9] S. International, "Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles," SAE International, Standard J3016_202104, 2021.
- [10] R. Siegwart, I. R. Nourbakhsh and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., Cambridge, MA: MIT Press, 2011.
- [11] Raspberry Pi Foundation, "Raspberry Pi 4 Model B product brief," Raspberry Pi Foundation, Educational Datasheet, Cambridge, 2019.

- [12] Espressif Systems, "ESP32 series datasheet," Espressif Systems, Technical Report, v4.3, Shanghai, 2022.
- [13] Espressif Systems, "ESP-IDF programming guide: FreeRTOS architecture," 2023. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/>. [Accessed 02 Apr 2026].
- [14] OmniVision Technologies, "OV5647 Color CMOS 5 Megapixel (2592x1944) Image Sensor with OmniBSI Technology," Santa Clara, CA, USA, 2015.
- [15] STMicroelectronics, "L298 Dual Full-Bridge Driver Datasheet (DocID1773 Rev 9)," Geneva, Switzerland, 2000.
- [16] Tower Pro, "MG955 Metal Gear Servo Datasheet," Shenzhen, China, n.d.
- [17] R. W. Erickson and D. Maksimović, Fundamentals of Power Electronics, 3rd ed., Cham, Switzerland: Springer, 2020.
- [18] T. Noergaard, Embedded Systems Architecture: A Comprehensive Guide for Engineers and Programmers, 2nd ed., Oxford, UK: Newnes (Elsevier), 2012.
- [19] Espressif Systems, "ESP32 Technical Reference Manual (v5.1)," Shanghai, China, 2023.
- [20] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed., New York, NY, USA: Pearson, 2018.
- [21] A. R. Smith, "Color gamut transform pairs," ACM SIGGRAPH Computer Graphics, vol. 12, no. 3, pp. 12-19, 1978.
- [22] OpenCV, "Changing Colorspaces - OpenCV 4.x Documentation," 2024. [Online]. Available: https://docs.opencv.org/4.x/d8/d01/group__imgproc__color__conversions.html.
- [23] OpenCV, "Changing the contrast and brightness of an image!," OpenCV Documentation, [Online]. Available: [\[https://docs.opencv.org/4.x/d3/dc1/tutorial_basic_linear_transform.html\]](https://docs.opencv.org/4.x/d3/dc1/tutorial_basic_linear_transform.html)(https://docs.opencv.org/4.x/d3/dc1/tutorial_basic_linear_transform.html).

- [24] OpenCV, "Smoothing Images," OpenCV Documentation. [Online]. Available: https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html.
- [25] R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed., Cham: Springer, 2022.
- [26] OpenCV, "Morphological Transformations," OpenCV Documentation. [Online]. Available: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html.
- [27] J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679-698, 1986.
- [28] P. V. C. Hough, "Method and means for recognizing complex patterns," U.S. Patent 3,069,654, Dec. 18, 1962.
- [29] J. Matas, C. Galambos, and J. Kittler, "Robust detection of lines using the progressive probabilistic Hough transform," *Computer Vision and Image Understanding*, vol. 78, no. 1, pp. 119-137, 2000.
- [30] S. Suzuki and K. Abe, "Topological structural analysis of digitized binary images by border following," *Computer Vision, Graphics, and Image Processing*, vol. 30, no. 1, pp. 32-46, 1985.
- [31] K. Ogata, Modern Control Engineering, 5th ed., Boston, MA, USA: Pearson, 2010.
- [32] R. C. Coulter, "Implementation of the Pure Pursuit Path Tracking Algorithm," Carnegie Mellon University, Robotics Institute, Tech. Report CMU-RI-TR-92-01, Jan. 1992.
- [33] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, 1998.
- [34] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 580-587.

- [35] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779-788.
- [36] G. Jocher, A. Chaurasia, and J. Qiu, "YOLO by Ultralytics," Ultralytics, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [37] S. J. Pan and Q. Yang, "A survey on transfer learning," IEEE Transactions on Knowledge and Data Engineering, vol. 22, no. 10, pp. 1345-1359, 2010.
- [38] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," arXiv:2004.10934, 2020.
- [39] ONNX, "Open Neural Network Exchange." [Online]. Available: <https://onnx.ai/>.
- [40] Ultralytics, "Thresholding in Image Processing," *Ultralytics Blog*. [Online]. Available: <https://www.ultralytics.com/vi/blog/thresholding-in-image-processing>. [Accessed: Jun. 24, 2026].